Predicción del tamaño de circuitos mínimos a través de la repetitividad de funciones booleanas

Prediction of minimum circuit size through repetitiveness of Boolean functions



Trabajo de Fin de Grado
Curso 2025–2026

Autor
Juan Diego Barrado Daganzo

Directores
Ismael Rodríguez Laguna
Narciso Martí Oliet

Calificación: 10

Doble Grado en Ingeniería Informática y Matemáticas
Facultad de Informática
Universidad Complutense de Madrid

10 de junio de 2026

A mi profesor, Pedro Recuenco.

Agradecimientos

A Ismael y a Narciso por su tiempo y su disponibilidad. Ha sido un gusto poder disfrutar del broche final a mi paso por la universidad compartiéndolo con personas tan entregadas e ilusionadas por esta ciencia. A Celia, por aguantarme en mis explicaciones interminables y por vivir con mi misma ilusión todo el proceso. A mis padres, por dedicar sus vidas siempre a construir la mía. Y a Dios, por darme la oportunidad de vivir con tanto entusiasmo este proyecto.

Resumen

Predicción del tamaño de circuitos mínimos a través de la repetitividad de funciones booleanas

El problema de P vs NP constituye el desafío abierto más importante de la informática teórica, siendo el estudio de la complejidad de circuitos booleanos una de las estrategias más prometedoras para separar ambas clases. Se analiza la problemática desde dos perspectivas complementarias: una revisión teórica de la demostración de cotas inferiores al tamaño de los circuitos mediante el razonamiento directo sobre su topología, y una propuesta metodológica experimental basada en medidas de complejidad que analizan propiedades vinculadas al tamaño de los mismos sin fundamentarse directamente en su estructura topológica.

En la vertiente teórica, se presenta una exposición didáctica de las mejores cotas inferiores demostradas hasta la fecha para el tamaño de circuitos sobre la base B_2. Se analiza en detalle la técnica de eliminación de puertas aplicada a la familia de los affine dispersers, recorriendo la evolución de resultados desde la cota inicial de $3n - o(n)$ hasta el actual máximo de $3{,}1n - o(n)$.

En la vertiente experimental, se investiga si ciertas medidas de complejidad basadas en la repetitividad de la tabla de verdad permiten caracterizar el tamaño del circuito mínimo. Para ello, se proponen y evalúan métricas basadas en la entropía de Shannon, el número de clasificaciones y, especialmente, lo que definimos como Forma Normal de Fourier (FNF). Los resultados en funciones de 5 bits muestran que la métrica asociada a la FNF mantiene una correlación significativa $(0, 4463)$ con el tamaño del circuito mínimo, respetando —en media— la monotonía en el tamaño del circuito mínimo.

Finalmente, se discuten las limitaciones de estas aproximaciones, analizando el marco de las demostraciones naturales de Razborov y Rudich y los límites de la técnica de eliminación de puertas, concluyendo que la métrica FNF ofrece una vía de investigación prometedora al evitar las barreras de naturalización.

Palabras clave

Affine dispersers, complejidad de circuitos, demostraciones naturales, forma normal de Fourier, funciones booleanas, problema P vs NP, tabla de verdad, técnica de eliminación de puertas, tamaño de circuito mínimo.

Abstract

Prediction of minimum circuit size through repetitiveness of Boolean functions

The P vs NP problem constitutes the most important open challenge in theoretical computer science, with the study of boolean circuit complexity being one of the most promising strategies to separate both classes. This problem is analyzed from two complementary perspectives: a theoretical review of the proof of lower bounds on circuit size through direct reasoning about their topology, and an experimental methodological proposal based on complexity measures that analyze properties linked to circuit size without being directly grounded in their topological structure.

On the theoretical side, we present a didactic exposition of the best lower bounds demonstrated to date for circuit size over the B_2 basis. The gate elimination technique applied to the family of affine dispersers is analyzed in detail, tracing the evolution of results from the initial $3n - o(n)$ bound to the current maximum of $3.1n - o(n)$.

On the experimental side, we investigate whether certain complexity measures based on truth table repetitiveness allow us to characterize the size of the minimal circuit. To this end, metrics based on Shannon entropy, the number of classifications, and, in particular, what we define as the Fourier Normal Form (FNF) are proposed and evaluated. The results for 5-bit functions show that the metric associated with the FNF maintains a significant correlation (0.4463) with the minimal circuit size, respecting —on average— the monotony of the minimal circuit size.

Finally, the limitations of these approaches are discussed by analyzing Razborov and Rudich's *natural proofs* framework and the limits of the gate elimination technique, concluding that the FNF metric offers a promising line of research by avoiding naturalization barriers.

Keywords

Affine dispersers, boolean functions, circuit complexity, Fourier Normal Form, gate elimination technique, minimum circuit size, natural proofs, P vs NP problem, truth table.

Índice general

1. Introducción **1**

1.1. Antecedentes y estado de la cuestión 2

1.1.1. El problema de P vs NP 2

1.1.2. La clase $P/poly$ y la complejidad de circuitos 3

1.2. Objetivos y plan de trabajo 4

2. Cotas inferiores para el tamaño de circuitos booleanos **7**

2.1. Preliminares: marco formal de los circuitos booleanos 8

2.2. Cota de $3n - o(n)$ para los *affine dispersers* 11

2.2.1. Definición de *affine disperser* 11

2.2.2. Generalización del teorema de eliminación de puertas 12

2.2.3. Procedimiento de búsqueda de las restricciones afines 14

2.2.4. Demostración del resultado principal 16

2.3. Cota de $(3 + 1/86)\, n - o(n)$ para los *affine dispersers* 18

2.3.1. Intuición general de la prueba 19

2.3.2. Sustituciones afines generalizadas 22

2.3.3. Normalización de circuitos 27

2.3.4. Sustituciones cuadráticas 29

2.3.5. Demostración del resultado principal 32

2.4. Cota de $3{,}1n - o(n)$ para los *affine dispersers* 38

2.4.1. Análisis fino de las *troubled gates* en las reglas de normalización 38

2.4.2. Demostración del resultado principal 41

3. Estudio experimental **43**

3.1. Repetición de patrones 45
3.1.1. Entropía de Shannon 45
3.1.2. Número de clasificaciones 47
3.2. Solapamiento de patrones 49
3.2.1. Forma normal de Fourier 49
3.2.2. Forma normal de Fourier generalizada 54

4. Limitaciones y restricciones **59**
4.1. Demostraciones naturales 60
4.2. Límite de la técnica de eliminación de puertas 63

5. Conclusiones y trabajo futuro **67**
5.1. Conclusiones 67
5.2. Trabajo futuro 68

Introduction **69**

Conclusions and Future Work **73**

Bibliografía **75**

A. Artefactos adicionales **77**

Índice de figuras

2.1. Ejemplo de circuito como grafo acíclico según la Definición 2.6. 9

2.2. Ejemplo de [6] para mostrar la aplicación del Lema 2.18. El subcircuito que computa P se simplifica y alimenta todas las entradas donde x_1 se insertase. La puerta P se elimina y se inserta la constante de la sustitución en todas aquellas puertas que alimentase. . . 16

2.3. Representación visual de la distinción de casos de la demostración del Teorema 2.19. . 18

2.4. Representación gráfica de la distinción de casos de la intuición de la prueba del $(3 + {}^{1}/_{86})n - o(n)$. 21

2.5. Representación gráfica de la estructura local de una *troubled gate* P. 22

2.6. Circuito cíclico consistente dado como ejemplo en [7] junto a las funciones booleanas de cada una de sus puertas. 23

2.7. Sistema lineal asociado al circuito cíclico de la Figura 2.6. El rango de la matriz de coeficientes muestra que el circuito es consistente. 24

2.8. Representación visual dada en [7] para aclarar la transformación del circuito descrita en la demostración de la Proposición 2.28. 26

2.9. Representación visual recogida en [7] de las reglas de normalización. 28

2.10. Ejemplo en [7] que muestra una situación en la que una sustitución cuadrática es la única forma de disminuir la medida lo suficiente. 29

2.11. Notaciones del caso 5 en la distinción de casos de la demostración del Teorema 2.39. . 35

2.12. Notaciones del caso 5.b) en la distinción de casos de la demostración del Teorema 2.39. 35

2.13. Ejemplo de la estructura local denominada *troubled pair*. 39

3.1. Descomposición recursiva de la tabla de verdad de la función paridad en términos de sus mitades. Los colores rojo y azul atenuados hacen evidente la relación de negación entre una submitad y otra dentro de cada mitad. 48

3.2. Ejemplo de la diferencia de la topología exigida a la estructura sintáctica de la combinación de armónicos booleanos: (a) la rigidez del árbol sintáctico fuerza la repetición de subestructuras, mientras que (b) el grafo dirigido acíclico (DAG) permite la reutilización de recursos propia de un circuito. 51

3.3. Comparación de los histogramas normalizados de la medida de complejidad de la Definición 3.18 por clase de tamaño. 55

3.4. Comparación de los parámetros de las distribuciones normales asociadas a los datos de la medida de complejidad de la Definición 3.18 por clase de tamaño. 56

4.1. Representación gráfica de las transformaciones de la parte relevante del circuito entre C, C' y C'' recogida en [9]. 65

Índice de tablas

3.1. Parámetros de la distribución normal aparente en la medida de complejidad de la Definición 3.18 por clase. 55

A.1. Clasificación de las 16 puertas lógicas en las clases $\odot$-puertas, $\oplus$-puertas y $\wedge$-puertas. . 77

A.2. Transformación de cada una de las 16 puertas en una puerta de $\odot$-puertas según la sustitución constante hecha sobre sus entradas. 78

A.3. Trivialización de las $\oplus$-puertas según la sustitución afín simple hecha sobre sus entradas. 78

Capítulo 1

Introducción

El problema de P vs NP está catalogado como uno de los problemas del milenio y no es para menos: no solo es el problema abierto más importante de la informática teórica, sino que cuestiona la naturaleza misma de la resolución de problemas. La pregunta central —"¿es intrínsecamente más difícil encontrar una solución que verificarla?"— puede parecer sencilla, pero ha permanecido sin respuesta desde su formulación en los años 70. Más aún, precisamente este problema ha definido los límites de lo que consideramos factible computacionalmente o no. De probarse $P = NP$ (con una demostración constructiva y bajo la suposición estándar de que eficientemente significa resoluble en tiempo polinómico), multitud de problemas como el plegado de proteínas o la demostración automática de teoremas quedarían resueltos de manera eficiente y transformarían nuestra sociedad al permitir avances sin precedentes en casi todas las áreas de conocimiento en las que la complejidad es un obstáculo.

No obstante, la intuición y la experiencia sugieren que la respuesta es $P \neq NP$ —aunque nadie ha sido capaz de probarlo formalmente—, por lo que la mayor parte del mundo académico dirige su trabajo en esa dirección. De entre las distintas estrategias que se han empleado, en los últimos años los esfuerzos se han centrado en tratar de separar las clases P y NP a través de la complejidad de sus circuitos booleanos. El argumento teórico se fundamenta en la clase $P/poly$, compuesta por todos los problemas de decisión computables por circuitos de tamaño polinómico, y consiste en probar que existe una función booleana en NP cuyo circuito mínimo sea de tamaño superpolinómico. Si bien es una de las estrategias más prometedoras, también es de las más esquivas. Sabemos que la mayor parte de las funciones booleanas no se pueden computar con circuitos polinómicos, pero paradójicamente no hemos sido capaces de encontrar ninguna, no hablemos ya de encontrar específicamente una de NP.

En ese contexto, tiene especial interés estudiar cuáles son las mejores cotas inferiores a la complejidad de los circuitos booleanos que hemos conseguido probar y para qué funciones lo hemos hecho. Por un lado, esto nos va a permitir comprender cuál es la herramienta más potente que tenemos para probar estas cotas y cómo se utiliza, esto es, la técnica de eliminación de puertas. Por otro, podremos observar en primera línea las dificultades que se presentan casi desde un inicio para traspasar la barrera de la linealidad.

Por nuestra parte, abordamos el problema desde una metodología distinta. En lugar de probar cotas inferiores en el tamaño de los circuitos, tratamos de estudiar experimentalmente si ciertas medidas de complejidad permiten caracterizar la complejidad del circuito mínimo que computa a una función booleana. En particular, definimos estas métricas en base a las repetitividades (evidentes o subyacentes) de la tabla de verdad.

Por último, tratamos de dirigir los esfuerzos de los futuros intentos por resolver el problema exponiendo los principales resultados sobre las limitaciones de la técnica de eliminación de puertas y de las características de las medidas de complejidad. Por un lado, presentamos funciones resistentes a la técnica de eliminación de puertas para descartar demostraciones que no tengan en cuenta las particularidades de la familia de funciones con las que se trabaja. Por otro, presentamos el resultado fundamental de las demostraciones naturales para justificar por qué es posible descartar ciertas medidas de complejidad antes incluso de analizar cualquiera de sus resultados experimentales. Por último, discutimos el potencial de mejora de las medidas de complejidad propias y proponemos líneas de investigación futuras.

1.1. Antecedentes y estado de la cuestión

A mediados del siglo XX, la definición de computabilidad (gracias a Church y Turing) ya había diferenciado formalmente los problemas que podían ser resueltos por un computador de los que no: había nacido el término de "indecidibilidad". No obstante, que un problema sea resoluble no implica que su solución pueda encontrarse en un tiempo razonable. Conocemos problemas resolubles en tiempo polinómico y problemas únicamente resolubles en tiempo exponencial; sin embargo, el verdadero interrogante lo plantean aquellos problemas cuyas soluciones son fáciles de verificar (i.e., en tiempo polinómico), pero para los que todavía no hemos hallado una forma eficiente de encontrarlas.

1.1.1. El problema de P vs NP

La clase P (Polynomial Time) comprende todos aquellos problemas de decisión que pueden ser resueltos por una máquina de Turing determinista en un número de pasos cuyo orden de complejidad es $O(n^k)$ —para algún k— donde n es el tamaño de la entrada. Por ejemplo, determinar si un listado de n elementos está ordenado es un problema de decisión de esta clase. La clase NP, por su parte, no significa "No Polinómico", sino Nondeterministic Polynomial Time. Esta clase agrupa problemas para los cuales, dada una posible solución, esta puede ser verificada en un número de pasos de orden polinómico por una máquina de Turing determinista[1]. Un ejemplo clásico es el problema del Sudoku: resolver un Sudoku de $n \times n$ celdas es extremadamente difícil a medida que n crece, pero si alguien nos entrega un tablero ya rellenado, verificar que no hay números repetidos en filas, columnas o subcuadrículas es una tarea trivial y rápida (polinómica). Es evidente que $P \subset NP$, así que la gran pregunta consiste en saber si $P = NP$ o por el contrario $P \subsetneq NP$.

En 1971, Stephen A. Cook introdujo en [4] la clase de problemas *NP*, el concepto de la *NP-completitud* y la noción de reducibilidad polinómica. Casi simultáneamente y de forma independiente, en la URSS, Leonid A. Levin llegó a conclusiones similares en [14], razón por la cual hoy conocemos al pilar de esta teoría como el Teorema de Cook-Levin. Este teorema demostró que el problema de satisfacibilidad booleana (SAT) es "tan difícil como cualquier otro" de la clase NP: si SAT puede resolverse eficientemente, entonces todos los problemas de NP también. Esta propiedad es la que se conoce como NP-completitud y desde su definición se han ido encontrando multitud de problemas que también son NP-completos.

El Teorema de Cook-Levin tiene una conclusión evidente. Si un problema NP-completo está en P, todo $NP \subset P$ y quedaría demostrado que $P = NP$. Por tanto, la pregunta es evidente: ¿podrá

[1] En realidad, la definición formal de la clase consiste en todos aquellos problemas que pueden ser resueltos en un número de pasos polinómico por una máquina de Turing no determinista, pero esta es una definición equivalente y más intuitiva para la exposición del capítulo.

resolverse SAT (o cualquier otro problema NP-completo) en tiempo polinómico? Este fue el germen del problema general que hoy conocemos como P vs NP y que consiste en probar o desmentir la afirmación $P = NP$ a través del estudio de $NP \subset P$, es decir, dilucidar si verificar una solución es tan difícil computacionalmente como encontrarla. De ser cierta, la conjetura implicaría que cualquier problema cuya solución podamos verificar rápidamente también posee un algoritmo oculto para ser resuelto rápidamente. No obstante, la mayor parte de la comunidad científica se inclina por la opción $P \neq NP$ basándose en que, tras décadas de búsqueda intensiva, no se han encontrado algoritmos polinómicos para los problemas más difíciles de NP.

En el año 2000, el Clay Mathematics Institute elevó el estatus de este problema al incluirlo entre los siete Millennium Prize Problems, otorgando una dotación de un millón de dólares a quien logre una demostración formal de $P = NP$ o $P \neq NP$. El motivo es que su resolución —aparte de ser difícil— podría tener un gran impacto más allá incluso de las matemáticas. Por un lado, si se demostrara que $P = NP$ (bajo las condiciones que indicamos antes), el mundo tal como lo conocemos cambiaría radicalmente. La mayoría de los sistemas de criptografía de clave pública (como RSA o el protocolo Diffie-Hellman) se basan en la asimetría entre la facilidad de multiplicar dos números primos (en P) y la dificultad de factorizar el resultado (en NP), por lo que de ser cierto $P = NP$ las conexiones dejarían de ser seguras. Por otro lado, la optimización logística, el diseño de fármacos y la demostración automática de teoremas matemáticos se volverían tareas triviales, acelerando el progreso científico y el desarrollo tecnológico de forma exponencial. Por otro, si se demostrara que $P \neq NP$, se confirmaría que la intuición humana es correcta: encontrar una solución es más difícil que juzgarla, y resolver un puzzle es intrínsecamente más complejo que comprobar que está bien hecho. Desde el punto de vista técnico, validaría la seguridad de nuestros sistemas actuales y permitiría a la comunidad centrarse en algoritmos de aproximación y heurísticas, sabiendo que la solución óptima exacta es inalcanzable para grandes volúmenes de datos.

1.1.2. La clase $P/poly$ y la complejidad de circuitos

En los esfuerzos de las últimas décadas ha aparecido una tercera clase de complejidad para tratar de resolver el problema: la clase $P/poly$. La clase $P/poly$ se define como el conjunto de lenguajes que pueden ser decididos por una familia de circuitos booleanos $\{C_n\}_{n\in\mathbb{N}}$ de tamaño polinómico. A diferencia de las clases de complejidad estándar, $P/poly$ representa un modelo de computación no uniforme. En un modelo uniforme, el reconocedor de un lenguaje debe reconocer todas las cadenas del mismo sin importar el tamaño n de estas; en cambio, en un modelo no uniforme, permitimos una familia de reconocedores —posiblemente distintos— para cada tamaño de entrada n. Por ejemplo, las expresiones regulares son un modelo de cómputo uniforme, pues el autómata finito no cambia entre instancias del lenguaje; sin embargo, los circuitos booleanos no son un modelo de cómputo uniforme, pues cuando cambia el número de entradas cambia también el circuito usado para computar las mismas funciones booleanas.

Mientras que analizar la complejidad computacional en una máquina de Turing requiere analizar el rastro de la ejecución en ella, el modelo de cómputo de los circuitos booleanos es estático y combinatorio: podemos contar sus puertas, medir su profundidad y analizar su topología como grafo acíclico. Por ello, a pesar de que el modelo de la máquina de Turing es el estándar teórico para definir la computabilidad, la comunidad científica ha virado hacia intentos por medir la complejidad computacional en este otro tipo de modelo de cómputo.

El razonamiento para tratar de demostrar $P \neq NP$ haciendo uso de $P/poly$ es el siguiente. Sabemos que[2] es un problema en $P/poly$, pero indecidible, luego no puede estar en P. $P \subset P/poly$, pues cualquier problema computable por una máquina de Turing en tiempo polinómico puede computarse por una familia de circuitos polinómicos. Si consiguiésemos encontrar un problema en NP que no estuviese en $P/poly$, necesariamente ese problema no podría estar en P, así que habríamos demostrado que $P \neq NP$. En consecuencia, los intentos por demostrar P vs NP han pasado por tratar de buscar una función booleana de n bits en 1 cuyo problema de decisión asociado esté en la clase NP y cuyo circuito mínimo requiera un tamaño superpolinómico de puertas.

Por tanto, la hoja de ruta de la demostración queda clara: encontrar un problema de decisión en NP que no sea computable por circuitos polinómicos. Lamentablemente, incluso en estas circunstancias el problema sigue siendo extremadamente difícil. De hecho, las dificultades que se siguen encontrando son más que sorprendentes. En base a los argumentos de conteo de Shannon [19], sabemos que casi la totalidad de las funciones booleanas requieren de un circuito de tamaño superpolinómico (concretamente $2^n/n$) para un n suficientemente grande; paradójicamente, no hemos sido capaces de encontrar ninguna. Este hecho pone en evidencia que la dificultad no es siquiera encontrar una función booleana cuyo problema de decisión asociado esté en NP y solo sea computable por circuitos superpolinómicos, sino encontrar funciones cualesquiera que sean "suficientemente complejas". Este es el motivo por el que en los últimos años también ha habido grandes esfuerzos en buscar cotas inferiores en el tamaño de los circuitos para ciertas funciones booleanas, independientemente de si su problema asociado está en NP o no.

1.2. Objetivos y plan de trabajo

El horizonte ideal que perseguimos es probar que efectivamente $P \neq NP$ y hacerlo a través de la complejidad de los circuitos booleanos, es decir, encontrar un problema de decisión en NP cuya familia de funciones booleanas no sea computable por circuitos polinómicos. Para ello, debemos ser capaces de demostrar cotas inferiores para funciones booleanas en el tamaño de sus circuitos. La herramienta principal para este propósito será proponer una medida de complejidad $\mu : \mathcal{F}_n \to \mathbb{R}$ que ordene de la misma manera que la complejidad del circuito mínimo el conjunto de funciones booleanas y permita probar más fácilmente qué funciones son "pequeñas" y cuáles "grandes". Después, bastaría con encontrar un problema en NP cuyas funciones sean "grandes" en base a nuestra medida de complejidad.

Este horizonte es claramente muy ambicioso y sirve para dirigir nuestros pasos, pero necesitamos establecer metas más sencillas a lo largo del camino que permitan hacer de punto de control en nuestros avances. Por tanto, los objetivos que perseguiremos, en orden, son los siguientes:

1. Comprender las demostraciones de las mejores cotas inferiores en el tamaño de los circuitos booleanos probadas hasta el momento y presentar una exposición didáctica de los contenidos.

2. Proponer medidas de complejidad propias basadas en las repetitividades evidentes o subyacentes que presenta la tabla de verdad de las funciones booleanas.

3. Construir una infraestructura software que permita estudiar experimentalmente estas medidas y que permita hacer conjeturas sobre su capacidad de demostrar cotas inferiores para el tamaño de circuitos booleanos.

[2]Cabe destacar que la inclusión es estricta. El problema de parada para máquinas de Turing y cadenas definidas en unario

4. Presentar el estudio experimental y, o bien probar formalmente por qué las métricas no sirven para probar cotas inferiores, o bien conjeturar por qué si lo hacen y proponer mejoras.

Los objetivos describen perfectamente la hoja de ruta que se ha seguido a lo largo del trabajo. El Capítulo 2 muestra la exposición didáctica de la sucesión de artículos que prueban la mejor cota inferior encontrada hasta el momento y la familia de funciones para la que se prueba. El Capítulo 3 presenta las medidas de complejidad propias, la infraestructura software construida y el estudio experimental asociado a cada una de ellas. Durante el desarrollo de estos dos capítulos, se hicieron evidentes ciertas limitaciones asociadas tanto a las demostraciones de las cotas inferiores como a las medidas de complejidad propuestas. Por este motivo, se incluye adicionalmente el Capítulo 4 para estudiar las limitaciones y restricciones de ambas aproximaciones y se prueban resultados para dirigir los futuros intentos de demostración por uno u otro método. Por último, el Capítulo 5 propone distintas líneas de trabajo prometedoras para continuar con los desarrollos más exitosos de este documento.

Capítulo 2

Cotas inferiores para el tamaño de circuitos booleanos

A pesar de los avances en la búsqueda de cotas inferiores para el tamaño de circuitos booleanos, el problema de encontrar una función explícita que se compute necesariamente a través de un circuito "grande" sigue siendo un problema abierto —y muy díficil—. Los esfuerzos por encontrar estas cotas inferiores para ciertas familias de funciones nos han dejado una sucesión de resultados que ponen de manifiesto la dificultad del problema y revelan lo lejos que aún estamos de lograr el objetivo final.

La primera cota obtenida con este propósito fue $2n - O(1)$ y se probó en primer lugar por Kloss y Malyshev en [12], para la función $\oplus_{i,j}(x_i \wedge x_j)$, y en segunda instancia por Schnorr en [18]. Posteriormente, Stockmeyer elevó la cota hasta $2{,}5n - O(1)$ para ciertas funciones simétricas [21] y en el mismo año un resultado alternativo de Paul dio la cota $2{,}5n - o(n)$ (ligeramente peor) para la función "*storage*" en [16]. Sería precisamente esta última la que sería mejorada por Blum para obtener por primera vez la cota $3n - o(n)$ en [3]. De nuevo, un resultado alternativo y completamente independiente alcanzaría esta misma cota por parte de [6] y daría lugar a la cadena de resultados progresivos que culminan con la que hasta ahora es la mejor cota inferior para circuitos booleanos sobre B_2, y que presentamos en este capítulo. Esta cadena comienza con la prueba alternativa para $3n - o(n)$ en [6], donde sientan las bases del razonamiento general, sigue con la mejora a $\left(3 + \frac{1}{86}\right) n - o(n)$ en [7], que contiene la nueva perspectiva principal del mismo resultado para poder conseguir esta mejora y, por último, finaliza con el resultado de $3{,}1n - o(n)$ en [15], donde se refinan levemente algunos detalles de la prueba anterior para llegar al máximo alcanzado hasta ahora.

Esta sucesión de artículos académicos evidencia cuán lejos estamos todavía de una cota suficientemente elevada como para encontrar explícitamente una de estas funciones "grandes" que sabemos que deben existir. Más aún, ponen de relieve que la dificultad está no ya en encontrar una cota exponencial, sino incluso en romper la barrera de la linealidad.

Hasta el momento, casi la totalidad de los intentos por probar cotas inferiores para el tamaño de circuitos booleanos se basan en la técnica de eliminación de puertas. Esta técnica consiste en demostrar que dado un número $r(n)$ de restricciones, imponer cada una de estas elimina un cierto número de puertas δ del circuito sin trivializarlo. De esta manera, el circuito original sin restricciones debía tener al menos un número $\delta \cdot r(n)$ de puertas para que el argumento anterior fuese cierto y, en consecuencia, el valor $r(n) \cdot \delta$ es una cota inferior del tamaño mínimo del circuito.

Teorema 2.1. *Sean $f : \mathbb{F}_2^n \to \mathbb{F}_2$ y C un circuito que la computa, supongamos que hay $r(n)$ restricciones sobre las variables de entrada que no trivializan la función $f|_{r(n)}$ y cada una de ellas permite eliminar δ puertas del circuito C' que computa $f|_{r(n)}$, entonces el circuito debía tener al menos $\delta \cdot r(n)$ puertas para computar f.*

La demostración requiere un análisis más detallado, pero la idea es intuitiva: si añadir una restricción en un circuito provoca la eliminación de δ puertas sin trivializar la función f que computa, entonces el circuito debía tener al menos δ puertas inicialmente para que esto sea posible. Veamos un ejemplo sencillo de cómo aplicar la técnica.

Ejemplo 2.2. Supongamos que queremos demostrar que la función paridad $x_1 \oplus \cdots \oplus x_n$ debe ser computada por un circuito de al menos $n-1$ puertas.

En primer lugar, imponer la restricción $x_n \leftarrow c$ no trivializa el circuito. En efecto, si lo hiciese, la función resultante sería independiente del valor de las $n-1$ variables restantes, pero claramente podemos encontrar dos configuraciones de estas con valores opuestos. Por otra parte y utilizando el mismo razonamiento, es evidente que la función paridad depende de la entrada x_n, por lo que en el circuito que computa la función paridad debe estar conectada al menos a una puerta —pues de lo contrario no intervendría en la función computada—. Una sustitución constante apropiada hace que dicha puerta únicamente dependa de una de sus entradas —o incluso que sea constante— y, en consecuencia, puede ser eliminada transformando adecuadamente el circuito. Por tanto, la restricción $x_n \leftarrow c$ elimina una puerta.

La idea ahora es afirmar que la nueva función consiste en la función original restringida al nuevo subconjunto —definido por la restricción $x_n = c$— y precisamente coincide con la función paridad de $x_1 \oplus \cdots \oplus x_{n-1}$ o su negación. Siendo así, deberíamos poder realizar el mismo razonamiento que hemos hecho para x_n pero esta vez para x_{n-1} e inductivamente podríamos imponer $n-1$ restricciones —todas las veces en las que podemos afirmar que la restricción $x_i \leftarrow c$ no trivializa la función— que eliminan al menos una puerta cada una.

Nota 2.3. Todos los resultados de la Sección 2.2, la Sección 2.3 y la Sección 2.4 son originales de [6], [7] y [15], pero están organizados y explicados con un enfoque didáctico propio. No obstante, la Proposición 2.35 y su demostración, la Subsección 2.3.1, la Nota 2.40 y el Teorema 2.14 con su demostración corresponden a enunciados propios y originales de este documento.

2.1. Preliminares: marco formal de los circuitos booleanos

El comienzo de nuestro estudio debe pasar por definir un marco formal común sobre el que no haya ningún tipo de ambigüedad con respecto a los conceptos de *circuito*, *puerta*, *operación*, etc. Es más, estaremos interesados en restringir el alcance de nuestro estudio a circuitos que computen funciones booleanas de n bits en 1, por lo que en realidad nuestra definición de circuito será algo menos general que la idea de circuito en lenguaje natural.

Definición 2.4 (Función booleana)**.** Sea $\mathbb{F}_2 = \{0, 1\}$ el cuerpo finito de dos elementos. Definimos una *función booleana* como cualquier función $f : \mathbb{F}_2^n \to \mathbb{F}_2$.

Definición 2.5 (Operación booleana)**.** Definimos el conjunto B_2 de *operaciones booleanas* como

$$B_2 = \{f : \mathbb{F}_2^2 \to \mathbb{F}_2\}$$

Definición 2.6 (Circuito)**.** Sea $G = (V, E)$ un grafo dirigido acíclico y sea $b : V \to B_2$. Decimos que $C = (G, b)$ es un *circuito booleano* de n entradas si:

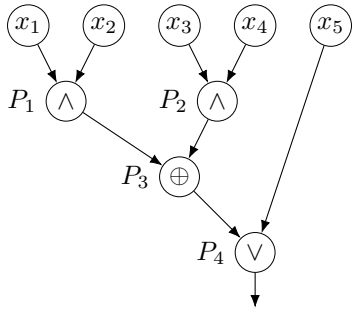


Figura 2.1: Ejemplo de circuito como grafo acíclico según la Definición 2.6.

- El conjunto V se puede particionar en tres conjuntos X, I y S definidos como

$$X = \{v \in V : gr^-(v) = 0\}$$
$$I = \{v \in V : gr^-(v) = 2\}$$
$$S = \{v \in V : gr^+(v) = 1\}$$

  y que verifican que $|X| = n$ y $|S| = 1$.

- La restricción $b|_X \equiv id$.

A los vértices de X los llaremos variables o puertas de entrada, a los vértices de I puertas internas y al vértice de S puerta de salida.

Habitualmente, denotaremos por $x_1, \ldots, x_n$ a cada una de las entradas y por Z a la puerta de salida. Si se etiqueta cada vértice con una letra mayúscula, para cada puerta interna P, será común referirse a la operación asignada por $b(P)$ como $p(x, y)$. Del mismo modo, a las puertas Q y R —que pueden ser tanto de entrada como internas— para las que existe una arista $Q \to P$ y $R \to P$ las denotaremos por "entradas de P". Es evidente que en ese caso la función booleana $p(x_0, y_0)$ está alimentada por $q(x_1, y_1)$ y $r(x_2, y_2)$, es decir, la puerta P computa $p(q(x_1, y_1), r(x_2, y_2))$ —o la correspondiente función con las entradas cambiadas de orden—. En la Figura 2.1 se puede ver un ejemplo de un circuito según esta definición. En este ejemplo P_3 es una puerta interna, b le asigna la operación $\oplus$ y sus entradas son P_1 y P_2.

Los circuitos booleanos tal y como los hemos definido computan funciones booleanas. Dichas funciones vienen dadas por la composición de forma recursiva de las operaciones asignadas a las puertas desde la salida hasta las entradas, donde se asigna la operación identidad.

Definición 2.7 (Función computada por C)**.** Sea $C = (G, b)$ un circuito booleano y la función $\phi : V \to (\mathbb{F}_2^n \to \mathbb{F}_2)$ definida por

$$\phi(x) = \begin{cases} x & \text{si } x \in \{x_1, \ldots, x_n\} \\ (b(x))(\phi(y_1), \phi(y_2)) & \text{en otro caso} \end{cases}.$$

Definimos la función $f : \mathbb{F}_2^n \to \mathbb{F}_2$ que computa como $f(x) = \phi(Z)$.

Nota 2.8. Nuestra definición de circuito permite interpretar el subgrafo que tiene como salida cualquier puerta interna P como un subcircuito en términos de la misma definición. Es por ello que hablar de *la función que computa una puerta* P tiene sentido, pues está definida por la función que computa el subcircuito mencionado y viene dada por $\phi(P)$. Por comodidad, para una puerta P nos referiremos a esta función como $f_P : \mathbb{F}_2^n \to \mathbb{F}_2$ en lugar de $\phi(P)$.

El conjunto B_2, que está compuesto por todas las funciones booleanas de dos vértices en uno, es finito y está compuesto por $2^{2^2} = 16$ operaciones distintas. Por identificarse con las puertas del grafo en los circuitos, en ocasiones también se refieren a ellas como las posibles 16 *puertas lógicas* con las que se pueden formar circuitos. Para los resultados posteriores de este capítulo, nos interesará diferenciar en tres grupos distintos las operaciones de este conjunto.

Proposición 2.9. *Sea B_2 el conjunto de todas las funciones $f : \mathbb{F}_2^2 \to \mathbb{F}_2$. Cualquier función $f \in B_2$ pertenece a uno y solo uno de los siguientes conjuntos:*

$$\begin{aligned}
&\odot\text{-puertas} = \{0, 1, x, y, \neg x, \neg y\} \\
&\oplus\text{-puertas} = \{f \in B_2 : f(x, y) = x \oplus y \oplus c \text{ con } c \in \mathbb{F}_2\} \\
&\wedge\text{-puertas} = \{f \in B_2 : f(x, y) = (x \oplus c_1) \wedge (y \oplus c_2) \oplus c_3 \text{ con } c_1, c_2, c_3 \in \mathbb{F}_2\}
\end{aligned}$$

A estos conjuntos también se los denomina como operaciones degeneradas, lineales y cuadráticas, respectivamente.

Demostración. En la Tabla A.1 del Apéndice A se muestra la clasificación completa. □

Nota 2.10. A pesar de utilizar el término *degeneradas* para referirnos a todas las puertas del conjunto de $\odot$-puertas, será habitual abusar de la notación para denotar por *degeneradas* únicamente a aquellas puertas que dependen solo de una de sus entradas y por *triviales* a aquellas puertas que computan una constante c.

En relación a lo anterior y de la mano de la técnica de eliminación de puertas, debemos definir cuándo podemos eliminar una puerta del circuito y cuándo no. Cualquier puerta del conjunto $\odot$-puertas puede eliminarse del circuito sin incrementar su tamaño. El motivo es evidente cuando se trata de alguna de las dos operaciones constantes o de alguna de las operaciones identidad. Si la operación que computaba cierta puerta es constante, entonces basta con transformar todas las puertas que alimenta sustituyendo la operación que tienen asignada por la operación que resulta de sustituir esa misma constante en la operación que figuraba —que de hecho hará que como mínimo la puerta sea degenerada—. Si la operación que computaba era la identidad, basta con conectar directamente la entrada de la puerta que eliminamos a la entrada de la puerta que alimentaba. No obstante, parece menos evidente cuando tenemos alguna de las dos negaciones. Es posible considerar circuitos booleanos donde las negaciones se encuentran restringidas exclusivamente a las variables de entrada, añadiendo únicamente una sobrecarga polinómica al tamaño total del circuito. El resultado se encuentra en [20] e implica una restricción estructural suficiente para caracterizar la complejidad de circuitos de los lenguajes en P sin pérdida de generalidad polinómica.

Tras el razonamiento anterior, parece relevante estudiar si cualquier puerta puede transformarse en una del conjunto $\odot$-puertas y de qué manera puede hacerse para poder generar puertas eliminables en el circuito, pues nuestros esfuerzos estarán centrados en que las restricciones que impongamos tengan tal efecto en algunas de estas y así poder eliminarlas.

Teorema 2.11 (Degeneración de puertas)**.** *Sea B_2 el conjunto de operaciones booleanas; entonces:*

- *Sea $P \in \wedge$-puertas, entonces $\exists c_1, c_2 \in \mathbb{F}_2$ tales que $P(c_1, y)$ es trivial y $P(c_2, y)$ es degenerada.*
- *Sea $P \in \oplus$-puertas, entonces $\exists c_1, c_2 \in \mathbb{F}_2$ y una sustitución $x = y \oplus c_2$ tales que $P(c_1, y)$ es degenerada y $P(y \oplus c_2, y)$ es trivial.*

Demostración. En la Tabla A.2 del Apéndice A figura una tabla donde se prueba el resultado. □

2.2. Cota de $3n - o(n)$ para los *affine dispersers*

Este primer artículo [6] sienta las bases formales de los otros dos que lo siguen. En él, primero se presentan los *affine dispersers*, las funciones booleanas a las que se les puede imponer cierto número de restricciones afines sin trivializar su circuito y que serán sobre las que se probará la cota inferior en el tamaño de sus circuitos. La premisa de la definición de este tipo de funciones permite evitar tener que demostrar, cada vez que se impone una restricción, que se sigue en las condiciones del Teorema 2.1 siempre y cuando la restricción impuesta sea afín.

Después, dado que demostrar que cada restricción elimina al menos δ puertas generalmente es una tarea complicada, se generaliza el Teorema 2.1 para que baste demostrar en su lugar que una cierta función μ sobre el circuito disminuye una cantidad δ cada vez que se impone una restricción, lo que permitirá tener más recursos para demostrar el resultado cuando se imponen las restricciones.

Posteriormente, se verá que la única herramienta con la que contamos para generar puertas triviales o degeneradas que poder eliminar es el Teorema 2.11, lo que se traduce en que nuestros esfuerzos deberán centrarse en encontrar sustituciones de constantes sobre las entradas de ciertas puertas y que estas se traduzcan en restricciones afines para poder hacer uso de las buenas propiedades de los *affine dispersers*. Para ello, será necesario probar una serie de resultados que nos proporcionen garantías a la hora de hacer las sustituciones.

Por último, se demostrará la cota inferior haciendo uso de todos los elementos previos. El desarrollo general de la prueba será el siguiente: nos situaremos sobre una cierta puerta P con una cierta estructura local, haremos una sustitución constante en una de sus entradas —que se traduzca como una restricción afín— y transformaremos el circuito adecuadamente para que la medida μ disminuya en la cantidad deseada sin modificar la función que el circuito computa (restringida, claro está, al nuevo subconjunto definido tras añadir la restricción). Además, será necesario probar que el procedimiento anterior siempre puede hacerse porque siempre figura una puerta P en el circuito con las características deseadas.

2.2.1. Definición de *affine disperser*

Recordemos que nuestros esfuerzos se enmarcan en el contexto de tratar de encontrar una función booleana que no se pueda computar con circuitos de tamaño polinómico, sino más "grandes". En realidad, no nos hace falta encontrar *explícitamente* una función, sino que podemos probar una cota inferior del tamaño de sus circuitos para funciones booleanas que cumplan ciertas propiedades, supuesto que exista alguna que las verifica. En este primer resultado [6], los autores Demenkov y Kulikov prueban una cota inferior para una familia especial de funciones booleanas que tiene buenas propiedades a la hora de utilizar el teorema de eliminación de puertas: los *affine dispersers*. Dentro de las hipótesis necesarias para aplicar el teorema de eliminación de puertas, la más complicada de demostrar habitualmente es la que requiere que el circuito no sea trivializado al imponer las restricciones y es precisamente este punto el que trata de garantizar esta clase de funciones.

Definición 2.12 (*Affine dispersers*)**.** Una función booleana $f : \mathbb{F}_2^n \to \mathbb{F}_2$ se dice que es un *affine disperser* de dimensión d si no es constante en ningún subespacio afín $D \subset \mathbb{F}_2^n$ de dimensión al menos d.

Proposición 2.13. *Sea f un affine disperser de dimensión d; entonces pueden imponerse $n - d$ restricciones afines sin trivializar su circuito.*

Demostración. Un conjunto formado por un máximo de $n-d$ ecuaciones afines —restricciones de tipo $\bigoplus_{i\in I} x_i = c$— forma un subespacio afín que tiene dimensión al menos d. Por la propia definición de *affine disperser*, f no puede ser constante sobre ninguno de estos subespacios afines. □

El resultado anterior prueba que la familia de los *affine dispersers* garantiza un mínimo de restricciones que pueden imponerse con la seguridad de que se sigue en las hipótesis del Teorema 2.1. No obstante, es necesario verificar que, tras imponerse una restricción, la función restringida al nuevo subconjunto es nuevamente un *affine disperser*, pues nos es necesario para poder seguir aplicando las mismas propiedades —aunque ahora podamos incorporar un número menor de restricciones—; y además, verificar que en efecto existen[1] funciones booleanas que son *affine dispersers*. El primer punto es trivial, pues cualquier función booleana que sea un *affine disperser* sobre un espacio afín también lo es sobre cualquier subespacio afín suyo. El segundo no lo es en absoluto y no es hasta la publicación de [2] que Ben-Sasson y Kopparty dan una construcción explícita de un *affine disperser* de dimensión en $o(n)$.

2.2.2. Generalización del teorema de eliminación de puertas

El objetivo de la prueba ahora es sencillo: si somos capaces de encontrar $n-d$ restricciones afines que eliminen al menos 3 puertas cada una, entonces habremos demostrado que al menos debe haber $3\cdot(n-d)$ puertas. Si $d \in o(n)$, entonces la cota inferior sería $3n - o(n)$ puertas y el resultado quedaría probado. Esta tarea no suele ser sencilla y, por este motivo, se opta por seguir otro procedimiento que surge de generalizar el resultado del Teorema 2.1.

Supongamos que, dado un circuito C, somos capaces de definir una métrica que depende del tamaño de este y de otra cierta función determinada por el circuito y, posiblemente, más parámetros. Por ejemplo, una función de estas características sería $\mu(C,x) = |C| + f(C,x)$. Si tras imponer una restricción somos capaces de transformar el circuito C en otro circuito C' que sigue computando la función —restringida al nuevo subconjunto de $\mathbb{F}_2^n$— y probamos que la métrica de C' disminuye al menos una cantidad δ, entonces $\mu(C,x) - \mu(C',x') \geq \delta$. De poder repetirse este proceso para las $r(n)$ restricciones, por inducción, tendríamos que $\mu(C,x) \geq r(n)\cdot\delta$ y, en ese caso:

$$|C| = \mu(C,x) - f(C,x) \geq r(n)\cdot\delta - f(C,x)$$

Esta expresión permite dar una cota inferior para el tamaño de los circuitos siempre y cuando se controle el valor de $f(C,x)$. La ventaja es que esta versión no necesita probar que cada restricción elimina δ puertas, sino solo que cada restricción disminuye la medida δ unidades. Por tanto, disponemos de una herramienta mucho más flexible a la hora de razonar sobre la eliminación de puertas, pues ahora están involucrados muchos más elementos que pueden ser "eliminados" para la consecución de nuestro objetivo. La formulación general de este desarrollo no necesita que la métrica μ esté definida como la suma que se ha dado como ejemplo, sino que únicamente esté acotada por una función g monótona e invertible con respecto al tamaño del circuito y esta es precisamente la generalización del Teorema 2.1.

Teorema 2.14 (Técnica de eliminación de puertas generalizada)**.** *Sea $\mathcal{C}$ el conjunto de circuitos booleanos, sea X un conjunto arbitrario de parámetros, sea $\mu : \mathcal{C}\times X \to \mathbb{R}$ una medida de complejidad y sea $g : \mathbb{N} \to \mathbb{R}$ una función estrictamente creciente e invertible. Supongamos que para todo circuito*

[1] Por motivos que quedarán claros más adelante, el interés estaba en encontrar *affine dispersers* de dimensión $o(n)$. La existencia de *affine dispersers* en general ya había sido probada mucho antes. Por ejemplo, el producto escalar de dos cadenas de bits es un *affine disperser* de dimensión $n+1$, donde n es el número de variables de entrada del producto escalar ($n/2$ por cadena).

$C \in \mathcal{C}$ y para cualquier parámetro $x \in X$, se verifica la relación:

$$\mu(C, x) \leq g(|C|).$$

Si ante una secuencia de r restricciones existe una secuencia de circuitos $C_0, C_1, \ldots, C_r$ y una secuencia de parámetros $x_0, x_1, \ldots, x_r \in X$ tales que:

1. *Cada paso disminuye la medida en al menos $\delta > 0$:*

$$\mu(C_{i-1}, x_{i-1}) - \mu(C_i, x_i) \geq \delta \quad \forall i \in \{1, \ldots, r\}$$

2. *El estado final está acotado inferiormente por una constante: $\mu(C_r, x_r) \geq \mu_0$*

entonces el tamaño del circuito original C_0 está acotado por:

$$|C_0| \geq g^{-1}(\mu_0 + r \cdot \delta).$$

Demostración. El primer punto, por inducción, garantiza que $\mu(C_0, x_0) - \mu(C_r, x_r) \geq r \cdot \delta$. Utilizando la hipótesis de que el estado final está acotado inferiormente, podemos aislar la medida del circuito original y acotarla inferiormente:

$$\mu(C_0, x_0) \geq \mu(C_r, x_r) + r \cdot \delta \geq \mu_0 + r \cdot \delta.$$

Una vez se ha dado una cota inferior para la medida del circuito original, la cota superior de g permite acoplar las desigualdades resultando en la expresión

$$g(|C_0|) \geq \mu(C_0, x_0) \geq \mu_0 + r \cdot \delta$$

y como g es estrictamente creciente por hipótesis, su inversa g^{-1} existe y preserva el orden de la desigualdad, así que:

$$|C_0| \geq g^{-1}(\mu_0 + r \cdot \delta).$$

□

La función μ del Teorema 2.14, en cierta manera, está midiendo cómo de complejo es el circuito con respecto a varios elementos en lugar de únicamente con respecto al número de puertas. Es por ello que la denominaremos medida de complejidad del circuito. Para nuestro caso concreto, Demenkov y Kulikov en [6] la definen como la suma del número de puertas internas y el número de variables de entrada que intervienen en el circuito (aquellas que tienen grado de salida al menos uno).

Definición 2.15 (Medida de complejidad)**.** Sea C un circuito booleano sobre B_2. Definimos su *medida de complejidad* como la función

$$\mu : \Omega \longrightarrow \mathbb{R}$$
$$\mu(C) = |C| + n(C)$$

donde $|C|$ y $n(C)$ denotan el número de puertas internas y el número de variables con grado de salida al menos 1 respectivamente.

Haciendo un desarrollo análogo al que introdujo el Teorema 2.14, para esta definición tendríamos

$$|C| = \mu(C) - n(C) \geq \delta \cdot (n - d) - n = (\delta - 1)n - \delta d$$

luego el objetivo va a ser encontrar $n - d$ restricciones afines que disminuyan la medida en al menos 4 unidades cada una (para que $\delta - 1$ sea 3), esto es, tenemos que demostrar que podemos encontrar $n - d$ restricciones que, de imponerse, cada una transforma C en un circuito C' con como poco 4 puertas internas o variables menos.

2.2.3. Procedimiento de búsqueda de las restricciones afines

¿Cómo saber qué restricciones afines trivializan o degeneran puertas para poder eliminarlas? El único resultado del que disponemos acerca de cómo trivializar o degenerar puertas es el Teorema 2.11. Por tanto, parece que la única herramienta de la que podemos hacer uso para eliminar una puerta P es hacer una sustitución de una constante en una de sus entradas, y de alguna manera esto debe poder traducirse en imponer una restricción afín sobre las variables. En ese caso, el procedimiento a seguir sería el siguiente:

1. Escogeremos una cierta puerta P del circuito C.
2. Sustituiremos una constante c sobre una de sus entradas para trivializar o degenerar dicha puerta.
3. Demostraremos que la sustitución anterior disminuye la medida de complejidad en al menos 4 unidades.
4. Traduciremos dicha sustitución de una constante como una restricción afín sobre el dominio de la función booleana que computa el circuito.
5. Describiremos cómo se simplifica el circuito C en otro C' que también computa la restricción de f al nuevo subespacio afín y que es más sencillo que C —en términos de la medida de complejidad definida—.

Aún siendo lo suficientemente astutos como para escoger de forma inteligente tanto P, como su entrada y la constante c que sustituir sobre ella —para disminuir la medida la cantidad deseada—, no queda demasiado claro cuándo podremos traducir esta sustitución en una restricción afín —que son las únicas válidas para trabajar—. Además, tampoco queda claro cómo debemos transformar el circuito después para obtener otro que también compute la restricción de la función al nuevo subespacio afín y que sea más sencillo en términos de la medida de complejidad.

Para la primera parte, siempre y cuando las entradas de la puerta P sean o una constante o una puerta G cuya función computada f_G sea una función afín, podemos ver que hacer constante dicha entrada se puede traducir en una restricción afín. En efecto, las sustituciones constantes $x_j = c$ y $\bigoplus_{j\in J} x_j \oplus c_0 = c$ se pueden ver como restricciones afines $x_j \oplus c = 0$ y $\bigoplus_{j\in J} x_j \oplus (c_0 \oplus c) = 0$. Parece razonable entonces suponer que las restricciones afines que utilizaremos para demostrar el resultado siempre surgirán de querer sustituir una constante sobre una entrada que es o bien una variable o bien una puerta que computa una función afín[2].

Para la segunda parte, también podemos observar que cualquier restricción afín puede interpretarse fácilmente como una sustitución sobre una cierta variable de entrada. En efecto, $x_j \oplus \bigoplus_{i\in I} x_i \oplus c = 0$ es lo mismo que $x_j = \bigoplus_{i\in I} x_i \oplus c$ por la propiedad involutiva de la operación *xor*. El circuito resultante es aquel donde en todos los lugares en los que intervenía x_j ahora interviene $\bigoplus_{i\in I} x_i \oplus c$.

De nuevo, si suponemos que las restricciones afines surgirán de sustituir una constante sobre una entrada que es o bien una variable o bien una puerta que computa una restricción afín, entonces podemos distinguir tres casos en función del tipo de restricción afín que genere esa sustitución constante: los correspondientes a las proposiciones 2.16, 2.17 y al Lema 2.18.

[2]Lo que, sin embargo, no implica que la puerta sea una $\oplus$-puerta. Una puerta *and* que toma el valor 0 en alguna de sus entradas computa la función constantemente cero, que es una función afín. Por tanto, cualquier subconjunto sobre el que se restrinja la función f que implique tal valor constante en alguna de sus entradas hace que esta $\wedge$-puerta compute de hecho una función afín.

Proposición 2.16 (Sustituciones constantes)**.** *Sea C un circuito. Denotemos por $x_1, \ldots, x_n$ sus variables de entrada y sea c una constante. La sustitución $x_1 \leftarrow c$ transforma C en otro circuito C' con las siguientes propiedades:*

- *Si P computaba la operación $p(x_1, t)$, pasa a computar $p(c, t)$.*
- *El número de puertas en C' es el mismo que en C.*
- *La topología del circuito en C' es la misma que en C.*
- *Si H computaba la función $h(x_1, \ldots, x_n)$, pasa a computar $h(c, \ldots, x_n)$.*

Demostración. Por brevedad, la demostración detallada se omite aquí y puede consultarse en [7]. □

Proposición 2.17 (Sustituciones afines simples)**.** *Sean C un circuito, $x_1, \ldots, x_n$ sus variables de entrada y c una constante. La sustitución $x_1 \leftarrow x_2 \oplus c$ transforma C en otro circuito C' con las siguientes propiedades:*

- *Si P computaba la operación $p(x_1, t)$, pasa a computar $\tilde{p}(x_2, t)$.*
- *El número de puertas en C' es el mismo que en C.*
- *La topología del circuito en C' es la misma que en C.*
- *Si H computaba la función $h(x_1, \ldots, x_n)$, pasa a computar $h(x_2 \oplus c, \ldots, x_n)$.*

Demostración. Por brevedad, la demostración detallada se omite aquí y puede consultarse en [7]. □

Lo razonable sería que a continuación figurase una proposición más que explicase cómo hacer sustituciones afines generalizadas $x_j = \bigoplus_{i \in I} x_i \oplus c$ manteniendo las buenas propiedades de las dos anteriores. Lamentablemente, este es un caso bastante más complicado, pues sustituir x_j por el valor de $\bigoplus_{i \in I} x_i \oplus c$ requiere de un subcircuito que compute esa función y, de no estar presente, esto aumentaría el número de puertas. Afortunadamente, el Lema 2.18 nos proporciona una herramienta menos fuerte que la que pedíamos, pero suficiente para hacer la distinción de casos necesaria más tarde. La Figura 2.2 muestra un ejemplo de cómo se materializa la transformación que describe el resultado.

Lema 2.18. *Sea P una $\oplus$-puerta de C que solo depende de $\oplus$-puertas de grado de salida 1 o de variables, entonces existe una variable x_j y un subconjunto —posiblemente vacío— $I = \{1, \ldots, n\} \setminus \{j\}$ tal que la restricción $x_j \oplus \bigoplus_{i \in I} x_i = c$ trivializa la puerta P y reduce $n(C)$ en al menos una unidad.*

Demostración. El primer paso es probar que la puerta P computa una función del tipo $x_j \bigoplus_{i \in I} x_i \oplus c_0$. Como cada entrada de P depende únicamente de $\oplus$-puertas o variables, existen dos subconjuntos de índices $L, R \subset \{1, \ldots, n\}$ tales que la puerta P computa

$$\left(\bigoplus_{l \in L} x_l \oplus c_L \right) \oplus \left(\bigoplus_{r \in R} x_r \oplus c_R \right) \oplus c_P = \bigoplus_{k \in L \cup R} x_k \oplus c_0$$

El conjunto $L \cup R$ debe tener al menos dos índices distintos, pues de lo contrario $\bigoplus_{k \in L \cup R} x_k = 0$ y P sería trivial, lo que no es posible por hipótesis. De esta manera, denotando a uno de ellos por j y agrupando los restantes en el conjunto que llamaremos I, tenemos probada esta primera parte.

El segundo paso es demostrar que imponer la restricción del lema trivializa la puerta P y reduce el número de entradas influyentes del circuito en al menos una unidad. Esto pasa por demostrar que somos capaces de recablear[3] el circuito de manera que cualquier puerta alimentada por x_j ya no lo esté —para que deje de ser influyente— y que somos capaces de hacerlo trivializando P y sin añadir ninguna puerta más. El método es el siguiente: como la restricción puede verse como la sustitución $x_j = \bigoplus_{i\in I} x_i \oplus c$, transformar C en un circuito C' que compute la función original con esta restricción puede hacerse inyectando $\bigoplus_{i\in I} x_i \oplus c$ allí donde x_j sea una entrada en el circuito C. La parte del circuito que computa $\bigoplus_{i\in I} x_i \oplus c$ tiene al menos $|I|-1$ puertas por la demostración que vimos en el Ejemplo 2.2, por lo que podemos eliminar esta parte del circuito y reconstruirla usando $|I|-1$ puertas conectadas en serie. Después bastaría utilizar la salida de esta parte en lugar de x_j en el resto del circuito. Nótese que hemos podido eliminar la topología previa del circuito y sustituirla por esta otra de $|I|-1$ puertas porque sabíamos a priori que todas las puertas involucradas tenían grado de salida 1, es decir, no tenían influencia más allá de alimentar P y, por tanto, no estamos distorsionando otras partes del circuito afectadas por estas. Por último, como P computa $x_j \bigoplus_{i\in I} x_i \oplus c$, tal restricción implica que P pasa a computar $\bigoplus_{i\in I} x_i \oplus c \bigoplus_{i\in I} x_i \oplus c_0 = c \oplus c_0$, luego se trivializa. □

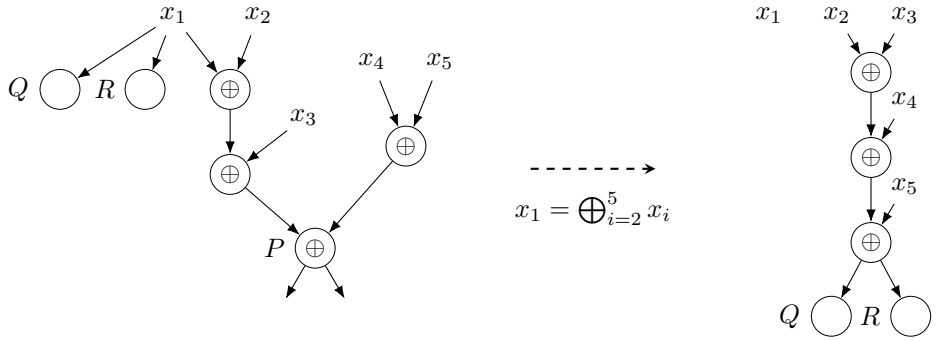


Figura 2.2: Ejemplo de [6] para mostrar la aplicación del Lema 2.18. El subcircuito que computa P se simplifica y alimenta todas las entradas donde x_1 se insertase. La puerta P se elimina y se inserta la constante de la sustitución en todas aquellas puertas que alimentase.

2.2.4. Demostración del resultado principal

Recapitulando, el objetivo de todo el desarrollo es probar que dado un *affine disperser* de dimensión d y el circuito C que lo computa, al menos $n-d$ veces podremos encontrar una puerta P sobre la que hacer una sustitución constante en una de sus entradas y disminuir al menos la medida de complejidad en 4 unidades —entre puertas internas y variables que intervienen eliminadas—. Estas sustituciones constantes siempre deben poder traducirse en restricciones afines para verificar que no trivializamos nunca el *affine disperser*, lo que nos asegura estar en las condiciones del Teorema 2.1.

A continuación, presentamos el resultado principal de la sección, que demuestra la cota inferior de $3n - o(n)$ para los *affine dispersers*.

Teorema 2.19. *Sea $f : \mathbb{F}_2^n \to \mathbb{F}_2$ un affine disperser de dimensión d, sea $A \subset \mathbb{F}_2^n$ un subespacio afín de dimensión D y sea C el circuito de n entradas que computa $f|_A$, entonces:*

$$\mu(C) \geq 4(D-d) \tag{2.1}$$

[3] Aunque en la prueba no es necesario, cuando hablamos de "recablear" en realidad también permitimos modificar la operación de cada puerta además de modificar las conexiones. Mientras se conserve la función que computa el circuito no hay problema al respecto, pues el interés está únicamente en no añadir puertas.

Demostración. Probamos la desigualdad por inducción en D. El caso base $D \leq d$ es trivial porque la medida es siempre positiva, luego $\mu(C) \geq 0 \geq 4(D-d)$. Para el caso inductivo recordemos que bastaría con probar que la medida disminuye en al menos 4 unidades al imponer una restricción del tipo $\bigoplus_{i \in I} x_i = c$ que transforme el circuito C en otro C', pues en ese caso

$$\mu(C) \geq 4 + \mu(C') \geq 4 + 4(D-1-d) = 4(D-d)$$

De esta manera, la prueba se reduce a demostrar que cada restricción disminuye la medida en al menos 4 unidades. Consideremos un orden topológico[4] de las puertas y escojamos la primera que no es una $\oplus$-puerta de grado de salida 1 a la que llamaremos P. Esta puerta siempre debe existir, pues de lo contrario el circuito C solo estaría formado por $\oplus$-puertas de grado de salida 1 y eso significaría que no es un *affine disperser*. En efecto, si el circuito solo tiene estas puertas, la función computada es $f(x) = \bigoplus_{i \in I} x_i \oplus c$ y esta es constante en el subespacio $\bigoplus_{i \in I} x_i = c$. Por otra parte, ninguna puerta que se trivialice en el desarrollo que viene a continuación puede ser la puerta de salida, pues hemos demostrado al inicio que se pueden imponer $n - d$ restricciones lineales sin trivializar el circuito. La siguiente distinción de casos (que puede visualizarse en la Figura 2.3) cubre la totalidad de situaciones en las que puede encontrarse dicha puerta P, luego demuestra que siempre existe la posibilidad de imponer alguna de las siguientes restricciones:

1. P es una $\oplus$-puerta de grado de salida al menos 2:

   El Lema 2.18 permite trivializar este tipo de puertas en estas condiciones. En consecuencia, trivializamos P, al menos degeneramos a sus sucesores y al menos una variable deja de ser influyente, luego $\mu(C) - \mu(C') \geq n(C) + |C| - (n(C) - 1) - (|C| - 3) = 4$.

2. P es una $\wedge$-puerta:

   a) P tiene al menos una puerta Q como entrada:

   Por ser P la primera en el orden topológico impuesto, la puerta Q solo puede ser una $\oplus$-puerta. De nuevo, utilizando el Lema 2.18 podemos trivializarla con el valor constante c que también trivialice P. Esto trivializa Q y P, al menos degenera al sucesor de P y una variable deja de ser influyente, luego $\mu(C) - \mu(C') \geq n(C) + |C| - (n(C) - 1) - (|C| - 3) = 4$.

   b) Ambas entradas de P son variables y al menos una —llamémosla x_i— tiene grado de salida al menos 2:

   Si sustituimos en esta variable la constante que trivializa P, entonces trivializamos P, al menos degeneramos a los sucesores de P, al menos degeneramos a los sucesores de x_i y al menos una variable deja de ser influyente. El problema a la hora de hacer el conteo es que los sucesores de x_i y los de P pueden coincidir, así que tendríamos que $|C| - |C'| \geq 2$ y necesitamos que sea 3. Distinguimos varias situaciones:

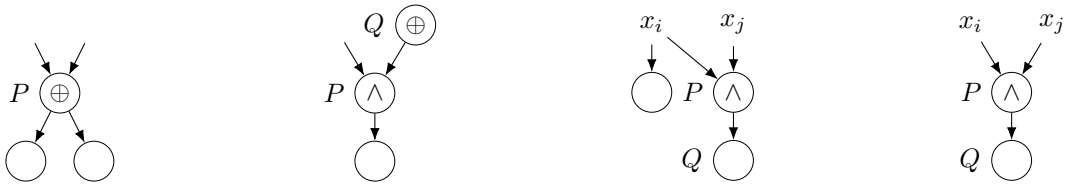
   1) P tiene más de un sucesor: trivialmente entonces sí sabemos que se eliminan 3 puertas.
   2) P tiene un sucesor y es distinto del de x_i: nuevamente, ahora sí sabemos que se eliminan 3 puertas (P, el sucesor de P y el sucesor de x_i).
   3) P tiene un sucesor y es el mismo que el x_i: entonces la sustitución también trivializa Q (por ser constantes sus dos entradas) y, en consecuencia, al menos degeneramos al sucesor de Q adicionalmente, luego se vuelven a eliminar tres puertas.

   En cualquiera de los tres casos propuestos se tiene que $\mu(C) - \mu(C') \geq n(C) + |C| - (n(C) - 1) - (|C| - 3) = 4$.

[4]Por orden topológico nos referimos al inducido por las aristas del grafo, esto es, $P \to Q$ implica que $P < Q$. Tal orden topológico existe porque, por definición, el grafo es dirigido y acíclico.

c) Ambas entradas de P son variables de grado de salida 1:
Asignando a una de ellas el valor que trivializa P, trivializamos P, degeneramos a su sucesor y reducimos en dos el número de variables influyentes en 2, luego $\mu(C) - \mu(C') \geq n(C) + |C| - (n(C) - 2) - (|C| - 2) = 4$.

□



(a) Caso 1 (b) Caso 2.a (c) Caso 2.b.3 (d) Caso 2.c

Figura 2.3: Representación visual de la distinción de casos de la demostración del Teorema 2.19.

Corolario 2.20. *Cualquier circuito sobre la base B_2 que compute un affine disperser de dimensión $d \in o(n)$ tiene al menos $3n - o(n)$ puertas.*

Demostración. Particularizando la inecuación 2.1 del Teorema 2.19 para cuando el subespacio afín sobre el que calculamos la cota inferior es todo $\mathbb{F}_2^n$, tenemos que $\mu(C) \geq 4(n-d)$. Por tanto, separando $|C|$ de $n(C) + |C| \geq 4(n-d)$ obtenemos la desigualdad:

$$|C| \geq 4(n-d) - n(C) \geq 4(n-d) - n = 3n - 4d \in 3n - o(n)$$

□

2.3. Cota de $(3 + 1/86)\, n - o(n)$ para los *affine dispersers*

En este segundo artículo [7], Find et al. parten del trabajo hecho en el artículo anterior para refinar los razonamientos básicos que ya se plantean en él.

Cuando uno trata de replicar la demostración de [6] buscando una distinción de casos que le permita probar que la métrica disminuye más de 4 unidades, se encuentra con varios obstáculos que no le permiten elevar la cota tanto como querría. Los autores elaboran todas las herramientas necesarias para sortearlos inteligentemente y conseguir elevar algo más la disminución de la medida en cada caso.

En primer lugar, generalizan el Lema 2.18 del artículo anterior para que sea posible hacer sustituciones constantes en puertas que computen funciones afines en una gama más amplia de situaciones que las que nos garantiza las hipótesis del lema. No obstante, para que el resultado sea cierto necesitamos ampliar la definición de circuito y permitir que haya ciclos sujetos a ciertas condiciones, como por ejemplo que las salidas de las puertas estén unívocamente determinadas por las entradas del circuito.

Tras esto, enuncian unas series de reglas de normalización para centralizar el proceso de simplificación del circuito una vez hechas las sustituciones; de forma que se preserven siempre ciertas propiedades esenciales para la demostración posterior. Un ejemplo de este tipo de propiedades a conservar es precisamente el de que los circuitos, a pesar de tener ciclos, deban tener unívocamente determinadas las salidas de sus puertas. De este modo, cualquier simplificación se realizará siempre enunciando la

regla que se aplica y esto permitirá despreocuparse de comprobar si estas tienen "daños colaterales" en el resto del circuito.

Posteriormente, se centran en sortear el principal obstáculo de la distinción de casos que se ha comentado al inicio. Este impedimento está causado por una estructura local llamada *troubled gate* que no permite disminuir lo suficiente la medida con sustituciones afines. Por ello, en algunas circunstancias los autores necesitan además poder imponer restricciones cuadráticas. Esto implica construir una nueva clase de funciones, los *rdq dispersers*, que mantienen las mismas propiedades que los *affine dispersers* cuando el subconjunto es lo que denominan una *fuente rdq* en lugar de un subespacio afín. Estos subconjuntos están formados por restricciones afines y también cuadráticas sujetas a ciertos requisitos. El resto de esfuerzos de este apartado se centran en demostrar que los *affine dispersers* son un subconjunto de este nuevo conjunto de funciones, y que demostrar el resultado para los *rdq dispersers* implica poder particularizar sus conclusiones para los *affine dispersers*.

Por último, como en el artículo anterior, se demostrará la nueva cota inferior haciendo uso de todos los elementos previos. El desarrollo general de la prueba será el mismo que entonces: nos situaremos sobre una cierta puerta P con una cierta estructura local, haremos una sustitución en una de sus entradas —que se traducirá en una restricción válida en las *fuentes rdq*— y transformaremos el circuito haciendo uso de las reglas de normalización para que siga computando la misma función sujeta a las nuevas restricciones. Habrá que demostrar que la medida de complejidad disminuye lo suficiente en cada caso y, como a priori usando la medida original la distinción de casos no era capaz de aumentar la cota inferior, será necesario dar una nueva medida de complejidad que de alguna manera tenga en cuenta un coste amortizado de todas estas nuevas herramientas.

2.3.1. Intuición general de la prueba

La demostración anterior de la cota de $3n - o(n)$ consistió en encontrar una buena distinción de casos para poder argumentar que siempre podía disminuirse la medida de complejidad del circuito en 4 unidades. Parece razonable que, una vez se ha leído tal demostración, se trate de buscar una demostración similar que pruebe que puede reducirse en 5 unidades. De encontrarse, por los mismos argumentos expuestos en la sección anterior, uno podría afirmar haber probado la cota $4n - o(n)$ para los *affine dispersers*.

Denotando por P a la primera $\wedge$-puerta topológicamente hablando y por I_1 e I_2 a sus entradas, consideremos la distinción que presenta la Figura 2.4.

1. Al menos una entrada —digamos I_1— es una puerta interna con grado de salida exactamente 1.

   a) La otra entrada I_2 es una puerta interna.

   Hacer una sustitución constante en I_2 que trivialice P elimina P, su sucesora, I_2 e I_1 (porque queda desconectada de la salida del circuito). Además, la sustitución constante se traduce en una sustitución afín sobre una variable, que se desconecta del circuito, luego $\Delta\mu \geq 4 + 1 = 5$.

   b) La otra entrada I_2 es una variable de entrada de grado exactamente 1.

   En este caso no se puede disminuir la medida en al menos 5 unidades, luego en principio sería un problema. No obstante, una sustitución constante sobre I_2 disminuye en 4 la medida (que está cerca) y elimina este caso. Como esto elimina la variable I_2 del circuito, este caso solo puede ocurrir un número $o(n)$ de veces, luego puede quedar absorbido por ese término.

   c) La otra entrada I_2 es una variable de entrada de grado mayor que 1.

Si el otro sucesor de I_2 coincide con el de P, entonces hacer una sustitución constante que trivialice el sucesor elimina P (porque queda desconectada de la salida del circuito), I_1 (por el mismo motivo), el sucesor de P y el sucesor del sucesor de P.

Si el otro sucesor de I_2 no coincide con el de P, entonces hacer una sustitución constante que trivialice P elimina P, su sucesor, I_1 y el sucesor de I_2.

En ambos casos se eliminan 4 puertas y se quita una variable del circuito, luego $\Delta\mu \geq 4 + 1 = 5$.

2. Al menos una entrada —digamos I_1— es una puerta interna con grado de salida al menos 2.

   Si el otro sucesor de I_1 coincide con el de P, entonces hacer una sustitución constante que trivialice el sucesor elimina P (porque queda desconectada de la salida del circuito), el sucesor de P, el sucesor del sucesor de P e I_1.

   Si el otro sucesor de I_1 no coincide con el de P, entonces hacer una sustitución constante que trivialice P elimina P, su sucesor, I_1 y el sucesor de I_1.

   En ambos casos se eliminan 4 puertas y se quita una variable del circuito (porque la sustitución constante en una puerta se traducirá en una sustitución afín sobre una variable), luego $\Delta\mu \geq 4 + 1 = 5$.

3. Al menos una entrada —digamos I_1— es una variable con grado de salida al menos 3.

   Si el otro sucesor de I_1 coincide con el de P, entonces hacer una sustitución constante que trivialice el sucesor elimina P (porque queda desconectada de la salida del circuito), el sucesor de P, el sucesor del sucesor de P y el otro sucesor de I_1.

   Si el otro sucesor de I_1 no coincide con el de P, entonces hacer una sustitución constante que trivialice P elimina P, su sucesor, y los otros dos sucesores de I_1.

4. Al menos una entrada —digamos I_1— es una variable de grado de salida exactamente 1.

   a) La otra entrada I_2 es una variable de grado de salida exactamente 1.

   En este caso no se puede disminuir la medida en al menos 5 unidades, luego en principio sería un problema. No obstante, una sustitución constante sobre I_1 disminuye en 4 la medida (que está cerca) y elimina este caso. Como esto elimina las variables I_1 e I_2 del circuito, este caso solo puede ocurrir un número $o(n)$ de veces, luego puede quedar absorbido por ese término.

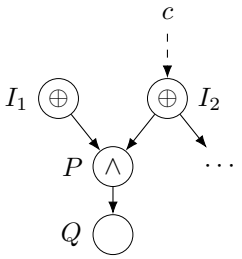
   b) La otra entrada I_2 es una variable de grado de salida al menos 2.

   Si el otro sucesor de I_2 coincide con el de P, entonces hacer una sustitución constante que trivialice el sucesor elimina P (porque queda desconectada de la salida del circuito), el sucesor de P y el sucesor del sucesor de P. Las variables I_2 e I_1 quedan desconectadas del circuito, luego $\Delta\mu \geq 3 + 2 = 5$.

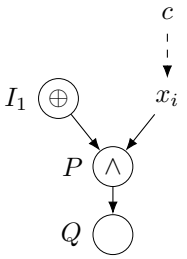
   Si el otro sucesor de I_2 no coincide con el de P, entonces hacer una sustitución constante que trivialice P elimina P, su sucesor y el sucesor de I_2. Las variables I_2 e I_1 quedan desconectadas del circuito, luego $\Delta\mu \geq 3 + 2 = 5$.

5. Las dos entradas son variables de entrada con grado de salida exactamente 2.

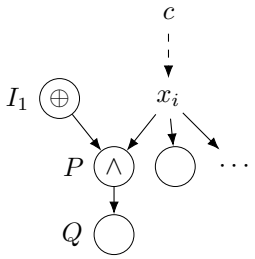
   En este caso no se puede disminuir la medida en al menos 5 unidades, luego en principio es un obstáculo para completar nuestra demostración. Además, este caso sí puede ocurrir con mucha más frecuencia porque no elimina las variables involucradas y nada impide que la posterior transformación del circuito genere de nuevo estructuras de este tipo.



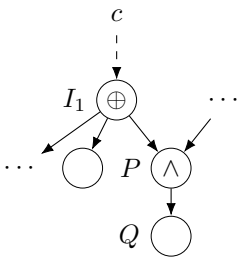
(a) Representación gráfica de la situación del caso 1.a



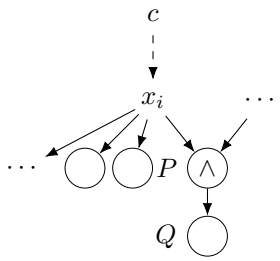
(b) Representación gráfica de la situación del caso 1.b



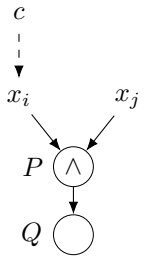
(c) Representación gráfica de la situación del caso 1.c



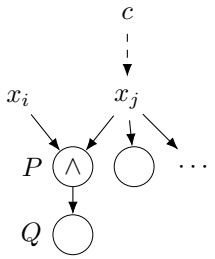
(d) Representación gráfica de la situación del caso 2



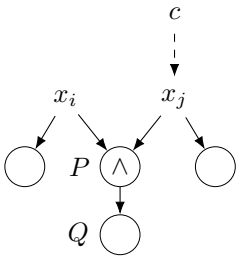
(e) Representación gráfica de la situación del caso 3



(f) Representación gráfica de la situación del caso 4.a



(g) Representación gráfica de la situación del caso 4.b



(h) Representación gráfica de la situación del caso 5

Figura 2.4: Representación gráfica de la distinción de casos de la intuición de la prueba del $(3+1/86)n-o(n)$.

Tras la distinción de casos, salta a la vista que hay una serie de elementos que es imprescindible garantizar si se quiere demostrar la cota de $4n - o(n)$.

Por un lado, lo primero que llama la atención es que la distinción ya no incluye un caso en que la puerta a trivializar es una $\oplus$-puerta. Esto provoca que la sustitución constante que tan alegremente estamos haciendo sobre una de las entradas de la puerta P ya no pueda garantizarse gracias al respaldo del Lema 2.18, pues es cierto que por la forma de elegir la puerta P sus entradas deben computar exclusivamente funciones afines, pero las puertas no tienen por qué ser $\oplus$-puertas de grado de salida 1 como pedían las hipótesis de ese resultado. Por tanto, lo primero que tendremos que hacer será generalizar el resultado que nos permite hacer sustituciones de constantes en $\oplus$-puertas. Esto es lo que motiva la Subsección 2.3.2. Además, es obvio que de alguna manera tendremos que justificar que siempre podremos encontrar al menos tantas $\wedge$-puertas como restricciones queremos imponer, pues de lo contrario la distinción de casos no sería posible.

Por otra parte, es evidente que el último caso no está resuelto y que la estructura local que en él se plantea es un obstáculo a la hora de alcanzar la cota de $4n - o(n)$. Lamentablemente, un argumento similar a los que se han hecho en casos como el $4a$ para poder justificar que tal situación es despreciable

no será posible, por lo que nos veremos obligados a enfrentarla de alguna manera. A partir de ahora la denominaremos *troubled gate*. La Figura 2.5 contiene una representación[5] gráfica.

Definición 2.21 (Troubled gate)**.** Sea P una puerta interna de un circuito C. Decimos que es una *troubled gate* si satisface estos tres requisitos:

1. P es una $\wedge$-puerta de grado de salida 1.
2. Ambas entradas de P están alimentadas por variables de entrada.
3. Ambas entradas de P tienen grado de salida 2.

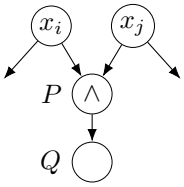


Figura 2.5: Representación gráfica de la estructura local de una *troubled gate* P.

La solución a la que llegan los autores es no tener en cuenta solo las puertas internas y las variables que intervienen en el circuito en la medida de complejidad, sino también el número de estructuras de este tipo. De esta manera, reducir el número de troubled gates también contará como parte de ese progreso para mejorar la cota inferior. Esto se hace a través de dos vías: por un lado demostrando que introducimos "pocas" cuando simplificamos el circuito y que hay "pocas" inicialmente y, por otro, incluyendo en el conteo de la distinción de casos el número de *troubled gates* eliminadas.

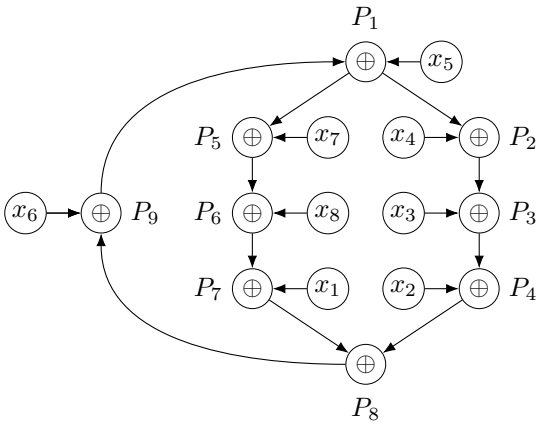
¿Qué es más importante, eliminar muchas puertas o variables o eliminar muchas *troubled gates*? Para tratar de evitar responder a esta pregunta se opta por hacer una suma ponderada en lugar de la suma absoluta que se hacía hasta ahora de todos estos elementos. La idea es estudiar cuánto se puede lograr disminuir la medida en cada caso y después ajustar las constantes para conseguir la disminución óptima. Esto es lo que motiva la Subsección 2.3.5.

2.3.2. Sustituciones afines generalizadas

Como comentamos al inicio de la Subsección 2.2.3, imponer restricciones afines también puede verse como hacer sustituciones de expresiones afines sobre variables de entrada y también demostramos cómo transformar el circuito para incorporar estas. Para ello, se enunciaron la Proposición 2.16, la Proposición 2.17 y el Lema 2.18. Sin embargo, el último resultado no nos proporcionaba un método general para cuando queríamos sustituir una constante en una entrada alimentada por una función afín cualquiera, sino solo por una que cumplía ciertos requisitos.

Ahora necesitamos ampliar el conjunto de situaciones donde ese resultado es aplicable: en lugar de requerir que la función computada solo dependa de $\oplus$-puertas de grado de salida 1, pediremos únicamente que no dependa de ninguna $\wedge$-puerta. Sin embargo, la generalización requiere cambiar el modelo de cómputo de los circuitos tal y como los hemos definido a un nuevo modelo que permite ciclos dentro de ellos, pues la transformación que sufre el circuito tras la sustitución puede generarlos. En la Figura 2.6a se puede observar un ejemplo de a qué nos referimos con circuito cíclico.

Definición 2.22 (Circuitos cíclicos)**.** Definimos un *circuito cíclico* C como un circuito en el sentido de la Definición 2.6 sin la condición de que el grafo G sea acíclico.



(a) Ejemplo de circuito cíclico consistente.

$$G_1 = x_1 \oplus x_2 \oplus x_3 \oplus x_4 \oplus x_5 \oplus x_6 \oplus x_7 \oplus x_8$$
$$G_2 = x_1 \oplus x_2 \oplus x_3 \oplus x_5 \oplus x_6 \oplus x_7 \oplus x_8$$
$$G_3 = x_1 \oplus x_2 \oplus x_5 \oplus x_6 \oplus x_7 \oplus x_8$$
$$G_4 = x_1 \oplus x_5 \oplus x_6 \oplus x_7 \oplus x_8$$
$$G_5 = x_1 \oplus x_2 \oplus x_3 \oplus x_4 \oplus x_5 \oplus x_6 \oplus x_8$$
$$G_6 = x_1 \oplus x_2 \oplus x_3 \oplus x_4 \oplus x_5 \oplus x_6$$
$$G_7 = x_2 \oplus x_3 \oplus x_4 \oplus x_5 \oplus x_6$$
$$G_8 = x_1 \oplus x_2 \oplus x_3 \oplus x_4 \oplus x_7 \oplus x_8$$
$$G_9 = x_1 \oplus x_2 \oplus x_3 \oplus x_4 \oplus x_6 \oplus x_7 \oplus x_8$$

(b) Funciones computadas por las puertas.

Figura 2.6: Circuito cíclico consistente dado como ejemplo en [7] junto a las funciones booleanas de cada una de sus puertas.

Dentro de los circuitos cíclicos, en realidad solo nos interesan aquellos que verifican que las salidas de todas sus puertas son consistentes a pesar de los ciclos, es decir, aquellos tales que para cada configuración de las entradas hay una única configuración determinista y estable de las salidas de cada puerta. Por tanto, necesitamos acotar algo más el subconjunto de circuitos cíclicos en el que en realidad queremos trabajar.

Un circuito —tanto cíclico como acíclico— se puede identificar con un sistema de ecuaciones en $\mathbb{F}_2$. En este sistema, las variables se corresponden con los valores de salida de cada puerta del circuito y las ecuaciones vienen dadas por las operaciones de cada puerta. En efecto, cualquier puerta P de entradas I_1 e I_2 que compute una operación $I_1 \circledast I_2$ genera la ecuación $P \oplus (I_1 \circledast I_2) = 0$. Por tanto, resolver el sistema de ecuaciones es determinar los valores de salida de cada una de las puertas (se entiende que parametrizados en función de los valores de entrada). Esto nos permite caracterizar de modo único los circuitos cuyas salidas son consistentes, que son con los que nos interesa trabajar.

Definición 2.23 (Circuitos cíclicos consistentes)**.** Sea C un circuito cíclico. Decimos que es *consistente* si su sistema de ecuaciones asociado tiene solución única.

Recordemos que todo este desarrollo viene de querer poder hacer sustituciones constantes sobre $\oplus$-puertas en situaciones más generales. Estas $\oplus$-puertas aparecen antes que las $\wedge$-puertas en el orden topológico de la distinción de casos por lo que no dependen de ellas. Luego el procedimiento que necesitamos solo puede generar ciclos en subcircuitos formados por $\oplus$-puertas —esta vez de grado de salida arbitrario—.

Para un circuito formado solo por $\oplus$-puertas, este sistema es lineal y, por tanto, se identifica unívocamente con dos matrices: la de coeficientes y la de términos independientes. El sistema de ecuaciones asociado tiene solución única si y solo si la matriz de coeficientes tiene rango máximo, así que disponemos de una caracterización sencilla de la consistencia para este caso particular. En la Figura 2.7 se muestra el sistema y en Figura 2.6b su solución única para el circuito cíclico de la Figura 2.6.

Además, la condición de que la restricción afín no dependa de $\wedge$-puertas nos permite entender los circuitos con los que vamos a trabajar como elementos formados por dos partes conectadas: una parte

[5] La representación es meramente informativa. Nótese que podría ocurrir que x_i y x_j compartan el otro descendiente.

formada por circuitos cíclicos lineales y otra por circuitos usuales. Estas dos partes están separadas entre sí por $\wedge$-puertas.

$$\begin{cases} G_1 = G_9 \oplus x_5 \\ G_2 = G_1 \oplus x_4 \\ G_3 = G_2 \oplus x_3 \\ G_4 = G_3 \oplus x_2 \\ G_5 = G_1 \oplus x_7 \\ G_6 = G_5 \oplus x_8 \\ G_7 = G_6 \oplus x_1 \\ G_8 = G_4 \oplus G_7 \\ G_9 = G_8 \oplus x_6 \end{cases} \Rightarrow \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \end{pmatrix} \cdot \begin{pmatrix} G_1 \\ G_2 \\ G_3 \\ G_4 \\ G_5 \\ G_6 \\ G_7 \\ G_8 \\ G_9 \end{pmatrix} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \\ x_8 \\ x_9 \end{pmatrix}$$

Figura 2.7: Sistema lineal asociado al circuito cíclico de la Figura 2.6. El rango de la matriz de coeficientes muestra que el circuito es consistente.

Definición 2.24 (Semicircuitos)**.** Sea X un circuito cíclico cuyas puertas son exclusivamente $\oplus$-puertas, con n variables de entrada y m salidas, y sea C un circuito booleano convencional de m entradas. Definimos un *semicircuito* como el circuito obtenido al conectar en serie las m salidas de X con las m entradas de C, de modo que el circuito global toma una entrada $x \in \mathbb{F}_2^n$ y computa $C(X(x))$. Asimismo, decimos que el semicircuito es *consistente* si el subcircuito cíclico X es consistente.

Nuestra intención es trabajar con semicircuitos para poder tener estas construcciones cíclicas que nos permitan utilizar el procedimiento ampliado que puede generarlas. Por tanto, a la hora de comprobar si un semicircuito es consistente, basta con comprobar de forma local que la matriz de su parte lineal cíclica es de rango máximo. Es muy razonable preguntarse si la cotas inferiores demostradas para semicircuitos también lo serán para circuitos convencionales. La respuesta es que sí porque es sencillo comprobar que, para múltiples salidas, los circuitos cíclicos son un modelo de cómputo más fuerte que los circuitos acíclicos, esto es, cualquier circuito acíclico que compute las mismas salidas que un circuito cíclico debe usar al menos tantas puertas como este; por tanto, si demostramos una cota inferior asumiendo que usamos un circuito cíclico queda demostrada automáticamenta para su versión acíclica.

Nota 2.25. Un ejemplo de por qué los circuitos cíclicos de múltiples salidas son un modelo de cómputo más fuerte que los circuitos acíclicos sería el de la Figura 2.6. Este posee 9 puertas internas que computan funciones booleanas en las que intervienen más de dos variables. Cualquier circuito acíclico que tratase de computar las mismas salidas usando 9 puertas tendría una puerta —la primera en el orden topológico— que computaría una función booleana en la que únicamente intervienen 2, luego no puede computar las mismas salidas con solo 9 puertas.

Nota 2.26. Por una cuestión de practicidad, de ahora en adelante utilizaremos el término circuito indistintamente para hablar de circuitos y semicircuitos.

Lema 2.27. *Sea C un semicircuito consistente formado por puertas xor y sea G una puerta que computa una función que depende de una entrada x_i; entonces existe un camino desde x hasta G.*

Demostración. Por brevedad, la demostración detallada se omite aquí y puede consultarse en [7]. □

Proposición 2.28 (Sustituciones afines complejas)**.** *Sea C un semicircuito consistente con variables de entrada $x_1, \ldots, x_n$; sea P una puerta que computa la función $f_P(x_1, \ldots, x_n) = x_1 \oplus \bigoplus_{i \in I} x_i \oplus c$ y que no es alcanzable desde las entradas por ninguna $\wedge$-puerta y sea $b \in \mathbb{F}_2$. Entonces es posible transformar C en otro circuito C' verificando las siguientes afirmaciones:*

1. *La operación que computa P es sustituida por una constante b.*
2. *El circuito C' tiene el mismo número de puertas que C más una puerta adicional P_0.*
3. *La variable x_1 queda desconectada en C'.*
4. *Todas las puertas comunes entre C y C' siguen computando la misma función en la restricción al subespacio $x_1 \oplus \bigoplus_{i \in I} x_i \oplus c \oplus b = 0$.*
5. *C' es un semicircuito consistente.*

Demostración. Como P no es alcanzable desde x_1 pasando por ninguna $\wedge$-puerta, el camino $x_1 \to P_1 \to \cdots \to P_{k-1} \to P_k = P$ —que sabemos que existe por el Lema 2.27— está formado únicamente por $\oplus$-puertas. Cada puerta P_i está alimentada por P_{i-1}, pero también por otra puerta T_i. Estas T_i pueden coincidir entre ellas o incluso ser alguna otra P_j. Puede verse una representación gráfica en la parte izquierda de la Figura 2.8, donde además se muestran posibles salidas A_i para cada puerta (pueden tener más) por claridad. La demostración consiste en probar que la transformación que se ve en la parte derecha de la representación es la que verifica las afirmaciones de la proposición. Formalmente, la transformación consiste en que P pasa a computar la operación constante b, x_1 es desconectado y sustituido por una puerta P_0 y cada P_i pasa a estar alimentado por P_{i+1} y T_{i+1}, incluyendo P_0.

Demostramos punto por punto las afirmaciones:

1. Trivial por construcción de la transformación.
2. Trivial por construcción de la transformación.
3. Trivial por construcción de la transformación.
4. Observemos que, inicialmente, cada puerta P_i computaba las siguientes funciones:

$$\begin{aligned}
P_1 &= x_1 \oplus T_1 && = x_1 \oplus T_1 \\
P_2 &= P_1 \oplus T_2 && = x_1 \oplus T_1 \oplus T_2 \\
&\vdots && \vdots \\
P_i &= P_{i-1} \oplus T_i && = x_1 \oplus T_1 \oplus \cdots \oplus T_i \\
&\vdots && \vdots \\
P_k &= P_{k-1} \oplus T_k && = x_1 \oplus \underbrace{T_k \oplus \cdots \oplus T_1}_{c \oplus \bigoplus_{i \in I} x_i}
\end{aligned}$$

Tras la transformación descrita, las puertas P_i pasan a computar las siguientes operaciones:

$$\begin{array}{lll}
P_k = b & = b & = b \\
P_{k-1} = P_k \oplus T_k & = b \oplus T_k & = \hat{x}_1 \oplus T_k \oplus \cdots \oplus T_1 \\
\vdots & \vdots & \vdots \\
P_i = P_{i+1} \oplus T_{i+1} & = b \oplus T_k \oplus \cdots \oplus T_{i+1} & = \hat{x}_1 \oplus T_{k-1} \oplus \cdots \oplus T_1 \\
\vdots & \vdots & \vdots \\
P_1 = P_2 \oplus T_2 & = b \oplus T_k \oplus \cdots \oplus T_2 & = \hat{x}_1 \oplus T_1 \\
P_0 = P_1 \oplus T_1 & = b \oplus \underbrace{T_k \oplus \cdots \oplus T_1}_{c \oplus \bigoplus_{i \in I} x_i} & = \hat{x}_1
\end{array}$$

lo que permite apreciar que cada una sigue computando la misma función en el nuevo subespacio, donde x_1 toma ahora el valor de $\hat{x_1} = \bigoplus_{i \in I} x_i \oplus c \oplus b$.

5. Por reducción al absurdo, supongamos que no. Esto implica que la combinación lineal de un subconjunto de filas de la nueva matriz es cero. La fila correspondiente a $P_k = b$ debe pertenecer a ella, pues de lo contrario solo tendríamos filas de la matriz de C con una coordenada adicional y sabemos que C era consistente. No obstante, esto significaría que si sumamos las líneas correspondientes del sistema (no solo de la matriz) para C, obtendríamos $G_k = \text{const} \oplus \bigoplus_{j \in J} x_j$, donde $1 \in J$ (obsérvese que x_1 fue reemplazada por Z en el nuevo sistema, y se canceló por nuestra suposición). Esto contradice la suposición del Lema de que G_k calcula la función $x_1 \oplus \bigoplus_{i \in I} x_i \oplus c$. Por lo tanto, la matriz para C' tiene rango completo.

□

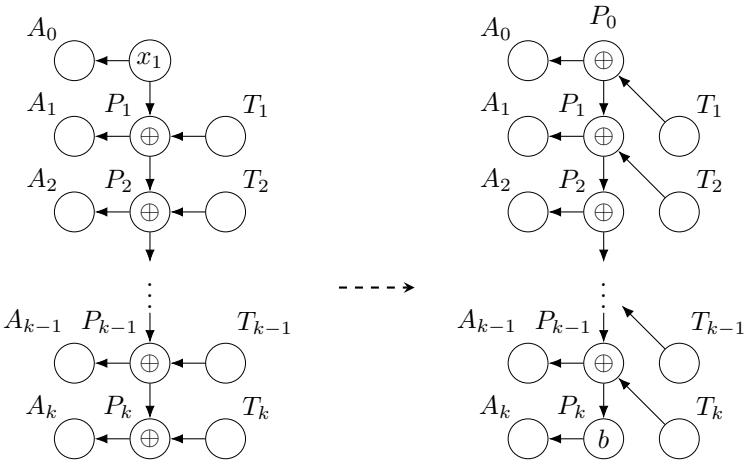


Figura 2.8: Representación visual dada en [7] para aclarar la transformación del circuito descrita en la demostración de la Proposición 2.28.

Los resultados que nos brindan la Proposición 2.16, la Proposición 2.17 y la Proposición 2.28 nos van a permitir realizar cómodamente las sustituciones de constantes en las entradas de las puertas siempre que esas entradas sean variables o puertas que computen funciones afines (con los requerimientos necesarios).

2.3.3. Normalización de circuitos

Una vez se impone alguna de las restricciones anteriores, el circuito pasa a contener puertas triviales o degeneradas que deben ser eliminadas. En el Teorema 2.19 nos bastó con describir cómo se transformaba el circuito tras la restricción para seguir computando la misma función restringida al nuevo subconjunto y hacerlo más sencillo en términos de la medida de complejidad.

Ahora la situación es distinta. Además de lo anterior, es importante que se conserven otras propiedades relevantes, como por ejemplo que el circuito resultante siga siendo consistente o que no se introduzcan demasiadas *troubled gates*. Por tanto, va a ser clave proporcionar un conjunto de reglas estandarizadas que nos permitan determinar de forma controlada cómo se realiza la transformación del circuito original en el circuito resultante de eliminar las puertas. De esta manera, cuando hagamos una distinción de casos similar a la que se hizo en el Teorema 2.19, y una vez identifiquemos cierta puerta que debe ser eliminada, podremos hacer afirmaciones como "aplicamos la regla 1 y se eliminan 3 puertas" con la garantía de que el circuito resultante seguirá verificando las propiedades que consideremos necesarias sin más comprobaciones. La Figura 2.9 muestra un ejemplo de la definición de cada regla.

Definición 2.29 (Circuito normalizado)**.** Sea C un circuito, sea P una puerta del circuito y sean I_1 e I_2 sus respectivas entradas. Decimos que el circuito C está *normalizado* si ninguna de las siguientes reglas de simplificación es aplicable:

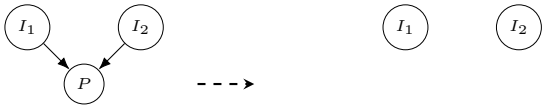
1. Si P tiene grado de salida 0 y no está etiquetado como la salida del circuito, entonces P se elimina.
2. Si P es trivial, entonces P se elimina y se sustituye[6] su constante en las puertas que alimente.
3. Si P es degenerada, entonces P se elimina recableando[7] dicha entrada a las puertas que P alimenta.
4. Si P tiene grado de salida 1 y alimenta una puerta Q que también está alimentada por una de las entradas de P, entonces P se elimina y se recablea la otra entrada a Q, posiblemente cambiando la operación de Q.
5. Si las entradas de P coinciden, entonces reemplazamos la operación $p(x, y)$ que computa P por la operación $p(x, x)$ dependiendo únicamente de una de las entradas. Tras esta transformación, P es trivial o degenerada y se aplica una de las reglas anteriores.

Este conjunto de reglas recibe el nombre de reglas de normalización.

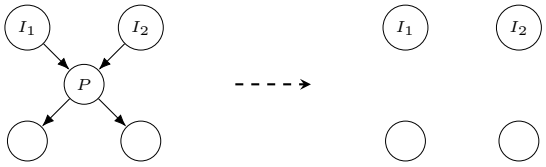
Si las reglas de normalización están bien definidas, nos van a permitir eliminar puertas conservando buenas propiedades sobre el circuito: las variables desconectadas siguen desconectadas, cualquier puerta interna P computa la misma función f_P que computaba, el semicircuito sigue siendo consistente, etc. Además, es imprescindible que estas no generen muchas *troubled gates* para no propiciar la aparición de demasiadas estructuras problemáticas. Como introdujimos al inicio de la sección, la nueva medida de complejidad tendrá en cuenta las *troubled gates* y si eliminar una puerta introduce

[6] Esto quiere decir que se hace una sustitución constante al estilo de la Proposición 2.16, pero considerando la salida P como una variable para el resto de subcircuito que sigue por debajo y haciendo la sustitución constante sobre esa variable.

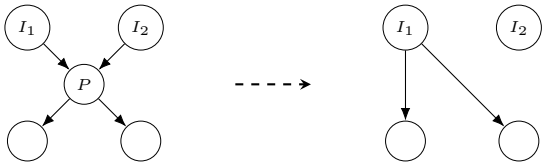
[7] Si P es degenerada, solo puede ser la identidad o la negación de una de sus entradas. Si es la identidad, efectivamente la transformación es únicamente reconectar la entrada correspondiente a las entradas alimentadas por P, pero si es la negación, además es necesario modificar las puertas a las que se conecta para trabajar con la negación aunque se alimente con la entrada sin negar.



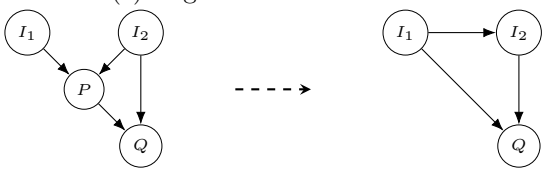
(a) Regla 1 de normalización



(b) Regla 2 de normalización



(c) Regla 3 de normalización



(d) Regla 4 de normalización

Figura 2.9: Representación visual recogida en [7] de las reglas de normalización.

muchas estructuras de este tipo lo que ganemos por un lado lo perderemos por el otro (a pesar de que podremos ajustar los pesos de la suma ponderada para minimizar el impacto).

Proposición 2.30. *Sea C un circuito no normalizado y C' el circuito resultante de aplicar una única regla de normalización, entonces:*

1. *C' tiene una puerta interna menos.*
2. *C' tiene como mucho cuatro troubled gates más.*
3. *Todas las puertas de C' siguen computando la misma función que en C.*
4. *Cualquier variable de entrada que no estuviese conectada con la salida de C sigue sin estarlo en C'.*
5. *Si C era un semicircuito consistente, C' lo sigue siendo.*

Demostración. Demostramos por puntos:

1. Por construcción, cada regla de la Definición 2.29 indica cómo eliminar cierta puerta P si se aplica, luego cada una elimina al menos una puerta.
2. Las puertas que pueden convertirse en una *troubled gate* en cada caso son I_1 e I_2 si son $\wedge$-puertas o bien las puertas que P alimenta si I_1 e I_2 son variables. En cualquier caso cada I_j puede crear a lo sumo dos *troubled gates*, luego como mucho pueden introducirse cuatro nuevas.

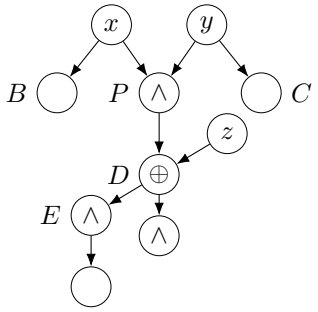


Figura 2.10: Ejemplo en [7] que muestra una situación en la que una sustitución cuadrática es la única forma de disminuir la medida lo suficiente.

3. De forma local, es claro que las modificaciones que se describen en las reglas —recableado de salidas y en ocasiones modificación de operaciones de las puertas— mantienen la función que computan las puertas involucradas. Si estas zonas locales donde se producen los cambios computan las mismas funciones, el resto de puertas que dependían de ellas también (por la recursividad de la función ϕ que define la función computada en una puerta).

4. Es el resultado directo del punto anterior. Si todas las puertas computan la misma función, en particular la puerta de salida también. Esto quiere decir que si su función era independiente de cierta variable x_i, tras las modificaciones también sigue siendo independiente de x_i.

5. Todas las puertas que no han sufrido cambios en sus operaciones siguen generando la misma ecuación. Las puertas a las que se les ha cambiado su operación (o las entradas) en realidad generan la misma ecuación, porque es el resultado de sustituir la ecuación que generaba P en la ecuación de estas puertas. Por tanto, el sistema de ecuaciones asociado sigue siendo consistente.

□

Por tanto, el conjunto de reglas de normalización anteriores nos proporciona un conjunto estandarizado de herramientas que pueden aplicarse una tras otra para disminuir la medida todo lo posible —una vez impuestas las restricciones correspondientes— y que permiten asegurar la conservación de las propiedades importantes que debe verificar nuestro circuito.

2.3.4. Sustituciones cuadráticas

La Definición 2.21 pone nombre a la situación problemática descrita en la distinción de casos al inicio de la Subsección 2.3.1, pero seguimos sin saber abordarla para eliminar las suficientes puertas o variables. Para este caso es evidente que las sustituciones afines no son suficiente si se quiere disminuir tanto como se necesita la medida del circuito. La solución a la que se llega en [7] es permitir imponer restricciones cuadráticas, es decir, aquellas del tipo $(x_i \oplus c_i) \wedge (x_k \oplus c_k) \oplus c_j$. La Figura 2.10 muestra una situación donde cualquier sustitución constante sobre x o y puede introducir una nueva *troubled gate* (la puerta que alimenta D), pero una sustitución cuadrática $z \leftarrow (x \oplus c_1) \wedge (y \oplus c_2) \oplus c_3$ apropiada puede eliminar P, D, E, F y la puerta que alimenta F.

En principio, esto supone un problema porque precisamente escogimos los *affine dispersers* por las garantías de no trivialidad ante restricciones afines que nos ofrecían y, a priori, las sustituciones

cuadráticas no tienen por qué[8] respetar esta propiedad. No obstante, si acotamos el alcance de las restricciones cuadráticas que podemos realizar y la manera de imponerlas, podemos obtener una clase más general de funciones de la que los *affine dispersers* son parte.

Definición 2.31 (Fuentes rdq)**.** Sea $X = \{x_1, \ldots, x_n\}$ un conjunto de variables de entrada, $\mathcal{R}$ un sistema de ecuaciones sobre X y $\mathfrak{P} = \{F, L, Q\}$ una partición de los índices de las variables de entrada en tres conjuntos disjuntos que verifica las siguientes condiciones:

- Para cada variable x_j con $j \in Q$, figura en $\mathcal{R}$ una ecuación del tipo

$$x_j = (x_i \oplus c_i) \cdot (x_k \oplus c_k) \oplus c_j$$

  donde $i, k \in F$.

- Las variables de la parte derecha de las ecuaciones cuadráticas se llaman *protegidas* y son disjuntas dos a dos.

- Para cada variable x_j con $j \in L$, figura en $\mathcal{R}$ una ecuación del tipo

$$x_j = \bigoplus_{i \in F_j \subseteq F} x_i \oplus \bigoplus_{i \in Q_j \subseteq Q} x_i \oplus c_j$$

Definimos una *read-once depth 2 quadratic source* —o *fuente rdq*— como cualquier subconjunto $R \subseteq \mathbb{F}_2^n$ definido por $\mathcal{R}$ que satisface las condiciones anteriores y su dimensión como $d = |F|$.

Definición 2.32 (*Rdq disperser*)**.** Una función booleana de $\mathbb{F}_2^n \to \mathbb{F}_2$ se dice que es un *rdq disperser* de dimensión d si no es constante en ninguna fuente rdq $R \subset \mathbb{F}_2^n$ de dimensión al menos d.

En realidad, hemos hecho una construcción muy similar a la de los *affine dispersers*: hemos definido un subconjunto de $\mathbb{F}_2^n$ y una función que no se trivializa en subconjuntos de este tipo de cierta dimensión, pero esta vez las restricciones que definen a estos subconjuntos no tienen por qué ser únicamente lineales, también tenemos cuadráticas siempre que se respeten ciertos requisitos sobre ellas. La utilidad de esta nueva construcción reside en que los *affine dispersers* son también *rdq dispersers* (con ciertos parámetros).

Proposición 2.33. *Sea $R \subseteq \mathbb{F}_2^n$ una fuente rdq de dimensión d, entonces R contiene un subespacio afín de dimensión al menos $d/2$.*

Demostración. Por cada restricción cuadrática $x_i = (x_i \oplus c_i)(x_k \oplus c_k) \oplus c_j$ del conjunto $\mathcal{R}$, podemos añadir una restricción afín $x_i = 0$. Esto permite transformar la restricción cuadrática en una restricción afín $x_j = c_i (x_k \oplus c_k) \oplus c_j$ y, por tanto, el subconjunto de $\mathbb{F}_2^n$ definido por la unión de estas y el conjunto de restricciones $\mathcal{R}$ es un subespacio afín. Por la definición de las fuentes rdq, las variables x_i a la derecha de las ecuaciones cuadráticas no aparecían como parte izquierda de ninguna de las ecuaciones lineales. Además, estas variables eran disjuntas dos a dos entre sí, por lo que el nuevo conjunto de restricciones tiene todas las lineales de $\mathcal{R}$ y todas las nuevas restricciones afines, y son linealmente independientes.

Las variables libres de la parte derecha de las ecuaciones cuadráticas son disjuntas dos a dos, por lo que inicialmente teníamos que $2|Q| \leq |F| = d$. En consecuencia, el número de restricciones afines introducidas de esta manera es como mucho $d/2$. □

[8] Por ejemplo, el conjunto de $n/2$ restricciones cuadráticas $x_1x_2 = x_3x_4 = \cdots = x_{n-1}x_n$ define un único punto así que cualquier función booleana es constante en él.

Corolario 2.34. *Sea $f : \mathbb{F}_2^n \to \mathbb{F}$ un affine disperser de dimensión d, entonces es rdq disperser de dimensión $2d$.*

Demostración. Si no fuese un rdq disperser de dimensión $2d$, habría alguna fuente rdq de dimensión $2d$ sobre la que fuese constante. Como esta fuente rdq contiene un subespacio afín de dimensión al menos d, el *affine disperser* sería constante sobre dicho subespacio, lo que no es posible. □

Cuando únicamente trabajábamos con restricciones afines, no había ningún impedimento a la hora de imponer varias de forma consecutiva. Una restricción afín se podía añadir al sistema formado por las impuestas hasta el momento y el nuevo subconjunto seguía siendo un subconjunto afín de $\mathbb{F}_2^n$. La situación ahora es diferente. Con la nueva definición de las fuentes rdq, no podemos imponer cualquier restricción cuadrática, pues, por ejemplo, debemos verificar la disparidad de variables de su parte derecha con las anteriores. Por este motivo, ahora sí necesitamos diseñar un proceso que, dada una fuente rdq, permita definir cómo son las restricciones que pueden añadirse conservando la propiedad de ser fuente rdq.

Proposición 2.35. *Sea $R \subseteq \mathbb{F}_2^n$ una fuente rdq y $\mathcal{R}$ el sistema de ecuaciones que la define, entonces:*

1. *Si x_j es una variable no protegida, cualquier ecuación*

$$x_j = (x_i \oplus c_i) \cdot (x_k \oplus c_k) \oplus c_j \text{ donde } i, k \in F$$

   donde x_i y x_k sean no protegidas mantiene la propiedad rdq cuando se añade a $\mathcal{R}$.

2. *Si x_j es una variable no protegida, cualquier ecuación*

$$x_j = \bigoplus_{i \in J} x_i \oplus c_j$$

   mantiene la propiedad rdq cuando se añade a $\mathcal{R}$.

3. *Si x_j es una variable protegida, cualquier sustitución constante $x_j = c$ mantiene la propiedad rdq cuando se añade a $\mathcal{R}$.*

4. *Si x_j es una variable protegida, cualquier sustitución afín $x_j = x_i$ para la otra x_i de la misma ecuación cuadrática mantiene la propiedad rdq cuando se añade a $\mathcal{R}$.*

5. *Si x_j es una variable protegida, cualquier sustitución afín $x_j = x_i \oplus 1$ para la otra x_i de la misma ecuación cuadrática mantiene la propiedad rdq cuando se añade a $\mathcal{R}$.*

Demostración. Demostramos cada afirmación por separado, contando con que $\mathcal{R}$ ya es un sistema de ecuaciones que verifica las condiciones de la definición de las fuentes rdq.

1. Como la variable x_j y sus variables de la parte derecha son no protegidas, la nueva ecuación no afecta en absoluto a las ecuaciones cuadráticas de $\mathcal{R}$. En todas las restricciones afines donde x_j fuese parte derecha como variable libre, ahora pasa a serlo formando parte del sumatorio de las variables cuadráticas y el resto se mantiene igual, luego siguen siendo ecuaciones afines válidas.

2. Como la variable x_j es no protegida, la nueva ecuación no afecta en absoluto a las ecuaciones cuadráticas de $\mathcal{R}$. Todas las ecuaciones afines de las que x_j participase en el sumatorio de variables libres se transforman absorbiendo las libres y las cuadráticas de la nueva ecuación en sus correspondientes sumatorios y absorbiendo la constante en la constante de la ecuación.

3. Como la variable es protegida, es la parte derecha de una ecuación cuadrática. Adicionalmente, puede ser la parte derecha de algunas ecuaciones afines y no puede ser nada más. Cualquier ecuación afín de la que participe se mantiene afín tras sustituir una constante en la variable, pues el cambio consiste en mover una variable del sumatorio libre a ser absorbida por la constante independiente y, como no hay más requisitos para este tipo de ecuaciones, $\mathcal{R}$ seguiría definiendo una fuente rdq por esta parte. La ecuación cuadrática $x_i = (x_j \oplus c_j) \cdot (x_k \oplus c_k) \oplus c_i$ de la que participa se transforma en la restricción afín $x_i = c_i$ si $c \oplus c_j = 0$ o en $x_i = x_k \oplus \tilde{c}$ si $c \oplus c_j = 1$. Esto elimina una ecuación cuadrática, luego si $\mathcal{R}$ ya cumplía los requisitos asociados a estas, entonces ahora también los cumple, y reescribe todas las ecuaciones afines de las que participaba x_i pasando del sumatorio de variables cuadráticas a ser absorbida por la constante, y posiblemente añadiendo una variable al sumatorio de las libres. Por tanto, la restricción afín se mantiene como tal. En consecuencia, $\mathcal{R}$ sigue definiendo una fuente rdq.

4. El razonamiento es exactamente el mismo al del caso anterior, pues es sencillo comprobar que esta vuelve a transformarse en una ecuación afín del estilo $x_i = x_k \oplus \tilde{c}$ o $x_i = \tilde{c}$.

5. El razonamiento es exactamente el mismo al del caso anterior.

□

Este último resultado nos va a permitir construir fuentes rdq de manera incremental, de manera que podremos ir imponiendo restricciones con la certeza de que siempre nos moveremos en subconjuntos que lo sean. Observemos que la situación es muy similar a la que teníamos con los *affine dispersers* en la sección anterior: siempre podemos hacer sustituciones afines sin dificultad. Las sustituciones delicadas son aquellas que son cuadráticas o que afectan a variables protegidas en ecuaciones cuadráticas, que deben hacerse correctamente para preservar la propiedad de fuente rdq del conjunto. Además, veremos en la demostración del resultado principal del artículo que no es necesario redescribir cómo son las reglas de normalización, pues por la forma de imponer estas restricciones cuadráticas estas se podrán traducir en varias restricciones afines sucesivas.

2.3.5. Demostración del resultado principal

La intuición que se mostró en la Subsección 2.3.1 puso de relieve la incapacidad de disminuir la medida lo suficiente con las herramientas que se tenían entonces. Por una parte, teníamos la necesidad de sustituir constantes en $\oplus$-puertas sin requisitos adicionales, lo cual quedó resuelto gracias a los circuitos cíclicos. Por otra, para abordar el caso desfavorable de las *troubled gates* hemos acotado su número y habilitado la posibilidad de hacer restricciones cuadráticas junto con las afines —sujetas a ciertas condiciones—.

El objetivo de este último desarrollo es poder disminuir todo lo posible la medida de complejidad. Sin embargo, como no siempre vamos a conseguir disminuir tanto como querríamos la medida —pues entonces la cota sería $4n - o(n)$—, medir la disminución como la suma de puertas y variables que se eliminan ya no es suficiente. Por ejemplo, una sustitución que elimine suficientes puertas, pero que añada muchas *troubled gates* será peor que una que haga lo mismo sin añadir estas.

La idea del argumento de [7] es tener en cuenta además de las puertas y las variables todos estos instrumentos nuevos que hemos introducido para construir con ellos una suma ponderada como métrica.

Definición 2.36 (Medida de complejidad)**.** Sea C un circuito y R una fuente rdq. Definimos la *medida de complejidad* del circuito restringido a la fuente como

$$\mu(C, R) = g + \alpha_Q \cdot q + \alpha_T \cdot t + \alpha_I \cdot i$$

donde g es el número de puertas internas, q es el número de restricciones cuadráticas en R, t es el número de *troubled gates*, i es el número de entradas influyentes —las variables que o bien están protegidas o bien están conectadas al circuito— y los α_k son constantes con ciertos valores.

El hilo argumental de la demostración de la cota específica es el mismo que el del artículo anterior: demostrar que cada transformación del circuito disminuye lo suficiente la medida. La diferencia es que el conteo ya no se hace exclusivamente de puertas y variables. Por ejemplo, si el objetivo fuese disminuir en 5 unidades la medida de complejidad, una sustitución que elimine 5 puertas a través de una sustitución cuadrática que además introduce una nueva troubled gate disminuiría la medida una cantidad $\delta = \alpha_I + 5\beta - \alpha_Q - \alpha_T$. De esta manera, al no determinar aún el valor de las constantes, podemos centrarnos en disminuir todo lo posible la medida en cada caso contabilizando todos estos elementos y fijar a posteriori los valores de las constantes para dar más peso a unos u otros.

Por tanto, si se tiene probada una cota superior para t e i y una cota inferior para μ, puede recurrirse al argumento planteado en el Teorema 2.14 para dar una cota inferior para el tamaño del circuito. Esta es precisamente la demostración que se plantea en [7]. La cota superior para las variables influyentes es evidente: n, pues es la misma que ya vimos en el Corolario 2.20. La cota superior para las *troubled gates* se prueba en el siguiente lema.

Lema 2.37. *Sea $f : \mathbb{F}_2^n \to \mathbb{F}$ un affine disperser de dimensión d y C un circuito que lo computa; entonces el número de troubled gates en C es menor que $\frac{n+5d}{2}$.*

Demostración. Por brevedad, la demostración detallada se omite aquí y puede consultarse en [7]. □

Lema 2.38. *Sea $f : \mathbb{F}_2^n \to \mathbb{F}$ un affine disperser de dimensión d y C un circuito que lo computa; entonces las reglas de normalización disminuyen la medida de complejidad en una cantidad $1 - 4\alpha_T$, que denominaremos β.*

Demostración. Cada regla de normalización elimina al menos 1 puerta e introduce como mucho 4 nuevas *troubled gates*, luego:

$$\mu(C, R) - \mu(C', R) = g + \alpha_Q q + \alpha_T t + \alpha_I i - (g - 1 + \alpha_Q q + \alpha_T(t + 4) + \alpha_I i) = 1 - 4\alpha_T$$

□

A continuación, se muestra el resultado que hace correctamente la distinción de casos que ya planteamos en la introducción, pero esta vez utilizando las nuevas herramientas desarrolladas. Esta vez, debemos demostrar que la medida disminuye lo suficiente; sin embargo, "suficiente" ya no se refiere a 5 unidades entre variables y puertas, sino al máximo posible en cada caso teniendo en cuenta también las restricciones cuadráticas y las *troubled gates*. Después, se ajustarán las constantes de la medida de complejidad para que se aprovechen todo lo posible las disminuciones conseguidas.

Teorema 2.39. *Sea $f : \mathbb{F}_2^n \to \mathbb{F}$ un rdq disperser de dimensión d, sea R una fuente rdq de dimensión $s \geq d + 2$ y sea C un semicircuito óptimo*[9] *consistente que computa $f|_R$; entonces existe otra fuente rdq R' de dimensión $s' < s$ y un semicircuito consistente C' que computa $f|_{R'}$ tal que:*

$$\mu(C, R) - \mu(C', R') \geq \delta(s - s') \text{ donde } \delta = \alpha_I + \min\left(\frac{\alpha_I}{2}, 4\beta, 3 + \alpha_T, 2\beta + \alpha_Q, 5\beta - \alpha_Q, \frac{5\beta + \alpha_Q}{2}\right)$$

[9] Con el menor valor de la medida de complejidad $\mu(C, R)$.

Nota 2.40. Los valores del interior del mínimo surgen del tipo de disminuciones que vamos a obtener. Todas suman un α_I porque cualquier sustitución elimina al menos una variable que deja de ser influyente (porque no es o no permanece como protegida). El añadido es a causa de los siguientes casos:

- $\alpha_I/2$: si introducimos el α_I exterior, vemos que quiere decir que la disminución viene de dos sustituciones afines que eliminan al menos 3 variables afines, es decir, $3/2\alpha_I$ por sustitución.
- 4β: la sustitución afín elimina al menos cuatro puertas internas (añadiendo posiblemente más *troubled gates*).
- $3 + \alpha_T$: la sustitución afín elimina 3 puertas internas y una *troubled gate* sin introducir más.
- $2\beta + \alpha_Q$: la sustitución afín elimina dos puertas y una restricción cuadrática (añadiendo posiblemente más *troubled gates*).
- $5\beta - \alpha_Q$: la sustitución cuadrática (por eso se resta α_Q) elimina al menos 5 puertas (añadiendo posiblemente más *troubled gates*).
- $\frac{5\beta+\alpha_Q}{2}$: si introducimos el α_I exterior, vemos que quiere decir que la disminución viene de dos sustituciones afines que eliminan 5 puertas internas (añadiendo posiblemente más *troubled gates*) y una restricción cuadrática.

Demostración. Como las reglas de normalización ni modifican la fuente rdq sobre la que está restringida la función ni aumentan la medida de complejidad, podemos asumir sin pérdida de generalidad que siempre estamos en la situación de un circuito normalizado.

A modo de ilustración del conjunto general de la prueba y debido a la extensión de casos a distinguir, mostramos solo una parte de la distinción. La prueba completa puede consultarse en [7]. Cuando se omitan el resto de casos se indicará con "En [7]" en el punto correspondiente.

1. Hay alguna variable protegida q que o bien alimenta una $\wedge$-puerta o bien alimenta dos puertas internas:

   Una sustitución constante en q hace que haya una variable influyente menos, elimina una restricción cuadrática y elimina al menos dos puertas internas: o bien trivializa la $\wedge$-puerta y al menos degenera a sus sucesoras o bien al menos degenera las dos puertas internas. Luego $\Delta\mu \geq 2\beta + \alpha_Q + \alpha_I > \delta$.

2. Hay alguna variable protegida de grado de salida 0:

   Una sustitución constante en la otra variable de la misma ecuación cuadrática elimina la restricción cuadrática y dos variables dejan de ser influyentes. Luego $\Delta\mu \geq 2\alpha_I + \alpha_Q > \delta$.

3. Hay alguna variable x que alimenta una $\wedge$-puerta T y la suma de los grados de salida de x y T es al menos 4:

   Una sustitución constante en x que trivialice T elimina al menos cuatro puertas. En efecto, trivializamos T y degeneramos al menos a los descendientes de x y de T. Si alguno de estos sucesores coincide entre ambos, ese se trivializa en lugar de solo degenerarse —porque tanto T como x son constantes—, así que también se degenera el sucesor del sucesor. Luego $\Delta\mu \geq 4\beta + \alpha_I > \delta$.

4. Hay alguna $\wedge$-puerta T alimentada por dos variables x e y con alguna de ellas (digamos x) de grado exactamente 1:

   Una sustitución constante en y que trivialice T elimina T y desinfluencia x e y (porque no eran protegidas). Luego $\Delta\mu \geq \beta + 2\alpha_I > \delta$.

5. Hay alguna $\wedge$-puerta alimentada por dos variables x e y y debido a los casos anteriores, solo puede ser una *troubled gate*:

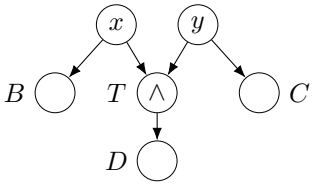


Figura 2.11: Notaciones del caso 5 en la distinción de casos de la demostración del Teorema 2.39.

   Como el circuito está normalizado, $B \neq D$ y $C \neq D$. Una sustitución constante en x (o en y) que trivialice T elimina al menos tres puertas. Si la sustitución elimina alguna más, tendríamos $\Delta\mu \geq 4\beta > \delta$ y estaría hecho. Si no, se eliminan 3 puertas, pero también una *troubled gate*. Por tanto, si la sustitución no añade otra tendríamos $\Delta\mu \geq 3\beta + \alpha_T > \delta$. En consecuencia, en la siguiente distinción de subcasos, nos enfrentaremos a situaciones donde solo tres puertas pueden ser eliminadas y se introduce al menos una *troubled gate*.

   a) Si ocurre que $B = C$, entonces:
   Se puede trivializar tanto P como B con una sustitución constante apropiada sobre x o y o con una sustitución afín $y = x \oplus c$ para una constante c apropiada. Entonces, como x e y son no protegidas, dejan de ser influyentes. Luego $\Delta\mu \geq 3\beta + 2\alpha_I > \delta$.

   b) Si ocurre que D alimenta a B y a C:
   La única *troubled gate* que puede surgir viene de la situación en la que D está alimentada por una variable u que es pasada a la puerta E.

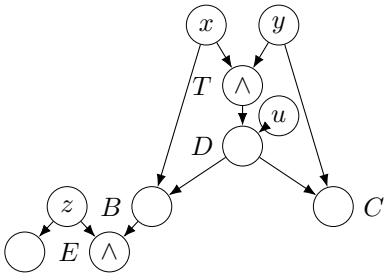


Figura 2.12: Notaciones del caso 5.b) en la distinción de casos de la demostración del Teorema 2.39.

   Necesariamente el grado de salida de D tiene que ser como mucho dos, porque de otra manera u sería una variable con grado de salida 3 cuando pasase a alimentar E y entonces no formaría una *troubled gate*. Por tanto, u pasa a E a través de B. Si el grado de salida de B es mayor que 2, entonces necesariamente $C = E$ o B alimenta a C (de otra manera u sería una variable con grado de salida 3). Ninguna de las dos opciones es posible, pues cualquiera de las dos implicaría que $B = D$. Por tanto B debe tener grado de salida 1. En ese caso, podemos hacer una sustitución constante sobre z y sobre y de manera que x deja de ser influyente. Luego $\Delta\mu \geq 3\alpha_I$ empleando dos sustituciones.

   c) En [7]

6. Hay una $\wedge$-puerta P que tiene como entradas una variable x de grado de salida 2 y una puerta interna Q de grado de salida 1:

   La variable x tiene que ser no protegida por el caso 1 y no puede alimentar Q por la regla 4 de normalización. Una sustitución constante en x que trivialice P, elimina P y sus sucesores, el otro sucesor de x y la puerta Q por tener grado de salida 0 y no ser la puerta de salida. Además, la variable x deja de ser influyente. Luego $\Delta\mu \geq 4\beta + \alpha_I > \delta$.

7. Hay una $\wedge$-puerta con una puerta interna Q como entrada. Si la otra entrada es una puerta interna E con grado de salida mayor, cambiamos los roles de Q y E.

   Por cómo se ha escogido P, la puerta Q es una $\oplus$-puerta que computa una función lineal $c \oplus \bigoplus_{i\in I} x_i$ —que además cumple que $I \neq \emptyset$ porque, si no, sería constante y habría sido eliminada por normalización—. Una sustitución constante en esta puerta viene dada por el procedimiento de la Proposición 2.28, que depende de que exista cierta variable no protegida x_j con $j \in I$ (que llamaremos sin pérdida de generalidad x_1).

   a) Existe dicha variable x_1:

   En este caso, podemos añadir la restricción $x_1 = b \oplus c \oplus \bigoplus_{i\in I\setminus\{1\}} x_i$ a nuestra fuente rdq para la constante b deseada en Q (la que trivializa P). La reconstrucción del circuito descrita en la Proposición 2.28 indica claramente cómo transformarlo para que Q sea trivial y x_1 quede desconectada del circuito; sin añadir *troubled gates* ni modificar grados de salida ni entrada de otras puertas. En consecuencia, aplicar las reglas de normalización hace que x_1 deje de ser influyente, elimina Q, P y todos los descendientes de ambas y, en el caso en que el grado de salida de E sea 1, el grado de salida pasa a ser cero y E pasa a ser una variable no influyente o una puerta inútil dependiendo de lo que fuera. Solo falta cuantificar la disminución de la medida:

   1) Si Q tiene grado de salida 2 o más:
      Trivializar Q elimina Q, E, el otro descendiente de Q y el descendiente de E (además de haber hecho que x_1 deje de ser influyente). Luego $\Delta\mu \geq 4\beta + \alpha_I > \delta$.
   2) Si Q tiene grado de salida 1 y E es una variable:
      Por el Caso 6, E tiene que tener grado 1 y por el Caso 1 tiene que ser no protegida. Además, no puede ser x_1 porque como tiene grado de salida 1 y ya alimenta la $\wedge$-puerta P, así que no puede alimentar también a Q como hace x_1. En consecuencia, la reconstrucción de la proposición no afecta a E. Por tanto, tras eliminar P y Q tanto x_1 como E dejan de ser influyentes. Luego $\Delta\mu \geq 2\beta + 2\alpha_I > \delta$.
   3) Si Q tiene grado de salida 1 y E es una puerta interna:
      Sabemos que E tiene que tener grado de salida 1 (si no, habríamos cambiado roles de Q y E como dijimos al inicio del Caso 7). De nuevo, E queda intacta tras la reconstrucción del circuito por la sustitución constante en la puerta Q (solo tiene un sucesor y es una $\wedge$-puerta). Tras la sustitución, se eliminan dos sucesores de Q, al menos uno de la $\wedge$-puerta y E se desconecta del circuito. Luego $\Delta\mu \geq 4\beta + \alpha_I > \delta$.

   b) Todas las variables x_i con $i \in I$ son protegidas:

   1) Las dos entradas de Q son variables y pertenecen a la misma ecuación cuadrática $w = (x_j \oplus c_j) \cdot (x_k \oplus c_k) \oplus c$:
      La sustitución afín $x_k \oplus \tilde{c}$ en x_j que trivializa P es una sustitución afín válida sobre una variable protegida según la Proposición 2.35. Esta sustitución elimina la restricción cuadrática, la puerta Q, la puerta P y sus descendientes. Luego $\Delta\mu \geq 3\beta + \alpha_Q + \alpha_I > \delta$.
   2) En [7]

□

Teorema 2.41. *Sea $f : \mathbb{F}_2^n \to \mathbb{F}$ un rdq disperser de dimensión d y C un semicircuito que lo computa, sean ciertas constantes $\alpha_Q, \alpha_T, \alpha_I \geq 0$ y $\alpha_T \leq 1/4$; entonces*

$$\mu(C, \emptyset) \geq \delta(n - d - 2)$$

Demostración. Probamos que, para un circuito óptimo C que compute $f|_R$ sobre cierta fuente rdq de dimensión s, se cumple que $\mu(C, R) \geq (s - d - 2)$, y lo hacemos por inducción sobre s. Una vez probado, bastaría afirmar que $\mathbb{F}_2^n$ es una fuente rdq de dimensión n.

El resultado es cierto para todo $s \leq d + 2$ porque la medida μ siempre es positiva. Para $s > d + 2$, suponemos cierto el resultado para todas las fuentes rdq de dimensión menor que s. Sea R una fuente rdq de dimensión s; por el Teorema 2.39, sabemos que existen un circuito C' y una fuente R' de dimensión s' tales que $\mu(C, R) - \mu(C', R') \geq \delta(s - s')$, luego:

$$\mu(C, R) \geq \mu(C', R') + \delta(s - s') \geq \delta(s' - d - 2) + \delta(s - s') = \delta(s - d - 2)$$

□

Corolario 2.42. *Sea $f : \mathbb{F}_2^n \to \mathbb{F}$ un affine disperser de dimensión $d \in o(n)$ y C un semicircuito que lo computa, sean ciertas constantes $\alpha_Q, \alpha_T, \alpha_I \geq 0$ y $\alpha_T \leq 1/4$; entonces*

$$|C| \geq \left(\delta - \frac{\alpha_T}{2} - \alpha_I\right) n - o(n)$$

Demostración. Por ser un *affine disperser*, tenemos que $q = 0$ en la medida de complejidad. Además, $i \leq n$ y $t \leq \frac{n}{2} + \frac{5d}{2}$ por el Lema 2.37; entonces:

$$\begin{aligned} g &= \mu - \alpha_Q q - \alpha_T t - \alpha_I i \\ &= \mu - \alpha_T t - \alpha_I i \\ &> \delta(n - 2d - 2) - \alpha_T \left(\frac{n}{2} + \frac{5d}{2}\right) - \alpha_I n \\ &= \left(\delta - \frac{\alpha_T}{2} - \alpha_I\right) n - \left(2\delta + \frac{5\alpha_T}{2}\right) d - 2\delta \\ &= \left(\delta - \frac{\alpha_T}{2} - \alpha_I\right) n - o(n) \end{aligned}$$

□

Teorema 2.43. *Sea $f : \mathbb{F}_2^n \to \mathbb{F}$ un affine disperser de dimensión d y C un semicircuito que lo computa; entonces*

$$|C| \geq \left(3 + \frac{1}{86}\right) n - o(n)$$

Demostración. El teorema se prueba calculando la configuración de constantes que nos proporciona el mayor valor del coeficiente que acompaña a n en el Corolario 2.42 sujeta a las restricciones impuestas sobre los valores de estas. Esto conforma el siguiente problema de programación lineal

$$\text{máx}\, z = \delta - \frac{\alpha_T}{2} - \alpha_I$$

Sujeto a las restricciones:

$$\begin{aligned} \beta + 4\alpha_T &= 1 \\ \alpha_Q, \alpha_T, \alpha_I, \beta &\geq 0 \\ \delta &\leq \alpha_I + \text{mín}\left(\frac{\alpha_I}{2}, 4\beta, 3 + \alpha_T, 2\beta + \alpha_Q, 5\beta - \alpha_Q, \frac{5\beta + \alpha_Q}{2}\right) \end{aligned}$$

cuyos valores óptimos vienen dados por

$$\begin{aligned} \alpha_T &= \frac{1}{43} \\ \alpha_Q &= 1 + 22\alpha_T = \frac{65}{43} \\ \alpha_I &= 6 + 2\alpha_T = 6 + \frac{2}{43} \\ \beta &= 1 - 4\alpha_T = \frac{39}{43} \\ \delta &= 9 + 3\alpha_T = 9 + \frac{3}{43} \end{aligned}$$

□

2.4. Cota de $3{,}1n - o(n)$ para los *affine dispersers*

Finalmente, en este último artículo [15], Li y Yang dan el último empujón a la cota anterior para llegar a la mejor cota encontrada hasta el momento.

De nuevo, este artículo continúa la línea planteada por los dos anteriores. La novedad con respecto a [7] es que aquí se hace un estudio más fino de las *troubled gates* para demostrar que no siempre son problemáticas y que no se introducen tantas como se asumía anteriormente en las simplificaciones del circuito.

Por un lado, esto permite reducir el número de *troubled gates* que introducen las reglas de normalización, luego la disminución asociada a lo que denominamos β aumenta. Por otro, permite hacer un estudio más exhaustivo en la distinción de casos que argumente caso a caso qué simplificaciones pueden introducir o no *troubled gates* en lugar de afirmar que cada puerta eliminada introduce a lo sumo dos. Estos dos elementos juntos permiten reformular el teorema que demuestra la cota y, tras buscar los valores de los parámetros adecuados como se hizo en el Corolario 2.42, aumentar la cota inferior hasta $3{,}1n - o(n)$.

2.4.1. Análisis fino de las *troubled gates* en las reglas de normalización

La discusión de casos de [7] tiene como principal cuello de botella las *troubled gates*. En efecto, allí se prueba que siempre se eliminan 4 puertas salvo en el caso en el que encontramos una estructura de este tipo, donde no es posible eliminar más de tres. La idea fundamental entonces fue incorporar el número de estructuras de este tipo —entre otros instrumentos adicionales— a la medida de complejidad; de esta manera, la desaparición —o no introducción— de este tipo de estructuras también podían contarse como parte de la disminución de la medida tras hacer las sustituciones. De este modo, se

pudo ajustar de forma óptima los pesos asociados a cada elemento de la medida de complejidad para elevar ligeramente por encima de $3n - o(n)$ la cota inferior.

En este artículo, los autores demuestran que la cota superior de la Proposición 2.30 para el número de *troubled gates* que pueden introducirse tras la normalización del circuito es, en realidad, una sobreestimación. En la Proposición 2.30, se argumentó que cada entrada I_1 e I_2 de la puerta P a eliminar introduce como mucho dos nuevas *troubled gates*, luego cada eliminación añadía como mucho 4 de estas. Sin embargo, en este artículo se prueba que una entrada I solo puede introducir dos *troubled gates* si se trata de una variable cuyo grado de salida disminuye a 2. En ese caso, las dos *troubled gates* son adyacentes y esta estructura en realidad puede considerarse una única estructura problemática y no dos. El motivo quedará claro más adelante.

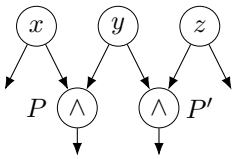


Figura 2.13: Ejemplo de la estructura local denominada *troubled pair*.

Definición 2.44 (*Troubled pair*)**.** Sean P y P' dos *troubled gates*, decimos que son *adyacentes* si al menos[10] comparten una variable de entrada y, en ese caso, definimos un *troubled pair* como el par (P, P').

Nota 2.45. A las variables de entradas compartidas las denotaremos por variables internas y a las no compartidas por variables frontera. En caso de no haber variables frontera, denominaremos de forma especial a ese *troubled pair* refiriéndonos a él como compacto.

Como a partir de ahora estos paquetes funcionan como una única *troubled gate*, hay que redefinir la forma de incorporar estas estructuras en la medida de complejidad del circuito. De este modo, ya no contaremos *troubled gates* sin más, sino paquetes de este tipo más *troubled gates* que verdaderamente estén aisladas. Esto motiva las siguientes definiciones.

Definición 2.46 (Empaquetado)**.** Sea C un circuito y $\mathcal{T}$ el conjunto de todos los *troubled pairs*. Un subconjunto $\mathcal{P} \subset \mathcal{T}$ es un *empaquetado* de C si

$$\forall (P_1, P_1'), (P_2, P_2') \in \mathcal{P} : \{P_1, P_1'\} \cap \{P_2, P_2'\} = \emptyset$$

y a cada par $A \in \mathcal{P}$ se le denomina *paquete*.

Definición 2.47 (Verdadera *troubled gate*)**.** Sea C un circuito y $\mathcal{P}$ un empaquetado suyo. Decimos que una *troubled gate* T es *verdadera* (con respecto a $\mathcal{P}$) si no pertenece a ningún paquete de $\mathcal{P}$.

Definición 2.48 (Potencial)**.** Sea C un circuito y $\mathcal{P}$ un empaquetado suyo. Definimos el *potencial* $\Phi(C, P)$ como la suma del número de paquetes y del número de verdaderas *troubled gates*.

Definición 2.49 (Medida de complejidad)**.** Sea C un circuito, $\mathcal{P}$ un empaquetado y R una fuente rdq. Definimos la *medida de complejidad* del circuito como

$$\mu(C, \mathcal{P}, R) = g + \alpha_I \cdot i + \alpha_Q \cdot q + \alpha_\phi \cdot \Phi(C, \mathcal{P})$$

donde g es el número de puertas internas, q es el número de restricciones cuadráticas de R y los α_k son constantes con ciertos valores.

[10]Nótese que el caso en el que comparten ambas también entra dentro de la definición.

Lema 2.50. *Sea C un circuito que computa un affine disperser de dimensión d, entonces para cualquier empaquetado $\mathcal{P}$ se tiene*

$$\Phi(C, P) \leq \frac{n + 5d}{2}$$

Demostración. Tal y como se ha definido el potencial, este es igual al número de *troubled gates* en total menos el número de paquetes. Como en el Lema 2.37 se demostró la cota superior del número de *troubled gates* en *affine dispersers*, con más razón el potencial tendrá la misma cota. □

La ventaja que otorga esta nueva definición de lo que son las estructuras problemáticas de la distinción de casos es que permite argumentar que como mucho se pueden introducir dos utilizando las reglas de normalización de circuitos. En efecto, el potencial puede aumentar con las transformaciones del circuito tras las eliminaciones de puertas únicamente de dos maneras: o bien se introducen nuevas verdaderas *troubled gates* o paquetes, o bien se deshace algún paquete en dos *troubled gates* disjuntas. Este último caso no es posible —como muestra el siguiente resultado—, así que podemos considerar únicamente el primer caso en la prueba para las reglas de normalización.

Proposición 2.51. *Si un paquete (P_1, P_2) es destruido después de aplicar una regla de normalización, entonces como mucho una puerta se mantiene como troubled gate.*

Demostración. Supongamos que no: asumamos que (P_1, P_2) es destruido y ambas puertas siguen siendo *troubled gates*. Obligatoriamente la o las variables de entrada compartida o compartidas por las puertas tienen que haber cambiado sus descendientes, pues de lo contrario ambas puertas seguirían siendo *troubled gates* y compartirían variables de entrada, luego el par no se habría deshecho. Sin embargo, las reglas de normalización solo cambian los descendientes de las puertas que alimentan a las puertas que se eliminan, luego al menos G_1 o G_2 es eliminada, lo que es una contradicción. □

Lema 2.52 (Lema de Normalización)**.** *Sea C un circuito no normalizado y C' el circuito resultante de aplicar una única regla de normalización de las descritas en la Definición 2.29; entonces la medida de complejidad de la Definición 2.49 disminuye al menos $1 - 2\alpha_\Phi$ para la regla 1 y $1 - \alpha_\Phi$ para todas las demás.*

Demostración. Presentamos la distinción para cada una de las reglas:

1. Las nuevas *troubled gates* que puede introducir la regla 1 son: o bien las propias I_1 o I_2 si son puertas internas que se han transformado en *troubled gates*, o bien las puertas que estas alimenten si son variables de entrada. En el primer caso solo se introducen como mucho dos *troubled gates*. En el segundo, conviene mencionar que como las *troubled gates* están alimentadas por variables de grado de salida 2, tanto I_1 como I_2 pueden introducir como mucho dos nuevas *troubled gates* cada una. Sin embargo, si realmente se introducen dos, podemos formar un paquete con ellas porque son adyacentes. De esta manera, en cualquiera de los dos casos tanto I_1 como I_2 introduce como mucho una *troubled gate* o un paquete, luego $\Delta\Phi \geq 2$ y $\Delta\mu \leq 1 - 2\alpha_\Phi$.

2. Las nuevas *troubled gates* que puede introducir la regla 2 son o bien la propia I_1, o bien las puertas que alimenta. Usando el mismo argumento que en el caso anterior, tenemos que $\Delta\Phi \geq 1$ así que $\Delta\mu \geq 1 - \alpha_\Phi$.

3. Las nuevas *troubled gates* que puede introducir la regla 3 son o bien la propia I_1 —únicamente cuando G es la puerta de salida—, o bien las puertas que alimenta. Usando el mismo argumento que en los casos anteriores, tenemos que $\Delta\Phi \geq 1$ así que $\Delta\mu \geq 1 - \alpha_\Phi$.

4. Las nuevas *troubled gates* que puede introducir la regla 3 son o bien la propia I_2 si es una puerta interna, o bien las puertas que alimenta si es una variable de entrada. Usando el mismo argumento que en los casos anteriores, tenemos que $\Delta\Phi \geq 1$ así que $\Delta\mu \geq 1 - \alpha_\Phi$.

5. Como esta regla es un paso que solo afecta a una puerta y luego se reduce a los casos anteriores, está probado.

□

Nota 2.53. Siguiendo una notación similar a la utilizada anteriormente, llamaremos β_0 a la cantidad $1 - 2\alpha_\Phi$ y β_1 a la cantidad $1 - \alpha_\Phi$.

Por tanto, si tenemos en cuenta en la suma ponderada el potencial en lugar de las *troubled gates*, la medida disminuye más gracias a aplicar las reglas de normalización que lo que disminuía en [7]. Esto ya es suficiente como para que haya interés en recalcular de nuevo la solución al problema de programación lineal planteado en el Teorema 2.43, pues en las mismas circunstancias hemos conseguido que β sea $1 - 2\alpha_\Phi$ (en el peor de los casos) en lugar de $1 - 4\alpha_T$. Sin embargo, aún tenemos margen de mejora con respecto al artículo anterior y es lo que motiva el apartado que viene a continuación.

2.4.2. Demostración del resultado principal

Aunque pudiera parecer que el término *verdadera troubled gate* únicamente servía para diferenciar las *troubled gates* que formaban un paquete de las que no, en realidad es bastante más acertado. En realidad, los paquetes no son estructuras problemáticas como lo son las verdaderas *troubled gates*, de ahí el nombre. El motivo es que la disminución de la medida al eliminar un paquete es mayor que la disminución de la medida de la distinción de casos del teorema —el δ que hemos manejado hasta ahora—; por tanto, de la misma forma que consideramos sin pérdida de generalidad el circuito completamente normalizado por defecto, también podremos considerarlo sin paquetes en la distinción de casos del teorema.

Proposición 2.54 (Eliminación de *troubled pair*)**.** *Sea C un circuito que computa una función $f : \mathbb{F}_2^n \to \mathbb{F}_2$ restringida a una fuente rdq R, sea $\mathcal{P}$ un empaquetado no trivial de C y supongamos que f es no trivial bajo dos sustituciones; entonces es posible realizar ciertas sustituciones de manera que la disminución de la medida de complejidad es al menos*

$$\delta_p = \text{mín}\left\{\frac{3}{2}\alpha_I, 3\beta_1 + \alpha_I + \alpha_Q\right\}$$

Demostración. Sea $(P_1, P_2) \in \mathcal{P}$, supongamos primero que el par no comparte ambas variables de entrada. Denotando por y a la variable compartida y por x y z a las otras dos, tenemos que:

1. Si y es protegida, hacemos la sustitución constante adecuada para que P_1 sea trivial. Esto trivializa P_1 y degenera P_2 y el único descendiente de P_1, luego en principio se eliminan 3 puertas en total y $\Delta\Phi \leq 3$. Como la sustitución de una constante en una variable protegida elimina una ecuación cuadrática y una variable influyente, entonces el decremento global es $\Delta\mu \geq 3 - 3\alpha_\Phi + \alpha_I + \alpha_Q$.

2. Si y es no protegida, hacemos sendas sustituciones constantes en x y z para trivializar P_1 y P_2. En este caso, las tres variables se vuelven no influyentes, luego $\Delta\mu \geq 3\alpha_I$.

Supongamos ahora que el par comparte ambas variables de entrada y denotemos por x e y respectivamente a estas:

1. Si alguna de las dos variables es protegida, podemos proceder como en el primer caso de la discusión anterior, luego $\Delta\mu \geq 3 - \alpha_\Phi + \alpha_I + \alpha_Q$.
2. Si las dos son no protegidas, existe una sustitución constante o una sustitución afín de la forma $y \leftarrow x \oplus b$ de manera que se trivializan ambas puertas (basta con enumerar todas las posibilidades y comprobarlo). De esta manera, x e y se vuelven no influyentes, luego $\Delta\mu \geq 2\alpha_i \geq \frac{3}{2}\alpha_I$.

□

Teorema 2.55. *Sea $f : \mathbb{F}_2^n \to \mathbb{F}$ un rdq disperser de dimensión d y C un semicircuito que lo computa. Para todo empaquetado $\mathcal{P}$ del circuito, se tiene que*

$$\mu(C, \mathcal{P}, \emptyset) \geq \delta(n - 2d - 2) \text{ donde } \delta = \alpha_I + \text{mín}\left(\frac{\alpha_I}{3}, 2\beta_1 + 2\alpha_Q, 4\beta_1, 3 + \alpha_\Phi, 5 - \alpha_Q, \frac{5 - 2\alpha_\Phi + \alpha_Q}{2}\right)$$

Demostración. La prueba consiste en una distinción exhaustiva de casos con la misma idea de fondo que la demostración del Teorema 2.39, pero con el nuevo análisis más fino de dónde y de qué manera se pueden introducir verdaderas *troubled gates*. Por ser similar y requerir un análisis tedioso, se anima al lector interesado a consultarla directamente en [15]. □

Teorema 2.56. *Sea $f : \mathbb{F}_2^n \to \mathbb{F}$ un affine disperser de dimensión d y C un semicircuito que lo computa, entonces*

$$|C| \geq 3{,}1n - o(n)$$

Demostración. El teorema se prueba calculando la configuración de constantes que nos proporciona el mayor valor del que acompaña a n en el Corolario 2.42 sujeta a las restricciones impuestas sobre los valores de estas. Esto conforma el siguiente problema de programación lineal

$$\text{máx}\, z = \delta - \frac{\alpha_\Phi}{2} - \alpha_I$$

Sujeto a las restricciones:

$$\begin{aligned} \alpha_\Phi &\leq \frac{1}{2} \\ \alpha_Q, \alpha_\Phi, \alpha_I &\geq 0 \\ \delta &\leq \alpha_I + \text{mín}\left(\frac{\alpha_I}{3}, 2 - 2\alpha_\Phi - 2\alpha_Q, 4 - 4\alpha_\Phi, 3 + \alpha_\Phi, 5 - \alpha_Q, \frac{5 - 2\alpha_\Phi + \alpha_Q}{2}\right) \end{aligned}$$

cuyos valores óptimos vienen dados por

$$\begin{aligned} \alpha_\Phi &= \frac{2}{10} \\ \alpha_Q &= \frac{18}{10} \\ \alpha_I &= \frac{96}{10} \\ \delta &= \frac{128}{10} \end{aligned}$$

□

Capítulo 3

Estudio experimental

El objetivo del trabajo experimental es trabajar con una metodología completamente distinta a la del capítulo anterior. En lugar de tratar de probar directamente el resultado sobre el circuito de las funciones booleanas, la meta es encontrar una medida de complejidad que esté intrínsecamente relacionada con el tamaño del circuito booleano sin estar explícitamente definida sobre este. Si fuésemos capaces de probar que las imágenes del problema de decisión definido por una familia de funciones booleanas $\{f_n\}_{n\in\mathbb{N}}$ (con una función para cada tamaño de entrada n) en $P/poly$ están acotadas por una constante, cualquier problema con infinitas funciones por encima de la cota no podría estar en $P/poly$, así que habríamos probado que tener un circuito superpolinómico es equivalente a puntuar por encima de cierto valor en nuestra métrica. Posteriormente, si fuésemos capaces de encontrar esa familia para nuestra métrica, habríamos encontrado la primera función booleana de circuito superpolinómico y, por último, si demostrásemos que dicho problema de decisión está en NP, entonces habríamos deducido que $P \neq NP$.

Definición 3.1 (Medida de complejidad)**.** Sea $\mathcal{F}_n$ el conjunto de las funciones booleanas de n bits en 1. Llamamos *medida de complejidad* a cualquier función

$$\mu : \mathcal{F}_n \to \mathbb{R}.$$

La particularidad de nuestro trabajo es que trataremos de definir estas medidas de complejidad atentiendo a la regularidad evidente o subyacente de las tablas de verdad de las funciones booleanas. Entendemos por tabla de verdad a la cadena binaria que surge de concatenar los valores de la función sobre todas las posibles configuraciones de las variables de entrada, donde el primer bit es el menos significativo y el último el que más.

Definición 3.2 (Tabla de verdad)**.** Sea $f \in \mathcal{F}_n$ una función booleana. Definimos su *tabla de verdad* como el vector $t(f) \in \mathbb{F}^{2^n}$ dado por

$$t(f) = (f(b_1(0), \ldots, b_n(0)), \ldots, f(b_1(2^n - 1), \ldots, b_n(2^n - 1)))$$

donde para cada $i \in \{0, \ldots, 2^n - 1\}$, su representación binaria es $i = \sum_{j=1}^{n} b_j(i) \cdot 2^{j-1}$ con $b_j(i) \in \{0, 1\}$.

Si bien el fundamento sobre el que se sostiene el estudio es muy sencillo de explicar, la formalización del término "regularidad" es un asunto delicado. Podemos considerar regular una tabla de verdad cuando observamos cierta periodicidad o repetición dentro de los bloques que la componen. Por ejemplo, la tabla de verdad $t(f) = 00110011001100110011001100110011$ se consideraría muy regular y, de hecho,

la función $f = x_2$ de la que proviene tiene una complejidad de circuito muy baja. Sin embargo, existen otras funciones muy regulares que a priori, por su tabla de verdad, no lo parecen. Por ejemplo, la función paridad, cuya tabla es $t(f_{paridad}) = 01101001100101101001011001101001$. En ese caso, la regularidad de la tabla de verdad surge de observar que la mitad derecha es la negación de la mitad izquierda y que además esto es cierto recursivamente en cada una de las mitades, como muestra la Figura 3.1.

El ingrediente común en ambos casos es que hemos sido capaces de describir las tablas de verdad de forma reducida y la única diferencia ha sido la forma en que lo hemos hecho. La primera podría reescribirse como $(0^2 1^2)^8$ y la segunda podría hacerlo como $f(n) = concat(f(n-1), \neg f(n-1))$ y $f(1) = 0$. De alguna manera, los ejemplos anteriores sugieren que las funciones booleanas de complejidad de circuito sencilla podrán reescribir sus tablas de verdad de forma muy compacta y las complejas no podrán hacerlo tanto. Sin embargo, surgen varias preguntas: ¿es esto una intuición o un resultado formal?, ¿qué lenguajes podemos utilizar para escribir la forma compacta? Esta noción de "escribir de forma reducida una cadena de caracteres" se formaliza a través de la teoría algorítmica de la información dada por la complejidad de Kolmogorov.

Definición 3.3 (Máquina universal)**.** Sea $U : \{0,1\}^* \to \{0,1\}^*$ una función computable que mapea cadenas binarias finitas a cadenas binarias. Decimos que es una *máquina universal* si cualquier función computable $f : \{0,1\}^* \to \{0,1\}^*$ se puede codificar como una cadena s_f tal que

$$\forall x \in \{0,1\}^*,\ U(s_f\ x) = f(x).$$

Definición 3.4 (Complejidad de Kolmogorov)**.** Sea U una máquina universal. Definimos la *complejidad de Kolmogórov de prefijo libre* de una cadena finita $x \in \{0,1\}$ como

$$K(x) := \text{mín}\{\ |c| : U(c) = x\ \}$$

es decir, la longitud del programa más corto que hace que U produzca x.

Nota 3.5. Elecciones distintas de máquinas universales de prefijo libre cambian la complejidad de Kolmogorov $K(x)$ únicamente en una constante aditiva. En efecto, una máquina universal U_1 se puede codificar como una cadena M_{U_1} de manera que puede "ejecutarse" en la otra máquina universal U_2. De este modo, si P_{U_1} es el programa más corto que genera x en U_1, la cadena $M_{U_1}\ P_{U_1}$ genera x en U_2. Como M_{U_1} no depende de x, su longitud es constante y, como el desarrollo a la inversa es análogo, entonces $|K_{U_1}(x) - K_{U_2}(x)| \leq c$.

Podemos entender esta complejidad como el límite matemático y absoluto de la máxima compresión que podemos alcanzar para cualquier secuencia de caracteres de un cierto alfabeto. Además, la Nota 3.5 demuestra que si se prueba una cota superior polinómica para la complejidad de Kolmogorov en un lenguaje, automáticamente se prueba para los demás. Dentro de nuestro estudio, esto implica que un circuito booleano puede considerarse como un programa capaz de computar una función booleana y, por tanto, generar su tabla de verdad, es decir, la codificación del circuito que las genera es también una descripción de la función booleana.

La consecuencia directa es que las funciones booleanas computables por circuitos de tamaño polinómico podrán describirse de forma más compacta por la descripción de su circuito en lugar de por su tabla de verdad. En efecto, si su circuito mínimo tiene s puertas, podemos numerarlas e identificar cada una de ellas con su representación binaria utilizando $\log_2(s)$ bits. Indicar el tipo de puerta como mucho son cuatro bits y quedarían únicamente las conexiones, para las que basta indicar los dos identificadores binarios de las puertas que la alimentan. Por tanto, la especificación completa del circuito consiste en un listado ordenado por número identificador de las dos conexiones de cada puerta, es decir, de

$s \cdot 2\log_2(s)$ bits, que está en $O(n^k)$ si $s \in O(n^k)$. Su complejidad de Kolmogorov estará acotada superiormente por el tamaño de su circuito mínimo, lo que implica que la información contenida en una tabla de verdad de 2^n bits puede ser descrita usando tan solo $O(n^k)$ bits.

Sin embargo, el recíproco no es cierto en el caso general. Una tabla de verdad podría poseer una complejidad de Kolmogorov muy baja (pudiendo ser generada por un programa muy corto en una máquina de Turing) pero requerir un tiempo o espacio exponencial para decodificarse. Al traducirlo al modelo no uniforme, podría requerir un circuito booleano de tamaño superpolinómico para ser evaluada debido a la falta de uniformidad o a la cantidad de recursos temporales que necesita el desencapsulado de la información.

Esta limitación de la complejidad de Kolmogorov pura[1] no invalida nuestra hoja de ruta, sino que precisamente la justifica. En virtud del contrarrecíproco, si logramos demostrar que la existencia de un circuito de tamaño polinómico implica una puntuación baja en nuestra métrica, la existencia de cualquier función que puntúe por encima de dicha cota garantizará de forma unívoca que su complejidad de circuito es superpolinómica. La limitación únicamente causa la aparición del fenómeno opuesto: podrían existir funciones extremadamente complejas para los circuitos que la compresión dada por nuestras medidas de complejidad clasifiquen erróneamente como sencillas.

Por ello, si bien la validez del contrarrecíproco asegura que cualquier familia de funciones en NP que demuestre ser incomprimible bajo las métricas aquí propuestas bastará para certificar que $P \neq NP$, trataremos de proponer medidas que aproximen una equivalencia bidireccional en este sentido. El mejor escenario sería encontrar una métrica perfecta donde una alta complejidad de circuito se traduzca siempre y sin excepciones en una alta puntuación en nuestra métrica, pues facilitaría mucho nuestro estudio.

3.1. Repetición de patrones

La primera propuesta de medidas de complejidad basa su cómputo en detectar cómo están relacionadas las distintas partes de la tabla de verdad entre sí sin tener en cuenta cómo se computa esta. Esto quiere decir que trataremos de capturar la regularidad únicamente a través de la relación directa entre bits de la tabla, de manera que formen subpatrones que nos sirvan para caracterizar sus regularidades. Son medidas de complejidad orientadas a cuantificar numéricamente la regularidad a través del "desorden" intrínseco de la tabla de verdad, más que a descubrir el por qué de la regularidad subyacente.

3.1.1. Entropía de Shannon

Al inicio del capítulo mostramos que toda función computable por circuitos polinómicos podía describir su tabla de verdad como una cadena de bits polinómica en lugar de exponencial y comentamos que, para nuestro estudio, nos interesa el contrarrecíproco. La compresión mínima absoluta, que revelaría qué tablas de verdad son comprimibles y cuáles no, vendría dada por la complejidad de Kolmogorov. Como esta es no computable, nosotros estudiaremos estimaciones empíricas a través de la Teoría de la Información de Shannon.

Dado un mensaje de longitud 2^n formado por distintos caracteres, la Teoría de la Información pretende definir una codificación binaria óptima que minimice la longitud en bits del mensaje completo. En lugar de codificar cada caracter con un número fijo de bits, se escoge una longitud menor para los caracteres que aparecen muy frecuentemente y una mayor para los más inusuales. El Ejemplo 3.6 ilustra esta

[1] Hay otras versiones que tienen en cuenta esta limitación en tiempo o espacio para evitar problemas de este tipo.

estrategia. En nuestro caso, lógicamente, no podemos considerar como caracteres los dígitos binarios, pues la codificación en ese caso es única; sin embargo, una alternativa es utilizar la codificación por bloques, es decir, considerar el alfabeto de todas las cadenas binarias de longitud 2^i y decidir, en base a la frecuencias de aparición de estos bloques en la tabla de verdad, cuál es la codificación óptima que la comprime al máximo. El Ejemplo 3.7 muestra la codificación óptima por árboles de Huffman de la tabla de verdad propuesta cuando se usa el alfabeto compuesto por los bloques de tamaño 2^1.

Ejemplo 3.6. Si consideramos la cadena *aaaabbcacaababacddcdbaaababbcdab* sobre el alfabeto $\Sigma = \{a, b, c, d\}$, la codificación dada por $a \to 0$, $b \to 10$, $c \to 111$, $d \to 110$ comprime la cadena original a la cadena

$$00001010111011100100111010110110111110100001001011111100010$$

en lugar de la cadena dada por la codificación estándar de 2 bits para cada letra ($a \to 00$, $b \to 01$, $c \to 10$, $d \to 11$), que daría la cadena

$$00000000010110001000000100010010111110110100000000010001011000001$$

Ejemplo 3.7. Sea la tabla de verdad 00000011000010101011100000000000, si escogemos la codificación dada por $00 \to 0$, $11 \to 10$, $10 \to 110$, $01 \to 111$, entonces la tabla de verdad queda reescrita como 000100011011011011011000000.

Más formalmente, estamos considerando una fuente aleatoria X que emite bloques aleatorios de tamaño 2^k de las $2^{\frac{n}{k}}$ posiciones consecutivas de la tabla de verdad de f. Según la Teoría de la Información de Shannon, el límite teórico de compresión sin pérdida está acotado inferiormente por su entropía, dada por

$$H(X) = -\sum_{j=1}^{n} p(x_j) \cdot \log_2(p(x_j)),$$

donde los x_j son todos los posibles bloques de tamaño 2^k. Si bien se utilizan algoritmos de codificación de prefijo como los árboles de Huffman para aproximar todo lo posible este límite en la práctica (en el caso de los árboles de Huffman, entre $H(X)$ y $H(X)+1$ bits por símbolo), podemos definir la medida de complejidad directamente en términos del límite teórico que determina la entropía.

Definición 3.8 (Medida de complejidad de la entropía de Shannon)**.** Sea $f \in \mathcal{F}_n$ y sea X una fuente aleatoria que emite bloques de tamaño 2^k de la tabla de verdad de f. Definimos la *medida de complejidad de la entropía de Shannon* como

$$\mu_S(f) = \frac{|t(f)| - H(X) \cdot 2^{n-i}}{|t(f)|}$$

Una pregunta muy razonable es pensar qué tamaño de bloque debemos escoger, pues es evidente que los bloques de tamaño 2^0 no son útiles y los de tamaño 2^n tampoco. No obstante, durante el estudio de qué tamaño de bloque era más apropiado, se observó que la familia de funciones booleanas $\{enum_k\}_{k=1}^{\infty}$, donde $enum_k \in \mathcal{F}_{2^k+k}$ es la enumeración de los primeros 2^{2^k} números utilizando 2^k bits, es "resistente" a la compresión sea cual sea el tamaño de bloque que se fije.

Proposición 3.9. *Sea $\{enum_k\}_{k=1}^{\infty}$ la familia de funciones booleanas cuya tabla de verdad es la enumeración de los primeros 2^{2^k} naturales en binario con 2^k bits. Para todo $i \in \mathbb{N}$, las funciones $enum_k$ con $k \geq i$ son incomprimibles por la entropía de Shannon para longitud de bloque 2^i.*

Demostración. Para evaluar la incompresibilidad por entropía de Shannon con longitud de bloque 2^i, debemos analizar la distribución de los bloques de tamaño 2^i dentro de $t(enum_k)$. Como $k \geq i$, 2^i

divide exactamente a 2^k y, por tanto, cada una de las cadenas de longitud 2^k que forman $t(enum_k)$ se puede particionar de manera exacta en $m = 2^{k-i}$ bloques contiguos de longitud 2^i.

Para un bloque arbitrario $x_j \in \{0,1\}^{2^i}$, queremos contar la frecuencia de aparición en la tabla de verdad completa. Como $t(enum_k)$ contiene todas las cadenas de $\{0,1\}^{2^k}$ una única vez y cada una está partida en m bloques más pequeños, entonces hay que contar cuántas de estas combinaciones de m bloques pequeños distintas contienen a x_j. Obsérvese que no puede haber configuraciones repetidas (porque entonces habría dos cadenas de longitud 2^k repetidas) ni faltar alguna (pues entonces faltaría alguna de longitud 2^k), así que están todas las posibles combinaciones de m bloques de longitud 2^i y una única vez cada una. Hay m posiciones en las que puede aparecer x_j y se pueden elegir las restantes de

$$\left(2^{2^i}\right)^{m-1} = 2^{2^i(m-1)} = 2^{2^k-2^i}$$

formas distintas, así que la frecuencia total de aparición del bloque x_j en $t(enum_k)$ es:

$$m \cdot 2^{2^k-2^i} = 2^{k-i} \cdot 2^{2^k-2^i}$$

La probabilidad de encontrar el bloque x_j al tomar un bloque de longitud 2^i al azar en $t(enum_k)$ es el cociente entre su frecuencia de aparición y el total de bloques:

$$p(x_j) = \frac{2^{k-i} \cdot 2^{2^k-2^i}}{2^{k-i} \cdot 2^{2^k}} = \frac{2^{2^k-2^i}}{2^{2^k}} = 2^{-2^i} = \frac{1}{2^{2^i}}$$

Esta probabilidad es uniforme y no depende de x_j. Por lo tanto, todos los posibles bloques de longitud 2^i son equiprobables y, en consecuencia, su entropía, máxima:

$$H = -\sum_{x_j \in \{0,1\}^{2^i}} p(x_j) \log_2 p(x_j) = -\sum_{x_j \in \{0,1\}^{2^i}} \frac{1}{2^{2^i}} \log_2 \left(\frac{1}{2^{2^i}}\right) = 2^{2^i} \cdot \left(\frac{1}{2^{2^i}} \cdot 2^i\right) = 2^i$$

Al ser la entropía máxima, la redundancia de la fuente es cero, lo que implica que la secuencia $t(enum_k)$ es estrictamente incomprimible (en el límite asintótico de Shannon) para bloques de longitud 2^i. □

La Proposición 3.9 pone de manifiesto que, sea cual sea el tamaño de bloque seleccionado, todas las funciones booleanas de la familia $\{enum_k\}_{k\in\mathbb{N}}$ a partir de un cierto índice son incomprimibles. El problema de decisión formado por las funciones que coinciden con las de $\{enum_k\}_{k\in\mathbb{N}}$ en los $n = 2^k + k$ y que son la función constante 0 en el resto está en $P/poly$ y contiene infinitas funciones que alcanzan el valor máximo de la métrica; por tanto, la métrica no permite realizar el argumento propuesto al inicio.

3.1.2. Número de clasificaciones

Si bien es cierto que la métrica anterior es capaz de capturar la intuición de que la regularidad no es un fenómeno local (porque tiene en cuenta que la repetición de los bloques puede estar diseminada por la tabla de verdad), asume ciertas condiciones que desde luego no parecen necesarias. En primer lugar, presupone que la regularidad debe tener manifestaciones compactas, es decir, en bloques que agrupan bits consecutivos. En segundo, asume que la única relación que pueden tener bloques entre sí es a través de la identidad, pero nada impide que otro tipo de transformaciones (por ejemplo, la negación) permitan relacionar unos bloques con otros. Por último, no justifica la razón por la que deben considerarse los bloques de regularidad en términos de las divisiones "canónicas" de la tabla de verdad en potencias de dos.

La Proposición 3.9 muestra una prueba incontestable de por qué la métrica no es útil para nuestro propósito; sin embargo, conviene estudiar ejemplos donde aparezan las suposiciones que hemos enumerado para conjeturar cómo debería tenerlas en cuenta la métrica propuesta. Por ejemplo, consideremos la función paridad, cuya tabla de verdad viene dada por

$$t(f_{paridad}) = 01101001100101101001011001101001.$$

Cualquier división en bloques de potencias de dos muestra que la tabla de verdad es combinación de dos bloques distintos. No obstante, un análisis más detallado muestra que la mitad derecha es la negación de la mitad izquierda y, no solo eso, sino que esto es cierto recursivamente dentro de cada una de las mitades. La Figura 3.1 muestra una representación gráfica de esta relación entre bloques. Por lo tanto, la tabla de verdad queda determinada unívocamente por su primer bit: basta con tomar este y concatenar su negado a la derecha, tomar este nuevo bloque y concatenar su negado a la derecha, y así sucesivamente, tantas veces como entradas tenga la función paridad.

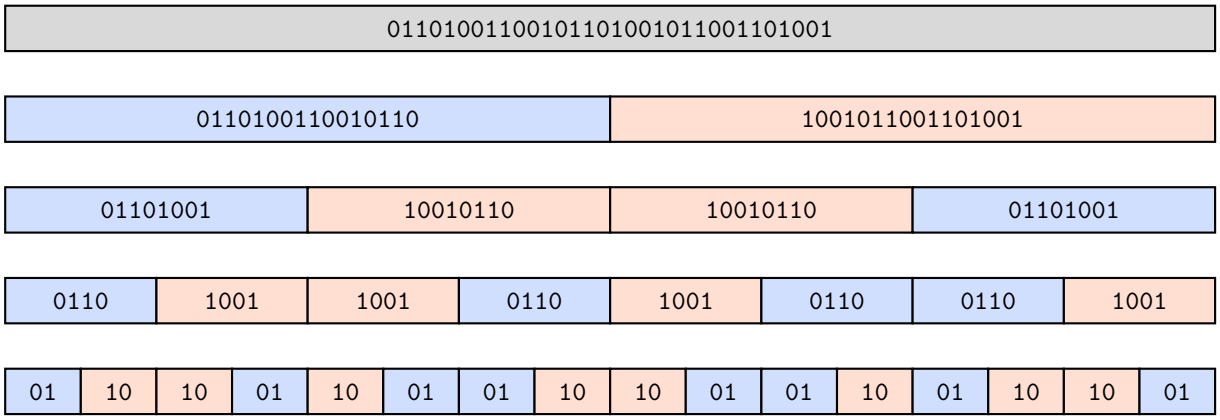


Figura 3.1: Descomposición recursiva de la tabla de verdad de la función paridad en términos de sus mitades. Los colores rojo y azul atenuados hacen evidente la relación de negación entre una submitad y otra dentro de cada mitad.

Esta relación de cualquier bit de la tabla con el primero da pie a pensar en relaciones entre bits no consecutivos. Para explorar estas relaciones no localizadas, podemos generalizar el conteo de frecuencias de bloques de tamaño 2^k en las posiciones "canónicas" al conteo de frecuencias de subsecuencias cualesquiera de tamaño k. Esta deslocalización no permite hacer codificaciones en el sentido anterior porque no sabríamos como dividir la cadena en "caracteres" que poder traducir, pero sí permite seguir considerando la frecuencia de aparición de cada subsecuencia. La conjetura es que tablas de verdad poco regulares serán muy expresivas, es decir, tendrán muchísimas subsecuencias distintas.

Definición 3.10 (Número de clasificaciones)**.** Sea $\mathbf{x} = (x_1, \ldots, x_m)$ una muestra de m puntos de $\mathbb{F}_2^n$. Definimos el *número de clasificaciones de* $\mathbf{x}$ *por* H como

$$\Pi_H(\mathbf{x}) = |\{(f(x_1), \ldots, f(x_m)) : f \in H\}|$$

Definición 3.11 (Medida de complejidad del número de clasificaciones)**.** Sea $\mathcal{X}_m$ el conjunto de todas las muestras ordenadas $\mathbf{x} = (x_1, \ldots, x_m)$ de m puntos de $\mathbb{F}_2^n$ y sea $f \in \mathcal{F}_n$. Definimos la *medida de complejidad del número de clasificaciones* como

$$\mu_{class}(f) = \sum_{m=1}^{n} |\{(f(x_1), \ldots, f(x_m)) : \mathbf{x} \in \mathcal{X}_m\}|$$

Nota 3.12. El orden de la muestra es imprescindible en la Definición 3.11 para que esta tenga sentido. En efecto, si se omite el orden en la definición la cantidad de vectores distintos se reduce únicamente a la distribución de ceros y unos en la tabla de verdad.

La medida de complejidad de la Definición 3.11 está inspirada en la función de crecimiento de la *Vapnik-Chervonenkis dimension* recogida en [5], cuyo objetivo es capturar la expresividad de un conjunto de funciones $H \subset \mathcal{F}_n$ y medir la complejidad de identificar una función de dicha familia si solo se dispone de un subconjunto de su tabla de verdad. Por ejemplo, en base a esta métrica las funciones constantes 1 y 0 son las más sencillas posibles porque solo tienen una clasificación para cada tamaño de subsecuencia. De nuevo, como no hay un criterio evidente que permita decidir cuál es el tamaño de subsecuencia adecuado, la medida de complejidad considera todos los posibles a través de la suma.

No obstante, la implementación algorítmica de esta métrica reveló que su cómputo era cuadrático en el tamaño de la tabla de verdad, como expone el Algoritmo 1 presentado en el Apéndice A. Este tipo de métricas cuyo cómputo es polinómico en la tabla de verdad —entre otras propiedades adicionales— son muy sospechosas de no ser aptas para separar las clases de P y NP, pues incurren en lo que conocemos como *demostraciones naturales*. En el Teorema 4.6 de la Sección 4.1 se prueba por qué es altamente improbable que cualquier medida de complejidad de este tipo tenga garantías de éxito, y posteriormente en la misma sección recuperamos el ejemplo de esta métrica para comprobar que efectivamente es una demostración natural.

3.2. Solapamiento de patrones

El conjunto de propuestas basadas únicamente en la repetición de subpatrones en la tabla de verdad carece de una propiedad clave para el estudio de los circuitos mínimos: la recursividad y combinación de patrones. Esta limitación motiva una propuesta diametralmente diferente: en lugar de buscar qué bloques o subsecuencias se repiten más a menudo, trataremos de detectar que una tabla de verdad es sencilla porque se trata de combinaciones lineales y cuadráticas de un conjunto "muy reducido" de subpatrones sencillos.

La principal diferencia con respecto a las medidas de complejidad anteriores es que estas no solo describen la regularidad intrínseca de la tabla de verdad, sino que están interesadas en cómo esta se computa. En cierta manera, podríamos decir que proporcionan el "programa" que genera su cómputo.

3.2.1. Forma normal de Fourier

Consideremos —sin tener en cuenta las constantes— las funciones más simples que conocemos: las funciones $f_i = x_i$. Por un lado, estas constituyen las funciones booleanas más sencillas en términos de complejidad, pues no requieren de ninguna puerta para su cómputo. Por otro, cualquier función booleana es una combinación de estas a través de los operadores $\{\wedge, \vee, \neg\}$, pues es un conjunto universal de operaciones booleanas. Además, de nuevo podemos prescindir de la puerta $\neg$ incorporando como posibles variables de entrada a las funciones $\neg x_i$, porque el número de puertas mínimo solo se degenera polinómicamente. Por tanto, parece natural definir la complejidad de una función booleana en términos de cuántas de estas funciones posee y cómo están combinadas para construir la función total.

Definición 3.13 (Armónicos fundamentales)**.** Sea $\mathcal{F}_n$ el conjunto de funciones booleanas de n bits en 1 bit. Definimos el conjunto de *armónicos fundamentales* como

$$\mathcal{X} = \{f \in \mathcal{F}_n : \exists i \in I \text{ tal que } f(x_1, \ldots, x_n) = x_i \text{ o } f(x_1, \ldots, x_n) = \neg x_i\}$$

y llamamos a cada x_i *armónico i-ésimo* y cada $\neg x_i$ como *armónico i-ésimo negado.*

Naturalmente, nuestra medida de complejidad debería juzgar cualquiera de estos armónicos como equivalentes, es decir, $\forall x, y \in \mathcal{X},\ \mu(x) = \mu(y)$. El interrogante es cómo determinar la complejidad de la combinación de estos armónicos entre sí para formar funciones booleanas más complejas. La combinación de estos armónicos fundamentales a través de las operaciones $\wedge$ y $\vee$ genera todas las posibles funciones booleanas de $\mathcal{F}_n$. Además, dota a cada $f \in \mathcal{F}_n$ de una estructura sintáctica donde: o bien se fuerza la topología de árbol, lo que implica entender la estructura como una expresión booleana; o bien se permite la topología de DAG, lo que implica entender la estructura como un circuito. La Figura 3.2 muestra un ejemplo de esta afirmación. Si nuestra métrica deduce la complejidad razonando sobre cómo están combinados los armónicos fundamentales y estas combinaciones dotan de una cierta estructura sintáctica a las funciones booleanas, parece lógico que nuestra métrica razone sobre la propia estructura para decidir la complejidad de la función booleana. Como queremos definir una medida de complejidad que correlacione con el tamaño del circuito mínimo, pero no razone directamente sobre este, parece natural que optemos por generar la estructura como árbol sintáctico.

No obstante, la estructura de árbol implica trabajar con expresiones booleanas y, en consecuencia, es necesario hacer una serie de observaciones acerca de cómo debe comportarse la medida de complejidad. La primera es la incapacidad de las expresiones booleanas de reutilizar cómputos en varios lugares de la fórmula: mientras la estructura de grafo permite conectar varios nodos al mismo subárbol sin necesidad de repetirlo, la estructura de árbol requiere duplicar subárboles idénticos allí donde sean necesarios (como muestra la Figura 3.2 para $x_1 \wedge w_2$). Si nuestra métrica no tiene en cuenta las múltiples apariciones de una misma combinación de armónicos a lo largo del árbol sintáctico, juzgará como más compleja una función que en realidad no lo es. La segunda está relacionada con la primera y es la existencia de más de una expresión para la misma función booleana, como pone de manifiesto el Ejemplo 3.14. Esto implica que o bien establecemos una expresión booleana "canónica" que permita comparar subárboles entre sí, o bien establecemos una relación de equivalencia que identifique subárboles equivalentes.

Ejemplo 3.14. Consideremos las siguientes dos expresiones booleanas distintas:

$$e_1 = (x_1 \wedge x_2) \vee (\neg x_2 \wedge x_3) \vee (\neg x_1 \wedge x_4)$$

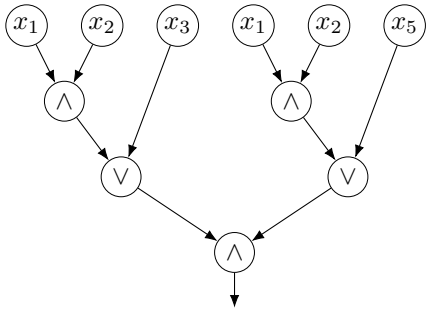
$$e_2 = (x_1 \wedge x_3) \vee (\neg x_1 \wedge x_4) \vee (x_2 \wedge \neg x_4)$$

Ambas expresiones son irreducibles (no existe una forma más compacta para ninguna de las dos) y, sin embargo, definen exactamente la misma función booleana, lo que demuestra que no existe unicidad si se utilizan las expresiones booleanas.

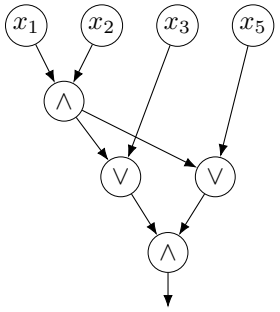
Las dificultades que plantea una métrica que deba ser definida en estos términos sugiere abordar el problema desde un ángulo distinto. Cada función booleana está biunívocamente identificada con su tabla de verdad: en particular, el armónico fundamental i-ésimo posee una tabla de verdad con un patrón de periodo 2^i. Como cada función booleana es combinación de los armónicos fundamentales, en realidad su tabla de verdad es un cierto solapamiento de estos patrones fundamentales. Por ejemplo, consideremos la función booleana

$$f_1(x_1, x_2, x_3, x_4, x_5) = x_5 \wedge x_3 \wedge x_1,$$

cuya tabla de verdad es $t(f_1) = 00000000000000000000010100000101$. Esta es el resultado de solapar adecuadamente las tablas de verdad de los armónicos fundamentales que forman f_1. La periodicidad de los armónicos que la componen, si bien se pierde al combinarlos, se mantiene en cierta manera subyacente en la tabla de verdad. Por ejemplo, la mitad izquierda queda completamente determinada



(a) Estructura sintáctica de árbol para la combinación de armónicos fundamentales (expresión booleana): subárboles idénticos en distintos lugares duplican su aparición.



(b) Estructura sintáctica de grafo para la combinación de armónicos fundamentales (circuito booleano): subárboles idénticos reaprovechan su cómputo.

Figura 3.2: Ejemplo de la diferencia de la topología exigida a la estructura sintáctica de la combinación de armónicos booleanos: (a) la rigidez del árbol sintáctico fuerza la repetición de subestructuras, mientras que (b) el grafo dirigido acíclico (DAG) permite la reutilización de recursos propia de un circuito.

por x_5 sin importar el resto de armónicos —pues en esa parte x_5 es 0— y la mitad derecha depende únicamente del resto de armónicos —pues en esa parte x_5 es 1—. Observando recursivamente la mitad derecha, podemos hacer un razonamiento similar con x_3 y así sucesivamente. Por supuesto, es posible hacer un razonamiento análogo para la función booleana

$$f_2(x_1, x_2, x_3, x_4, x_5) = x_5 \vee x_3 \vee x_1,$$

cuya tabla de verdad es $t(f_2) = 01011111010111111111111111111111$. En ese caso, la mitad derecha queda completamente determinada por x_5 sin importar el resto de armónicos y la mitad izquierda depende únicamente del resto, donde se puede hacer un razonamiento recursivo.

Este razonamiento revela algunas propiedades esenciales de esta descomposición en armónicos superpuestos cuando se consideran por separado la operación $\vee$ y $\wedge$. Por un lado, si los armónicos se ordenan lexicográficamente, se puede considerar cada función como un armónico fundamental x_i sobre el que se ha superpuesto otra función que no depende de x_i y que además admite el mismo razonamiento de forma recursiva. Por otro, precisamente esta consideración permite observar que si comparásemos la complejidad de f_1 con otra función $\tilde{f}_1 = x_5 \wedge g(x_1, \ldots, x_4)$, la medida de complejidad debería diferenciar a ambas por la diferencia de complejidad entre las subfunciones $x_3 \wedge x_1$ y $g(x_1, \ldots, x_4)$. Si las descomposiciones son mínimas en el sentido de que no hay redundancias del tipo $x_i \wedge \neg x_i$ o $x_i \vee \neg x_i$, entonces los dos problemas que identificamos para las expresiones booleanas —representación única y reutilización de cómputos— en principio quedan resueltos (pues las subfunciones ya no contienen a x_i).

El reto está en combinar la descomposición $\wedge$ de armónicos y la descomposición $\vee$ de armónicos en una única versión que permita utilizar ambas operaciones simultáneamente para poder representar todas las funciones de $\mathcal{F}_n$. La idea es que conforme se va construyendo el árbol sintáctico la periodicidad de los armónicos fundamentales se va difuminando en la tabla de verdad, pero siguen gobernando la complejidad de la función que se construye. De esta manera, tablas de verdad aparentemente enrevesadas quedan expuestas a través de cómo combinan la simplicidad de los armónicos fundamentales subyacentes. La estructura debería preservar la unicidad y la no redundancia que han mostrado las versiones por separado.

Definición 3.15 (Forma normal de Fourier)**.** Sea $\mathcal{F}_n$ el conjunto de funciones booleanas de n bits en 1 bit, sea $f \in \mathcal{F}_n$ y sea $\prec$ la relación de orden total sobre $\mathcal{X}$ definida como:

$$\neg x_1 \prec x_1 \prec \neg x_2 \prec x_2 \prec \cdots \prec \neg x_n \prec x_n.$$

Dada la forma normal disyuntiva mínima de f primera lexicográficamente, que denotaremos de ahora en adelante por $FND_{min}(f)$, definimos su *forma normal de Fourier (FNF) de* f a partir de esta de la siguiente manera:

- Si $FND_{min}(f)$ es una constante 0 o 1, su FNF es la propia constante.
- Si $FND_{min}(f)$ no es constante, entonces:
  1. Denotando por $x_i \in \mathcal{X}$ al literal de mayor orden según $\prec$ presente, la forma normal disyuntiva se descompone como:

     $$FND_{min}(f) = (x_i \wedge e_1) \vee (e_2)$$

     donde e_1 es la subexpresión compuesta por los implicantes primos que contenían a x_i (extrayendo x_i de ellos) y e_2 es la función compuesta por los implicantes primos que no contenían a x_i.
  2. Recalculamos recursivamente e_1 y e_2.

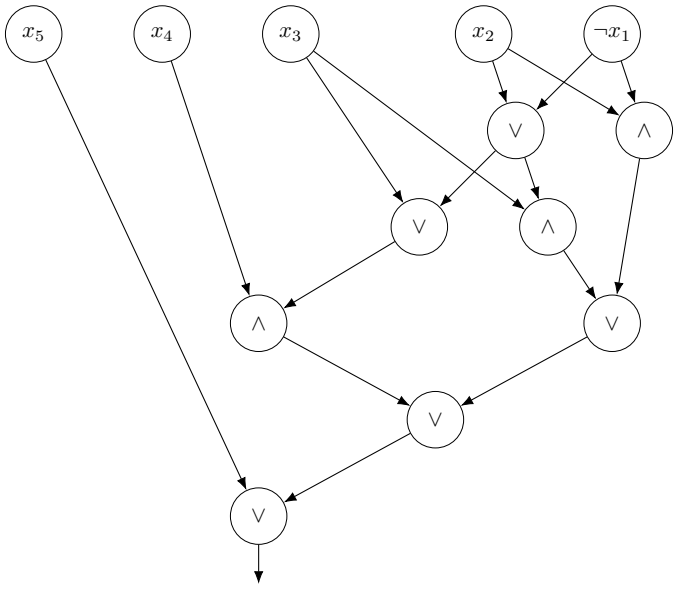
Ejemplo 3.16 (Cálculo de la FNF)**.** Sea f la función booleana cuya tabla de verdad viene dada por $t(f) = 00000000001011001110000100110010$. Una FND mínima para esta viene dada por

$$FND_{min}(f) = (\neg x_1 \wedge x_2) \vee (\neg x_1 \wedge x_3) \vee (\neg x_1 \wedge x_4) \vee (x_2 \wedge x_3) \vee (x_2 \wedge x_4) \vee (x_3 \wedge x_4) \vee (x_5).$$

Si seguimos el procedimiento descrito en la Definición 3.15, entonces obtenemos la expresión (omitiendo las constantes y los paréntesis innecesarios para simplificar la notación) dada por

$$FNF(f) = x_5 \vee (x_4 \wedge (x_3 \vee x_2 \vee \neg x_1) \vee (x_3 \wedge (x_2 \vee \neg x_1) \vee (x_2 \wedge \neg x_1)))$$

y cuya estructura sintáctica asociada puede representarse gráficamente como



Proposición 3.17. *Sea $f \in \mathcal{F}_n$ y sean $f_1, f_2 \in \mathcal{F}_n$ las funciones booleanas computadas por las expresiones booleanas e_1 y e_2 tales que $FNF(f) = (x_i \wedge e_1) \vee e_2$. Se verifican las siguientes propiedades:*

1. *La disyunción de implicantes primos factorizados de e_1 es la forma normal disyuntiva mínima primera lexicográficamente de f_1.*
2. *La disyunción de implicantes primos de e_2 es la forma normal disyuntiva mínima primera lexicográficamente de f_2.*
3. *Sean $FNF(f)$, $FNF(f_1)$ y $FNF(f_2)$ las formas normales de Fourier construidas de forma independiente. Estas verifican la igualdad:*

$$FNF(f) = x_i \wedge FNF(f_1) \vee FNF(f_2).$$

Demostración. Probamos cada uno de los puntos:

1. Por reducción al absurdo, supongamos que no. Como e_1 no depende de x_i, entonces f_1 no puede depender de x_i, lo que implica que cualquier FND mínima no contiene x_i como literal. Si tomamos $FND_{min}(f_1)$e incorporamos x_i a cada uno de los implicantes primos que la forman, junto con e_2 una $FND_{min}(f)$ más pequeña para f, luego no es posible.
2. El razonamiento es completamente análogo, pero sin incorporar x_i.
3. Trivial por los dos puntos anteriores.

□

La Proposición 3.17 es de vital importancia por varios motivos. El primero es que, aunque la Definición 3.15 es estructural sobre la $FND_{min}(f)$, en realidad es recursiva sobre las FNF de las dos funciones definidas por las dos subexpresiones e_1 y e_2. La principal consecuencia es que permite entender una FNF como combinación de FNFs, que se alinea bastante con la motivación inicial de la métrica. El segundo es que, una vez unificado el criterio lexicográfico para que la FND mínima sea única, permite afirmar que la descomposición es única, lo que habilita razonar sobre la expresión booleana sin ambigüedades.

Esta descripción recursiva es muy similar a la descomposición en serie de Fourier, donde una función analítica es descrita en términos de un solapamiento de armónicos más sencillos que forman ondas complejas al solaparse entre sí, de ahí el nombre que recibe. En general, la forma normal de Fourier está describiendo una función booleana f en términos de otras funciones booleanas más sencillas y, a su vez, estas en otras más sencillas, etc. hasta llegar a los armónicos fundamentales. Si dijimos que la medida de complejidad debía juzgar la complejidad en términos de cómo se combinan las subfunciones que van solapando sus patrones sobre la tabla de verdad, parece evidente cuál debe ser la definición de la métrica propuesta.

Definición 3.18 (Métrica fnfs)**.** Sea $f \in \mathcal{F}_n$ y sea $FNF(f)$ su forma normal de Fourier. Definimos la *medida de complejidad de la combinación de armónicos fundamentales* como el número de distintas FNFs que conforman el árbol de manera recursiva.

La medida de complejidad de la Definición 3.18 captura a la perfección los dos requisitos que pedimos para ella al inicio. Por una parte, como la descomposición es única, para cada función booleana hay una y solo una forma de calcular la medida. Por otra, tiene en cuenta la reutilización de cómputo al

contar el número de subfunciones distintas. En consecuencia, posee todos los ingredientes necesarios para poder ser utilizada.

Para las pruebas experimentales, decidimos restringirnos al conjunto de funciones booleanas de 5 bits en 1 bit. El número de funciones de 5 bits en 1 bit asciende a la astronómica cantidad de 2^{2^5} funciones, es decir, más de cuatro mil millones de funciones booleanas; por lo que parece razonable que sea el límite de lo que podemos computar exhaustivamente (para 6 bits, la cantidad de funciones es el cuadrado de esta). Para el estudio exahaustivo, se ha desarrollado un motor de cómputo alojado en [1], cuyo objetivo es proporcionar un entorno fiable, controlado y eficiente para calcular los valores de las métricas[2] sobre las funciones booleanas, para medir el comportamiento de los valores en relación al tamaño del circuito mínimo y para visualizar los resultados de las pruebas. Ante la magnitud de las funciones a estudiar y la posible complejidad del cómputo de las métricas en cada una,el diseño ha estado centrado en la paralelización y cálculo masivo de datos de forma eficiente.

Una vez restringidos al conjunto de funciones 5 bits en 1 bit, uno de los elementos clave para la realización del experimento ha sido la clasificación por tamaño de circuito mínimo dada por Knuth en [13] para todas ellas. Además, la librería de terceros [11] desarrollada por Goucher ha sido un elemento imprescindible para manejar con comodidad su implementación física. Ha permitido calcular eficientemente y de forma exhaustiva el tamaño del circuito mínimo y etiquetar cada función booleana con el tamaño de su circuito mínimo (de 0 a 12). El objetivo final del trabajo de cómputo sobre una métrica era generar los siguientes artefactos para cada uno de los conjuntos de funciones formados en base al tamaños de circuito mínimo (de ahora en adelante *clase de tamaño*) su posterior análisis:

- Un histograma de los valores proporcionados por la métrica para esa clase de tamaño;
- Un registro de las funciones que han obtenido la puntuación máxima dentro de la clase y cuál ha sido esta puntuación máxima;
- Un registro de las funciones que han obtenido la puntuación mínima dentro de la clase y cuál ha sido esta puntuación mínima.

Los resultados obtenidos para la medida de complejidad de la Definición 3.18 han sido bastante satisfactorios. En primer lugar, la Figura 3.3 y la Figura 3.4 contienen la información visual de los valores de la métrica por clase de tamaño. En ella se pueden observar que, si bien existe cierta variabilidad dentro de cada clase de tamaño, la distribución de valores de la métrica en cada una respeta —al menos en media— la complejidad del circuito mínimo de las funciones booleanas y además lo hace con una distribución aparentemente normal. Los parámetros están especificados en la Tabla 3.1.

Podemos decir que la métrica está capturando de alguna manera la complejidad del circuito mínimo de las funciones booleanas. Es más, la correlación de Pearson de la medida de complejidad con respecto al tamaño del circuito mínimo es de 0,4463, quedando relativamente cerca de los mejores valores obtenidos hasta ahora por otros trabajos previos en la misma línea (consultar en [10]). Sin embargo, ¿a qué se debe la variabilidad de los valores dentro de una misma clase de tamaño?

3.2.2. Forma normal de Fourier generalizada

La variabilidad dentro de una misma clase para la medida de la Definición 3.18 se debe a una limitación clave que presenta la forma normal de Fourier con respecto a la clasificación óptima de [13]. Mientras que esta última es capaz de calcular el circuito mínimo de $\neg f$ sin más que modificar el circuito

[2] No solo de esta sino de cualquiera que se implemente, de manera que se intercambien fácilmente.

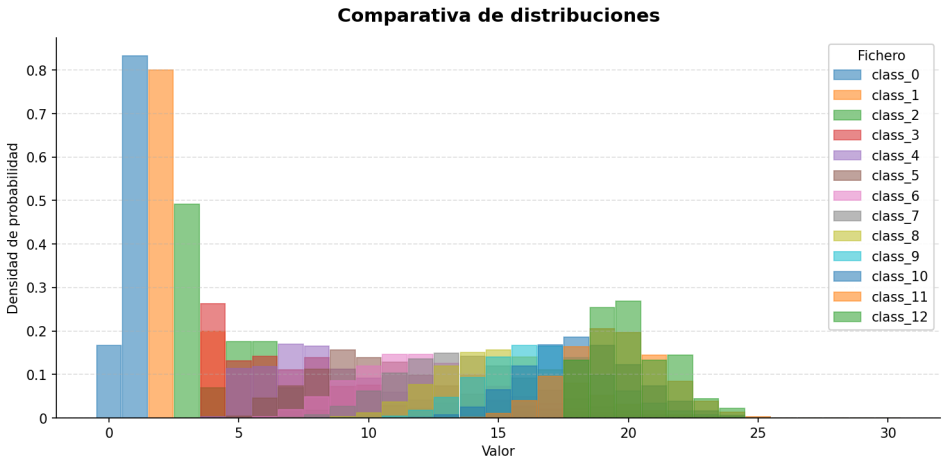


Figura 3.3: Comparación de los histogramas normalizados de la medida de complejidad de la Definición 3.18 por clase de tamaño.

Clase de tamaño	Media (μ)	Desviación estándar (σ)
0	0.83	0.37
1	2.40	0.80
2	4.32	1.48
3	6.66	2.42
4	8.80	3.24
5	10.71	2.99
6	12.33	2.87
7	13.82	2.72
8	15.32	2.56
9	16.79	2.35
10	18.17	2.12
11	19.46	1.89
12	20.13	1.50

Tabla 3.1: Parámetros de la distribución normal aparente en la medida de complejidad de la Definición 3.18 por clase.

mínimo de f añadiendo una puerta $\neg$ al final, la forma normal de Fourier requiere recomputar la descomposición para $\neg f$ sin ninguna relación con f.

Ejemplo 3.19. Si se considera la función booleana f_1 cuya tabla de verdad es

$$t(f_1) = 11111100111100111110110110110111,$$

su puntuación de la medida de complejidad de la forma normal de Fourier es 11, pues (obviando paréntesis innecesarios) tiene como expresión:

$$FNF(f_1) = x_5 \wedge (x_3) \vee \neg x_5 \wedge (\neg x_3 \wedge (\neg x_1)) \vee x_4 \wedge (x_2) \vee \neg x_4 \wedge (\neg x_2) + x_3 \wedge (x_1)$$

No obstante, si se considera su negada, es decir, la función de tabla de verdad

$$t(\neg f_1) = 00000011000011000001001001001000,$$

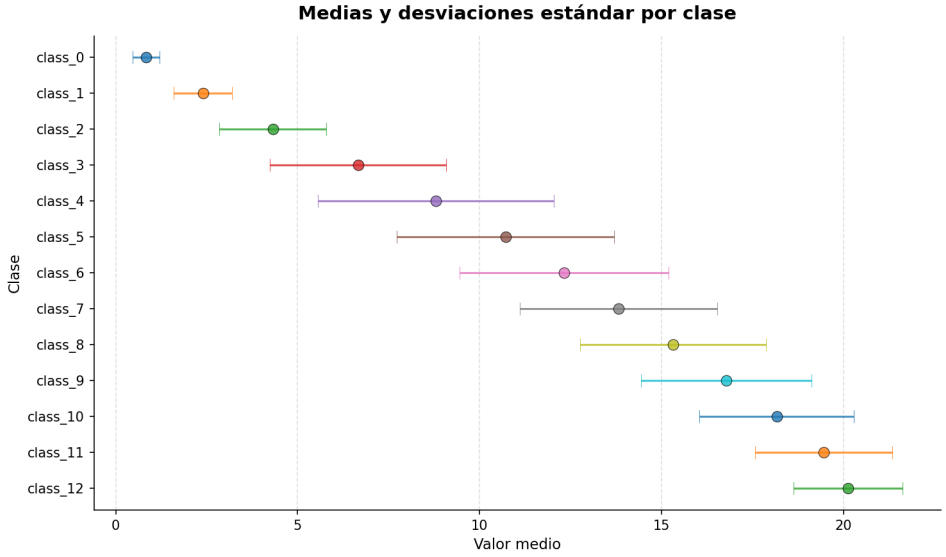


Figura 3.4: Comparación de los parámetros de las distribuciones normales asociadas a los datos de la medida de complejidad de la Definición 3.18 por clase de tamaño.

entonces la puntuación es 22 porque la expresión es

$$\begin{aligned} FNF(\neg f_1) =& (x_5 \wedge (x_4 \wedge (\neg x_3 \wedge (\neg x_2)))) \vee \\ & \neg x_4 \wedge \neg x_3 \wedge x_2 \vee \\ & \neg x_5 \wedge (x_4 \wedge (x_3 \wedge (\neg x_2 \wedge (\neg x_1)))) \vee \\ & \neg x_4 \wedge (x_3 \wedge (x_2 \wedge (\neg x_1)))) \vee \\ & x_4 \wedge (\neg x_3 \wedge (\neg x_2 \wedge (x_1))) \vee \\ & \neg x_4 \wedge (\neg x_3 \wedge (x_2 \wedge (x_1))) \end{aligned}$$

Un posible intento de solución sería calcular tanto $FNF(f)$ como $FNF(\neg f)$ y seleccionar la de menor complejidad; para ello, bastaría con adaptar la definición original de modo que se admitan negaciones entre los nodos de la estructura. Sin embargo, conviene notar que $FNF(\neg f)$ se construye a partir de $FND_{min}(\neg f)$, que coincide exactamente con la negación de $FNC_{min}(f)$. Por lo tanto, si $FND_{min}(\neg f)$ es "sencilla", $FNC_{min}(f)$ también lo será. Esto demuestra que no hace falta introducir negaciones entre los nodos del árbol; en su lugar, basta con permitir que f se descomponga tanto en su forma normal conjuntiva como en su forma normal disyuntiva para construir $FNF(f)$. En adelante diferenciaremos ambas formas como FNF_{fnd} y FNF_{fnc}.

Es sencillo preever cómo debería cambiar la Definición 3.15 para poder implementar recursivamente $FNF_{fnc}(f)$ a partir de la $FNC_{min}(f)$. El motor de cálculo implementa ambas versiones de la $FNF(f)$ y permite observar que para la f del Ejemplo 3.19, en efecto, se tiene que $FNF_{fnd}(f) = FNF_{fnc}(\neg f)$ y que $FNF_{fnd}(\neg f) = FNF_{fnc}(f)$. Lo interesante sería definir $FNF(f)$ permitiendo que algunos nodos se construyan a partir de la $FNF_{fnd}(f)$ y otros nodos a partir de la FNF_{fnc} en una misma estructura sintáctica, es decir, o bien se tiene

$$FNF(f) = (x_i \wedge FNF(f_1)) \vee FNF(f_2)$$

o bien se tiene

$$FNF(f) = (x_i \vee FNF(\tilde{f}_1)) \wedge FNF(\tilde{f}_2)$$

en función de cual sea la descomposición cuya medida de complejidad es mínima y así recursivamente.

Por desgracia, el tiempo de cómputo de los valores de la medida de complejidad de esta nueva definición de la forma normal de Fourier no es factible para los recursos hardware disponibles por nuestra parte, por lo que no se ha podido probar exhaustivamente su resultado. No obstante, cabe destacar que esta tampoco es la solución correcta. La razón es que la solución que buscamos no consiste en elegir vorazmente la versión mínima de cada nodo hijo, sino que buscamos que la intersección de las subfunciones de ambos hijos de cada nodo sea máxima, para que la medida de complejidad sea lo más pequeña posible. Este es un problema mucho más difícil, de lejos más costoso y desde luego abierto a explorar.

Capítulo 4

Limitaciones y restricciones

Los dos capítulos anteriores, si bien muestran vías distintas para abordar el problema de $NP \subset P/poly$, dejan abiertas varias preguntas con respecto a cuáles deberían ser los próximos pasos. Es muy razonable que tras observar la demostración completa de la cota inferior para los *affine dispersers* uno se pregunte hasta dónde será posible llegar con la técnica de eliminación de puertas; o que tras observar los resultados empíricos del estudio experimental se pregunte cómo deberían ser las métricas que tengan garantías de éxito. La intención de este capítulo es orientar cualquier trabajo futuro que pueda seguir las mismas líneas que aquí se presentan, mostrando una serie de resultados que ponen de manifiesto los límites de las técnicas utilizadas y las características que deben tener todos los intentos de demostración de este resultado que pretendan ser exitosos.

Por un lado, la dirección que debe tomar cualquier intento de separar las clases NP y $P/poly$ queda orientado gracias al trabajo de Razborov y Rudich en [17]. Este permite explicar el porqué del estancamiento de la mayoría de demostraciones de separación de clases de complejidad. El resultado explica que los intentos de demostración que cumplen la definición de lo que denominan *demostraciones naturales* no están teniendo éxito no porque no seamos lo suficientemente habilidosos, sino porque su éxito es tan difícil como lo sería encontrar un algoritmo que rompa los generadores pseudoaleatorios y estos hasta el momento se consideran seguros. Incumplir la definición de demostración natural requiere que la propiedad que pretenda usarse sea o muy costosa de computar sobre la tabla de verdad o sea cierta para un conjunto muy reducido del total de funciones booleanas, luego estas serán las propiedades que buscaremos en las medidas de complejidad actuales y futuras.

Por otra parte, el arduo desarrollo para demostrar la mejor cota encontrada hasta el momento sugiere una incapacidad de, o bien la técnica de eliminación de puertas o bien nuestra forma de construir las demostraciones que la utilizan para demostrar cotas superpolinómicas. En el artículo [9] los autores Golovnev et al. estudian en profundidad estas cuestiones y demuestran que existen funciones muy resistentes a la técnica de eliminación de puertas, de manera que para estas no se pueden demostrar cotas superpolinómicas con esa técnica. En consecuencia, cualquier demostración que no incorpore en la topología del circuito información específica de la familia de funciones utilizada no podrá demostrar cotas superpolinómicas así, pues la distinción de casos estaría basada sobre un circuito genérico que bien podría ser el de una de estas funciones resistentes.

4.1. Demostraciones naturales

Durante la década de los 90, ante el estancamiento de la comunidad científica para obtener demostraciones que separaran clases de complejidad —específicamente el desafío de probar que $NP \not\subseteq P/poly$—, Razborov y Rudich se propusieron investigar la raíz de la parálisis. El resultado de su trabajo fue presentado en [17]. En él se prueba que la inmensa mayoría de intentos por separar las clases $P/poly$ y NP se habían basado hasta el momento en demostrar que los elementos de $P/poly$ poseen una cierta propiedad de la que carece alguno de los de NP y que habitualmente la propiedad utilizada cumplía una serie de requisitos que hacían muy poco probable que pudiese tener éxito. A este tipo de demostraciones en las que se hace uso de una propiedad poco recomendable es a lo que denominaron *demostraciones naturales*.

Para empezar, lo primero es definir qué entendemos por "propiedad". En ocasiones, utilizamos este término para hablar del motivo por el que una serie de funciones booleanas guardan relación entre sí. En otras, hablamos de una "propiedad" describiendo una condición formal que deben cumplir todas las funciones booleanas que decimos que la verifican. En realidad, el verdadero factor común a todas estas formas de hablar de "propiedades" es el hecho de que se refieren a un subconjunto de funciones $C_n \subset \mathcal{F}_n$. De este modo, de ahora en adelante consideraremos indistinguible una "propiedad" del conjunto de funciones booleanas que delimita.

Definición 4.1 (Propiedad natural)**.** Sean Γ y Λ dos clases de complejidad. Decimos que una propiedad combinatoria C_n es una propiedad Γ-*natural de densidad* δ_n si contiene un $C_n^* \subseteq C_n$ que verifica:

- Constructividad: el problema de decisión $f_n \overset{?}{\in} C_n^*$ está en Γ.
- Densidad: $|C_n^*| \geq \delta_n \cdot |\mathcal{F}_n|$

Por ejemplo, una propiedad P-natural de densidad $\delta_n = 2^{-O(n)}$ es aquella que es capaz de comprobarse en tiempo polinómico —con respecto a la tabla de verdad— para cierto subconjunto suficientemente grande de funciones que la verifican. Concretamente, debe ser comprobable polinómicamente para al menos $|\mathcal{F}_n| / 2^{O(n)}$ del total de funciones.

Definición 4.2 (Propiedad útil)**.** Sea Λ una clase de complejidad. Decimos que una propiedad combinatoria C_n es una *propiedad útil contra* Λ si $\forall \{f_n\}_{n=1}^{\infty}$ tal que $\exists \{f_{n_k}\}_{k=1}^{\infty} \subset C_n$ la sucesión $\{f_n\}_{n=1}^{\infty} \notin \Lambda$.

Por ejemplo, supongamos que el circuito mínimo de los *affine dispersers* hubiese sido necesariamente exponencial. En ese caso, cualquier problema de decisión que verifique que su sucesión de funciones —en el número de entradas— tiene una subsucesión de *affine dispersers* no podría pertenecer a la clase $P/poly$ porque siempre habría instancias del problema que no serían no computables por circuitos polinómicos. Por tanto, "ser *affine disperser*" sería una propiedad útil contra $P/poly$.

Definición 4.3 (Demostraciones naturales)**.** Cualquier prueba de una cota inferior acerca de la no pertenencia de una función explícita al conjunto Λ se dice $\Gamma-$natural contra Λ con densidad δ_n si enuncia una $\Gamma-$propiedad natural de densidad δ_n que además es útil contra Λ.

El resultado principal, particularizado para $P/poly$ y NP, establece que cualquier demostración natural de densidad $2^{-O(n)}$ que tuviese éxito rompería la seguridad de los generadores pseudoaleatorios y creemos casi con total seguridad que esto no es posible. Los generadores pseudoaleatorios o PRG son funciones sobre cadenas booleanas que dada una entrada de longitud $k_1 \in \mathbb{N}$ —llamada semilla—

generan una cadena de longitud $k_2 \in \mathbb{N}$ que consideramos el valor aleatorio. El término "seguro" se utiliza para afirmar que las cadenas que genera son indistinguibles de cadenas aleatorias computacionalmente, es decir, que, pese a no ser aleatorio, distinguirlo de un generador verdaderamente aleatorio no es factible desde un punto de vista computacional.

Definición 4.4 (Dureza de un generador pseudoaleatorio)**.** Sea $G_k : \mathbb{F}_2^k \to \mathbb{F}_2^{2k}$ un generador pseudoaleatorio y sean $x \in \mathbb{F}_2^k$ e $y \in \mathbb{F}_2^{2k}$ dos cadenas aleatorias en sus respectivos espacios. Definimos la *dureza* de G_k como el mínimo S para el que existe un circuito de tamaño menor que S tal que:

$$|\mathbf{P}\left[C(G_k(x)) = 1\right] - \mathbf{P}\left[C(y) = 1\right]| \geq \frac{1}{S}.$$

A la diferencia anterior se la denomina *ventaja* y se denota por ΔP.

Proposición 4.5. *Si un generador $G_k : \mathbb{F}_2^k \to \mathbb{F}_2^{2k}$ es S-duro, no existe ningún circuito de tamaño T significativamente menor que S de manera que la ventaja sea mayor o igual que $1/S$.*

Demostración. Supongamos que no. En ese caso existe un circuito C de tamaño $T < S$ de manera que su ventaja es $\frac{1}{S}$. No obstante, en ese caso

$$|\mathbf{P}\left[C(G_k(x)) = 1\right] - \mathbf{P}\left[C(y) = 1\right]| \geq \frac{1}{S} > \frac{1}{T},$$

luego S en realidad no cumplía la definición de dureza. □

Un PRG competente es indistinguible de una fuente verdadera de aleatoriedad, es decir, que si se genera una secuencia de M cadenas nadie debería ser capaz de distinguir si las generó nuestro PRG o una fuente verdaderamente aleatoria. Si se dispone de un circuito cuya ventaja es positiva, por muy pequeña que sea, es posible construir un algoritmo para descubrir si efectivamente la secuencia es o no aleatoria. La media de veces X que el circuito responde 1 es un estimador puntual de la probabilidad tras la distribución aleatoria de las M cadenas. Para decidir entre la probabilidad de que sean aleatorias $\mathbf{P}\left[C(y) = 1\right]$ y la probabilidad de que sean pseudoaleatorias $\mathbf{P}\left[C(G_k(x)) = 1\right]$, es necesario definir sendas regiones de aceptación para el valor de la estimación de X. Una posible forma de hacerlo es establecer una frontera a medio camino entre ambas, es decir, escoger el punto medio $T = (\mathbf{P}\left[C(y) = 1\right] + \mathbf{P}\left[C(G_k(x)) = 1\right])/2$ como separación. La desigualdad de Chernoff proporciona una cota superior para estimar la probabilidad de que X esté demasiado lejos de su verdadero valor, es decir, la probabilidad de rechazar la probabilidad correcta

$$\mathbf{P}\left[|X - p| > \delta\right] \leq 2e^{-2M\delta^2}.$$

Por tanto, basta con escoger $\delta = \frac{\Delta P}{2}$ para que la frontera de rechazo de ambas coincida en T. De este modo, la probabilidad de error queda acotada de la siguiente manera:

$$\mathbf{P}\left[ERROR\right] \leq 2e^{-2M\left(\frac{\Delta P}{2}\right)^2} = 2e^{\frac{-M\Delta P^2}{2}}.$$

Una elección suficiente[1] del nivel de confianza sería exigir que la probabilidad de error sea como mucho del orden de $2^{-O(n)}$, lo que se traduce en que $M\Delta P^2 \sim n$. La definición de dureza es clave en este punto. Si nuestro algoritmo es factible computacionalmente deberá computarse como mucho por circuitos de complejidad $S \in O(2^n)$ y, en ese caso, bastaría con que nuestro PRG fuese $2^{O(n)}-$duro para garantizar

[1] Esta elección garantiza que la probabilidad de error sea significativamente menor que la de sufrir un fallo de hardware por un rayo cósmico (para un n suficientemente grande).

que romper el generador no es factible computacionalmente, ya que requiere una cantidad exponencial de muestras hacerlo:

$$M\Delta P^2 \sim n \Leftrightarrow M \sim \frac{n}{\Delta P^2} \Rightarrow M \in O(n \cdot 2^n),$$

Por tanto, distinguir el PRG de una fuente aleatoria con la suficiente confianza es infactible computacionalmente aún utilizando un algoritmo de complejidad polinómica. Por este motivo, consideramos que los PRG $2^{O(n)}-$duros son seguros; porque o bien si se quiere tener una ventaja igual de potente se requiere un número exponencial de muestras o bien si se quiere un número de muestras razonables hace falta un circuito excesivamente grande.

La prueba del teorema principal se basa en el hecho de que, de existir una demostración natural, habríamos encontrado un distinguidor polinómico que rompería los PRGs que consideramos seguros.

Teorema 4.6. *Si existe una función $2^{n^\varepsilon}-$dura, entonces no existe ninguna prueba P/poly-natural de densidad $2^{-O(n)}$ contra P/poly.*

Demostración. Supongamos, por reducción al absurdo, que existe una propiedad natural C_n de densidad $2^{-O(n)}$ contra *P/poly* y, sin pérdida de generalidad, supongamos que $C_n = C_n^*$. Por la definición de prueba natural, C_n cumple tres condiciones sobre el conjunto de las funciones booleanas de n variables (cuyas tablas de verdad tienen tamaño $N = 2^n$):

1. Constructibilidad: C_n es computable por un algoritmo que se ejecuta en tiempo polinómico respecto al tamaño de la tabla de verdad, es decir, existe un circuito de tamaño $2^{O(n)}$ que evalúa C_n.
2. Densidad: C_n es satisfecha por una fracción significativa de todas las funciones posibles. Si tomamos una tabla de verdad $y \in \{0,1\}^N$ uniformemente al azar, $\mathbf{P}[C_n(y) = 1] \geq \frac{1}{N^d} = 2^{-O(n)}$ para algún $d > 0$.
3. Utilidad: ninguna función que pueda ser calculada por un circuito de tamaño polinómico en n satisface C_n.

Por hipótesis del teorema, existe una función 2^{n^ε}-dura que puede utilizarse (haciendo uso de construcciones criptográficas estándar como las presentadas en [8]) para construir un generador pseudoaleatorio $G : \{0,1\}^k \to \{0,1\}^N$ que sea 2^{k^δ}-duro y donde la longitud de la semilla k sea polinómica con respecto a n. Si se escoge un k suficientemente grande, podemos garantizar que el PRG tiene una dureza S significativamente mayor que $2^{O(n)}$, es decir, que la mayor ventaja que se puede obtener es significativamente menor que $\frac{1}{2^{O(n)}}$.

Utilicemos ahora nuestra propiedad natural C_n como el distinguidor estocástico C introducido anteriormente y evaluemos su comportamiento sobre cadenas de longitud N:

- Si la cadena es verdaderamente aleatoria, la probabilidad de que el distinguidor devuelva 1 es, por la condición de densidad, $\mathbf{P}[C_n(y) = 1] \geq 2^{-dn}$.
- Si la cadena es pseudoaleatoria, $y = G(x)$ con $x \in \{0,1\}^k$, entonces y es la tabla de verdad de una función de n variables que se puede calcular simulando el propio generador G. Como G se evalúa en tiempo polinómico en n, esta función tiene una complejidad de circuito pequeña y, por la utilidad de C_n, no puede cumplir la propiedad, luego $\mathbf{P}[C_n(G(x)) = 1] = 0$.

En consecuencia, la ventaja de nuestro distinguidor C_n es:

$$\Delta P = |\mathbf{P}\left[C(y) = 1\right] - \mathbf{P}\left[C(G_k(x)) = 1\right]| \geq 2^{-dn} - 0 = 2^{-dn} = 2^{-O(n)}$$

Hemos construido un circuito C_n con una ventaja de $\frac{1}{2^{O(n)}}$ y de tamaño $2^{O(n)}$, pero la dureza del PRG garantizaba que la ventaja máxima obtenible por circuitos $2^{O(n)}$ era significativamente menor que $\frac{1}{2^{O(n)}}$. Por tanto, hemos violado la dureza del generador pseudoaleatorio y esto entra en contradicción con la existencia de la función 2^{n^ε}-dura. □

Indudablemente, las consecuencias del Teorema 4.6 dependen del hecho no probado de que las hipótesis sean ciertas. Sin embargo, esta conjetura está ampliamente aceptada dentro de la comunidad científica por el uso intensivo de generadores pseudoaleatorios en aplicaciones críticas como la ciberseguridad. La consecuencia fundamental, más allá de que la conjetura sea o no cierta, es que separar NP de $P/poly$ haciendo uso de una demostración natural es al menos tan difícil como romper la seguridad de los PRGs $2^{O(n)}$-seguros. Por tanto, cualquiera que decida intentar probar el problema haciendo uso de una demostración natural debe saber que nada le impide tener éxito, pero que la dimensión de la tarea es equiparable a romper la seguridad de Internet; por lo que es recomendable no buscar propiedades naturales y útiles contra $P/poly$, es decir, debemos buscar aquellas que o bien no sean constructivas o bien no sean densas.

Un ejemplo representativo de cómo este resultado puede dirigir la búsqueda de medidas de complejidad es la propuesta en la Definición 3.10. A pesar de que la intuición estaba fundamentada sobre argumentos sólidos, hemos podido descartar directamente la medida porque su cómputo era polinómico con respecto al tamaño de la tabla de verdad, como ilustra el Algoritmo 1 del Apéndice A. Este hecho, sumado a que sabemos que la mayoría de funciones booleanas deben computarse por circuitos grandes, hace que, de estar verdaderamente relacionada con el tamaño del circuito, la medida de complejidad pueda considerarse una demostración natural.

4.2. Límite de la técnica de eliminación de puertas

Los resultados del Capítulo 2 han puesto de relieve la dificultad para continuar elevando la cota inferior de la función más compleja —en términos del tamaño de su circuito— que hemos sabido encontrar y demostrar. El inmenso despliegue técnico y teórico que requiere aumentar menos de una décima la cota anterior es indicativo de una limitación inherente de la técnica de eliminación de puertas o de la forma en que la estamos utilizando para construir las demostraciones. Precisamente esta es la conclusión a la que llegan en [9] parte de los autores de las cotas anteriores, donde estudian el alcance y limitaciones que tiene esta herramienta para buscar cotas superpolinómicas. El resultado de su trabajo es un enunciado similar al de las demostraciones naturales: cierto esquema de demostración nunca podrá superar las cotas lineales haciendo uso de la técnica de eliminación de puertas. Por tanto, o bien encontramos una nueva técnica o bien debemos asegurarnos de construir adecuadamente las demostraciones que hagan uso de esta.

Generalmente, las demostraciones basadas en la técnica de eliminación de puertas que hemos visto requieren de una medida de complejidad no negativa μ, una familia de sustituciones S y un decremento constante δ y se fundamentan en probar dos enunciados:

- La medida de complejidad μ es creciente con respecto al tamaño del circuito.
- Para cualquier función $f \in \mathcal{F}_n$, una sustitución $\rho \in S$ o bien trivializa f o bien decrementa la medida al menos δ unidades.

El primer punto es el que nosotros generalizamos con el Teorema 2.14 y es el que permite utilizar cotas inferiores de la medida de complejidad para acotar inferiormente el tamaño del circuito (como hemos

visto en el Capítulo 2). El segundo punto es el que se prueba a través de una distinción exhaustiva de casos y el hecho de que sea "para toda función" es el elemento clave de la limitación de la técnica de eliminación de puertas. La razón es que la información relativa a la familia de funciones con las que se trabaja (por ejemplo, los *affine dispersers* en el Capítulo 2) nunca participa en definir cómo es la topología de los circuitos que las computan. Únicamente esta información se utiliza para afirmar que se pueden imponer cierto número de restricciones sin trivializar la función. En consecuencia, la distinción de casos se hace para circuitos arbitrarios que, en principio, podrían computar una función cualquiera. Sin embargo, de existir funciones resistentes (i.e., que no permiten cotas superpolinómicas) a la técnica de eliminación de puertas, cualquier distinción de casos que no particularice la topología del circuito se está haciendo, a priori, también sobre los circuitos de estas, luego no pueden superar las cotas polinómicas.

Este es el primer hecho fundamental que se prueba en [9]: para cualquier sustitución ρ, existen funciones resistentes a la técnica de eliminación de puertas que no permiten alcanzar cotas superpolinómicas. En otras palabras, existen funciones booleanas para las que el decremento de la medida de complejidad está acotado, lo que limita la cota inferior que se puede alcanzar con ellas.

Ejemplo 4.7. Sea $f \in \mathcal{F}_n$ una función cualquiera. Consideremos la función $\tilde{f} = f \diamond \mathrm{MAJ}_3 \in \mathcal{F}_{3n}$ que es el resultado de sustituir la variable de entrada x_i de f por la función mayoría de tres nuevas variables de entrada x_{i1}, x_{i2}, x_{i3}, es decir:

$$f \diamond \mathrm{MAJ}_3 = f\left(\mathrm{MAJ}_3(x_{11}, x_{12}, x_{13}), \ldots, \mathrm{MAJ}_3(x_{n1}, x_{n2}, x_{n3})\right).$$

Ninguna sustitución $x_{ij} \leftarrow \rho$ puede eliminar más de 5 puertas.

En efecto, asumamos sin pérdida de generalidad que la sustitución es $x_{11} \leftarrow \rho$. Denotemos por C al circuito mínimo de $\tilde{f}$, por C' al circuito mínimo de $\tilde{f}|_{x_{11} \leftarrow \rho}$ y por C'' al circuito construido a partir de C' sustituyendo x_{12} y x_{13} por $MAJ_3(x_{11}, x_{12}, x_{13})$, como se muestra en la Figura 4.1. El cambio realizado para construir C'' suprime la sustitución ρ, por lo que C'' computa nuevamente $\tilde{f}$. Por cómo se ha construido C'', por ser 5 el número mínimo de puertas necesarias para computar MAJ_3 y por la optimalidad de C, se tiene

$$|C| \leq |C''| \leq |C'| + 5.$$

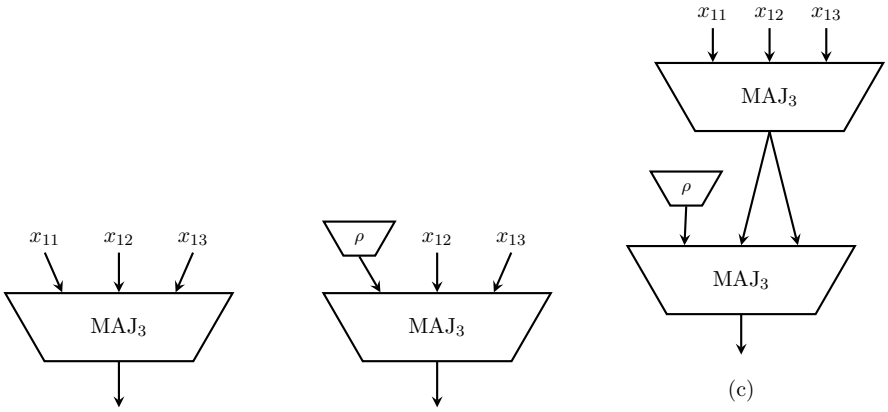
Consecuentemente, la disminución asociada a una sustitución ρ cualquiera está acotada por 5.

Lema 4.8. *Sea $h \in \mathcal{F}_n$ y $m > 0$. La función $f = h \diamond MAJ_{2m+1} \in B_{n(2m+1)}$ satisface las siguientes propiedades:*

- *La complejidad de f es similar a la de h: $gates(h) \leq gates(f) \leq gates(h) + 4{,}5(2m+1)n$.*
- *Para cualquier m-sustitución ρ, $gates(f) - gates(f|_\rho) \leq 4{,}5(2m+1)m$.*

El Ejemplo 4.7 muestra una función para la cual no es posible eliminar más de 5 puertas a través de sustituciones, y el Lema 4.8, la generalización de este límite superior en función del gadget utilizado. Si la medida de complejidad es directamente el tamaño del circuito, entonces queda demostrado que hay funciones resistentes a la técnica de eliminación de puertas para esa medida. No obstante, generalmente la medida de complejidad tiene más elementos además del tamaño del circuito (por ejemplo, la medida de la Definición 2.15) y, en ese caso, no queda probado nada al respecto[2]. En consecuencia, es necesario generalizar estos resultados para más medidas de complejidad posibles.

[2]Por ejemplo, para la medida de complejidad de la Definición 2.15 que se ha mencionado bastaba con probar que se podía disminuir la medida lo suficiente entre puertas eliminadas y variables desconectadas; por tanto, nada impide que se pueda disminuir lo suficiente la función del Ejemplo 4.7, pues podría compensarse con variables desconectadas.



(a) Representación gráfica de la parte correspondiente a C.

(b) Representación gráfica de la parte correspondiente a C'.

(c) Representación gráfica de la parte correspondiente a C''.

Figura 4.1: Representación gráfica de las transformaciones de la parte relevante del circuito entre C, C' y C'' recogida en [9].

Definición 4.9 (Medida de complejidad subaditiva)**.** Sea $\mathcal{F}_{n,m}$ el conjunto de funciones booleanas de n entradas y m salidas. Decimos que una función $\mu : \bigcup_{n,m\in\mathbb{N}} \mathcal{F}_{n,m} \to \mathbb{R}$ es una *medida de complejidad subaditiva* si:

- Para todas las funciones $s_1\mathcal{F}_{n_1,m_1}$ y $s_2 \in \mathcal{F}_{n_2,m_2}$, se verifica $\mu((s_1, s_2)) \leq \mu(s_1) + \mu(s_2)$, donde $(s_1, s_2) \in \mathcal{F}_{(n_1+n_2),(m_1+m_2)}$ es la "mezcla" de s_1 y s_2 que comprende las entradas y salidas (independientes) de ambas funciones.
- Para todas las funciones $g \in \mathcal{F}_{n,m}$ y $f \in \mathcal{F}_{(m+r),k}$, se verifica $\mu(h) \leq \mu(f) + \mu(g)$, donde $h(x, y) = f(g(x), y) \in \mathcal{F}_{(n+r),k}$.

Definición 4.10 (m-estabilidad débil)**.** Una función $g(X)$ es *débilmente m-estable* si, para todo $Y \subseteq X$ de tamaño $|Y| \leq m$, existen dos asignaciones $\tau_0, \tau_1 : X \setminus Y \to \{0, 1\}$ a las variables restantes tales que $g|_{\tau_0}(Y) \equiv 0$ y $g|_{\tau_1}(Y) \equiv 1$.

Nota 4.11. La Definición 4.10 se puede interpretar como que, tras la asignación τ_0 (τ_1), la función no depende de las variables restantes Y.

Teorema 4.12. *Sean μ una medida subaditiva, $f \in \mathcal{F}_n$ cualquier función, $g \in \mathcal{F}_k$ una función débilmente m-estable y $h = f \diamond g \in \mathcal{F}_{nk}$. Para cada m-sustitución ρ, se tiene*

$$\mu(h) - \mu(h|_\rho) \leq m \cdot \mu(g).$$

Demostración. Por brevedad, la demostración detallada se omite aquí y puede consultarse en [9]. □

El Teorema 4.12 es una herramienta excelente para mostrar que existen funciones resistentes para ciertas medidas de complejidad. Si la disminución de la medida está acotada superiormente por γ para m-sustituciones, entonces la cota inferior se deduce de realizar n/m sustituciones de este tipo asumiendo que se alcanza el límite superior:

$$\mu(f) \geq \gamma \frac{n}{m}$$

Cuando la medida de complejidad es, por ejemplo, la de la Definición 2.15, entonces

$$|C_f| + n(C_f) \geq \gamma\frac{n}{m} \Rightarrow |C_f| \geq \gamma\frac{n}{m} - n(C_f) \geq \gamma\frac{n}{m} - n = \left(\frac{\gamma}{m} - 1\right) n$$

y como $\gamma = m \cdot \mu(g) = m \cdot (|C_g| + n(C_g)) \leq m \cdot (|C_g| + k) = c \in O(1)$, entonces la mejor cota inferior vendría dada por $(c-1)n \in O(n)$.

No obstante, hemos visto en el Capítulo 2 otras medidas de complejidad para las que este resultado no nos es útil. Por ejemplo, consideremos la medida $\mu_\alpha(f) = |C_f| + \alpha n(C_f)$ con $\alpha > 0$. En ese caso, podemos hacer un razonamiento análogo para deducir la cota inferior en términos del límite superior del decremento de la medida de complejidad:

$$|C_f| + \alpha n(C_f) \geq \gamma\frac{n}{m} \Rightarrow |C_f| \geq \gamma\frac{n}{m} - \alpha n(C_f) \geq \gamma\frac{n}{m} - \alpha n = \left(\frac{\gamma}{m} - \alpha\right) n,$$

pero ahora $\gamma = m \cdot (|C_g| + \alpha n(C_g)) \leq m \cdot (|C_g| + \alpha k) \in O(\alpha)$, luego la mejor cota inferior viene dada por:

$$\left(\frac{\gamma}{m} - \alpha\right) n = (k \cdot \alpha + c - \alpha)n = ((k-1)\alpha + c)n \in O(\alpha n)$$

En consecuencia, uno podría escoger α tan grande como necesite para demostrar que la cota puede ser tan grande como se quiera. Por ejemplo, si escogemos $\alpha \sim \log n$ habríamos demostrado una cota superlineal.

Para dar un límite superior más ajustado en caso de este tipo de medidas de complejidad en las que se utiliza una suma ponderada, los autores proponen el siguiente desarrollo.

Definición 4.13 (m-estabilidad)**.** Una función $g(X)$ es m-estable si, para todo $Y \subseteq X$ de tamaño $|Y| \leq m+1$ y cualquier $y \in Y$, existe una asignación $\tau : X \setminus Y \to \{0,1\}$ de las variables restantes tal que $g|_\tau(Y) \equiv y$ o $g|_\tau(Y) \equiv \neg y$. Es decir, tras la asignación τ, la función depende únicamente de la variable y.

Teorema 4.14. *Sea $f \in \mathcal{F}_n$, $g \in B_k$ una función m-estable y $h = f \diamond g \in \mathcal{F}_{nk}$; entonces, para todo $\alpha \geq 0$ y toda sustitución ρ se tiene*

$$\mu_\alpha(h) - \mu_\alpha(h|_\rho) \leq m \cdot (|C_g| + \alpha).$$

Demostración. Por brevedad, la demostración detallada se omite aquí y puede consultarse en [9]. □

El Teorema 4.14 es el resultado principal y permite probar que existen funciones booleanas para las que la mejor cota inferior posible es lineal cuando se utiliza la técnica de eliminación de puertas. En efecto, la cota inferior de $\left(\frac{\gamma}{m} - \alpha\right) n$ ahora alcanza su valor máximo para $\gamma = m(|C_g| + \alpha) \in O(\alpha)$, así que:

$$\left(\frac{\gamma}{m} - \alpha\right) n = (\alpha + c - \alpha)n = c \cdot n \in O(n).$$

En [9] además se prueba la existencia de ciertas funciones m-estables que permiten particularizar este resultado para imponer la cota $11n - o(n)$. Por tanto, salvo que se incorpore información de la familia de funciones a la distinción de casos del circuito, en principio la cota máxima alcanzable a través de la técnica de eliminación de puertas es $11n - o(n)$.

Capítulo 5

Conclusiones y trabajo futuro

Terminamos la exposición resumiendo las conclusiones a las que hemos llegado tras del estudio exhaustivo de distintas aproximaciones a la búsqueda de funciones booleanas con circuito mínimo de tamaño superpolinómico y proponiendo los próximos pasos que sería interesante explorar por parte de cualquiera que quisiese extender el contenido de este trabajo.

5.1. Conclusiones

La complejidad del modelo de cómputo de las máquinas de Turing ha desplazado la atención hacia modelos más sencillos como el de los circuitos booleanos en los intentos por demostrar que $P \neq NP$. En esa línea se ha abierto un nuevo subproblema: encontrar problemas de decisión que no sean computables por circuitos de tamaño polinómico con respecto al número de sus entradas, y es precisamente ahí donde se está poniendo el foco en los últimos tiempos.

En la búsqueda de estas funciones existen principalmente dos vías. La primera está basada sobre razonamientos directamente relacionados con la topología del circuito mínimo asociado a las funciones booleanas. Para ello, la principal herramienta disponible es la técnica de eliminación de puertas y la mejor cota obtenida hasta el momento es $3{,}1n - o(n)$. Sin embargo, el trabajo reciente que exploramos en la Sección 4.2 del capítulo anterior ha puesto de relieve la manera en que debe aplicarse esta técnica si se quieren alcanzar cotas superpolinómicas y queda abierta la pregunta de cómo construir las demostraciones teniendo en cuenta esas observaciones. La segunda vía está basada en confeccionar medidas de complejidad que razonen sobre propiedades que presumiblemente estén relacionadas indirectamente con el tamaño de los circuitos booleanos. De ser capaces de construir una medida de complejidad únicamente acotada para los problemas de decisión en $P/poly$ y de probar que cierto problema es no acotado, habríamos encontrado precisamente uno de esos problemas para los que sus funciones requieren circuitos superpolinómicos. En esa línea, el trabajo expuesto en la Sección 4.1, también del capítulo anterior, muestra qué condiciones debe verificar toda medida de complejidad si espera tener garantías de éxito.

Siguiendo esta última línea, hemos basado nuestro trabajo en definir medidas de complejidad que traten de explotar la implicación entre tamaño del circuito booleano y las repetitividades aparentes o subyacentes de las tablas de verdad de las funciones booleanas. Hemos concluido que existe una relación directa entre la incompresibilidad de la tabla de verdad y el tamaño del circuito booleano que la computa, probando que existe una base formal que sustenta nuestro trabajo. En ese contexto, la

medida más prometedora ha sido la basada en la forma normal de Fourier, que ha permitido observar una correlación de las pruebas empíricas con el tamaño del circuito mínimo en funciones booleanas de 5 bit en 1 bit. Además, la medida parece prometedora porque verifica las condiciones impuestas por las demostraciones naturales para tener garantías de éxito.

5.2. Trabajo futuro

Los resultados experimentales obtenidos en la Subsección 3.2.1 parecen capturar de alguna manera la complejidad del circuito, equiparándose incluso en la correlación con el tamaño del circuito a algunas de las mejores métricas de trabajos previos en esta línea. Por tanto, parece una línea prometedora a seguir explorando de la que, además, quedan abiertas varias cuestiones interesantes.

En primer lugar, no se ha probado —y, de hecho, parece difícil— un argumento que decida cuál es el valor máximo que puede alcanzar la medida de complejidad y cuál es la función "maliciosa" para la que se alcanza. De la misma manera, parece que el límite inferior dentro de cada clase de tamaño está lo suficientemente cerca de su valor real como para sospechar que el error absoluto podría ser polinómico con respecto a n. De ser capaces de resolver ambos puntos, podríamos seguir probando el resultado aunque la correlación no fuese perfecta. En efecto, si probásemos que la clase de tamaño más alta como mínimo alcanza una puntuación exponencial (para lo que es útil conocer el valor máximo y la función maliciosa) y demostrásemos que el error absoluto del mínimo de cada clase de tamaño es polinómico con respecto al tamaño del circuito mínimo, entonces habríamos probado la existencia de una función con al menos una complejidad de circuito exponencial (las que puntuen el mínimo alcanzable dentro de la clase de tamaño más alta). Si, además, fuésemos capaces de probar que existe un problema NP-completo con infinitas funciones booleanas en esa clase de tamaño, entonces habríamos demostrado que ese problema no pertenece a $P/poly$ y, entonces, que $P \neq NP$.

En segundo lugar, la distribución del valor medio de las puntuaciones por clase de tamaño sigue una progresión monótona que parece respetar la monotonía del tamaño del circuito mínimo y la desviación típica dentro de cada clase no parece excesivamente alta (en línea con lo comentado en el párrafo anterior). Sería interesante seguir explorando cómo podemos disminuir la desviación típica dentro de las distribuciones de cada clase refinando la forma normal de Fourier con variaciones como la que se presenta en la Subsección 3.2.2. Probablemente, la línea de trabajo más útil en este sentido sea abordar un criterio de selección de implicantes primos para la generación de la FND mínima desde la que se construye la FNF, pues en nuestro desarrollo ese punto solo ha tenido el cometido de hacer la representación única, pero no óptima.

Por último, los armónicos fundamentales parecen "canónicos" en el sentido de que verdaderamente representan los bloques fundamentales con los que están construidos el resto de funciones booleanas. No obstante, la selección de las operaciones booleanas a través de las que son combinados ha sido $\{\wedge, \vee\}$ por elección propia y sería interesante estudiar el comportamiento de la medida de complejidad en base a otro conjunto universal de puertas. Además, sería interesante estudiar cómo se comporta la composición de armónicos con respecto a la permutación de las entradas, pues en principio debería conservar la dificultad entre funciones booleanas dentro de la misma clase de equivalencia dada por [13].

Introduction

The P vs NP problem is classified as one of the Millennium Prize Problems, and for good reason: it is not only the most important open problem in theoretical computer science, but it also questions the very nature of problem-solving. The central question—"is finding a solution inherently more difficult than verifying it?"—may seem simple, but it has remained unanswered since its formulation in the 1970s. Furthermore, this precise problem has defined the boundaries of what we consider computationally feasible or not. If $P = NP$ were to be proven (via a constructive proof and under the standard assumption that efficiently means solvable in polynomial time), a multitude of problems such as protein folding or automated theorem proving would be solved efficiently. This would transform our society by enabling unprecedented advances in almost every field of knowledge where complexity acts as an obstacle.

Nonetheless, intuition and experience suggest that the answer is $P \neq NP$—even though no one has been able to prove it formally—which is why most of the academic world directs its work in that direction. Among the various strategies that have been employed, in recent years efforts have focused on trying to separate the classes P and NP through the complexity of their Boolean circuits. The theoretical argument is based on the class $P/poly$, which is composed of all decision problems computable by polynomial-size circuits, and consists of proving that there exists a Boolean function in NP whose minimum circuit is of super-polynomial size. Although it is one of the most promising strategies, it is also one of the most elusive. We know that the vast majority of Boolean functions cannot be computed using polynomial circuits, but paradoxically, we have not been able to find a single one, let alone find one specifically within NP.

In this context, it is of particular interest to study what are the best lower bounds on Boolean circuit complexity that we have managed to prove and for which functions we have done so. On one hand, this will allow us to understand our most powerful tool for proving these bounds and how it is used, namely, the gate elimination technique. On the other hand, we will be able to observe firsthand the difficulties that arise almost from the very beginning when trying to break through the linearity barrier.

For our part, we approach the problem from a different methodology. Instead of proving lower bounds on circuit size, we attempt to experimentally study whether certain complexity measures allow us to characterize the complexity of the minimum circuit that computes a Boolean function. In particular, we define these metrics based on the repetitiveness (either evident or underlying) of the truth table.

Finally, we attempt to guide the efforts of future attempts to solve the problem by presenting the main results regarding the limitations of the gate elimination technique and the characteristics of complexity measures. On one hand, we present functions that are resistant to the gate elimination

technique to rule out proofs that do not take into account the particularities of the family of functions being analyzed. On the other hand, we present the fundamental result of natural proofs to justify why it is possible to rule out certain complexity measures even before analyzing any of their experimental results. Lastly, we discuss the potential for improvement of our own complexity measures and propose future lines of research.

Background and State of the Art

By the mid-20th century, the definition of computability (thanks to Church and Turing) had already formally differentiated between problems that could be solved by a computer and those that could not: the term "undecidability" was born. However, the fact that a problem is solvable does not imply that its solution can be found within a reasonable timeframe. We know of problems solvable in polynomial time and problems that are only solvable in exponential time; nevertheless, the real question is raised by those problems whose solutions are easy to verify (i.e., in polynomial time), but for which we have not yet found an efficient way to discover them.

The P vs NP Problem

The class P (Polynomial Time) comprises all those decision problems that can be solved by a deterministic Turing machine in a number of steps whose order of complexity is $O(n^k)$ —for some k— where n is the input size. For instance, determining whether a list of n elements is sorted is a decision problem belonging to this class. On the other hand, the class NP does not mean "Non-Polynomial", but rather Nondeterministic Polynomial Time. This class groups problems for which, given a candidate solution, it can be verified in a polynomial number of steps by a deterministic Turing machine[1]. A classic example is the Sudoku problem: solving an $n \times n$ Sudoku puzzle becomes extremely difficult as n grows, but if someone hands us a pre-filled grid, verifying that there are no repeated numbers in rows, columns, or subgrids is a trivial and fast (polynomial) task. It is evident that $P \subset NP$, so the great question lies in knowing whether $P = NP$ or, on the contrary, $P \subsetneq NP$.

In 1971, Stephen A. Cook introduced the class of NP problems, the concept of *NP-completeness*, and the notion of polynomial reducibility in [4]. Almost simultaneously and independently in the USSR, Leonid A. Levin reached similar conclusions in [14], which is why today we know the cornerstone of this theory as the Cook-Levin Theorem. This theorem proved that the Boolean satisfiability problem (SAT) is "as hard as any other" problem in the NP class: if SAT can be solved efficiently, then all problems in NP can be as well. This property is known as NP-completeness, and since its definition, a multitude of other problems have been found to be NP-complete.

The Cook-Levin Theorem leads to an evident conclusion. If an NP-complete problem is in P, then all of $NP \subset P$, and it would be proven that $P = NP$. Therefore, the question is clear: can SAT (or any other NP-complete problem) be solved in polynomial time? This was the seed of the general problem we know today as P vs NP, which consists of proving or disproving the statement $P = NP$ through the study of $NP \subset P$—that is, elucidating whether verifying a solution is computationally as difficult as finding it. If true, the conjecture would imply that any problem whose solution we can verify quickly also possesses a hidden algorithm to be solved quickly. Nevertheless, most of the scientific

[1] In reality, the formal definition of the class consists of all those problems that can be solved in a polynomial number of steps by a nondeterministic Turing machine, but this is an equivalent and more intuitive definition for the exposition of this chapter.

community leans toward the $P \neq NP$ option based on the fact that, after decades of intensive search, no polynomial algorithms have been found for the hardest problems in NP.

In the year 2000, the Clay Mathematics Institute elevated the status of this problem by including it among the seven Millennium Prize Problems, offering a one-million-dollar prize to whoever achieves a formal proof of $P = NP$ or $P \neq NP$. The reason is that its resolution —aside from being difficult— could have a massive impact even beyond mathematics. On one hand, if it were proven that $P = NP$ (under the conditions mentioned earlier), the world as we know it would change radically. Most public-key cryptography systems (such as RSA or the Diffie-Hellman protocol) rely on the asymmetry between the ease of multiplying two prime numbers (in P) and the difficulty of factoring the result (in NP); thus, if $P = NP$ were true, connections would no longer be secure. On the other hand, logistical optimization, drug design, and automated mathematical theorem proving would become trivial tasks, exponentially accelerating scientific progress and technological development. On the alternative hand, if it were proven that $P \neq NP$, human intuition would be confirmed: finding a solution is harder than judging it, and solving a puzzle is intrinsically more complex than checking if it is done correctly. From a technical standpoint, it would validate the security of our current systems and allow the community to focus on approximation algorithms and heuristics, knowing that the exact optimal solution is unreachable for large datasets.

The Class $P/poly$ and Circuit Complexity

In the efforts of recent decades, a third complexity class has emerged to try to resolve the problem: the class $P/poly$. The class $P/poly$ is defined as the set of languages that can be decided by a family of Boolean circuits $\{C_n\}_{n\in\mathbb{N}}$ of polynomial size. Unlike standard complexity classes, $P/poly$ represents a non-uniform model of computation. In a uniform model, the recognizer of a language must recognize all strings within it, regardless of their size n; conversely, in a non-uniform model, we allow a family of recognizers —possibly different from one another— for each input size n. For example, regular expressions are a uniform model of computation, as the finite automaton does not change between instances of the language; however, Boolean circuits are not a uniform model of computation, since changing the number of inputs also changes the circuit used to compute those same Boolean functions.

While analyzing computational complexity on a Turing machine requires examining the execution trace within it, the computation model of Boolean circuits is static and combinatorial: we can count their gates, measure their depth, and analyze their topology as an acyclic graph. For this reason, even though the Turing machine model is the theoretical standard for defining computability, the scientific community has shifted toward attempts to measure computational complexity in this alternative model of computation.

The reasoning behind trying to prove $P \neq NP$ by leveraging $P/poly$ is as follows. We know that[2] is a problem in $P/poly$ but is undecidable, meaning it cannot be in P. $P \subset P/poly$, since any problem computable by a Turing machine in polynomial time can be computed by a family of polynomial circuits. If we could manage to find a problem in NP that is not in $P/poly$, that problem would necessarily not be in P, thereby proving that $P \neq NP$. Consequently, attempts to settle P vs NP have turned toward seeking a Boolean function mapping n bits to 1 whose associated decision problem is in the NP class and whose minimal circuit requires a super-polynomial number of gates.

Therefore, the roadmap for the proof becomes clear: find a decision problem in NP that is not computable by polynomial circuits. Unfortunately, even under these circumstances, the problem remains extremely difficult. In fact, the obstacles encountered along the way are more than surprising.

[2]It is worth noting that this inclusion is strict. The halting problem for Turing machines and strings defined in unary

Based on Shannon's counting arguments [19], we know that nearly all Boolean functions require a circuit of super-polynomial size (specifically $2^n/n$) for a sufficiently large n; paradoxically, we have been unable to explicitly find even a single one. This fact highlights that the difficulty lies not just in finding a Boolean function whose associated decision problem is in NP and is only computable by super-polynomial circuits, but in finding any explicit functions whatsoever that are "sufficiently complex." This is why, in recent years, there have also been massive efforts focused on finding lower bounds for circuit sizes of certain Boolean functions, regardless of whether their associated problem is in NP or not.

Objectives and Work Plan

The ideal horizon we pursue is to prove that $P \neq NP$ and to do so through the complexity of Boolean circuits—that is, to find a decision problem in NP whose family of Boolean functions is not computable by polynomial circuits. To achieve this, we must be able to prove lower bounds for Boolean functions regarding their circuit size. The primary tool for this purpose will be proposing a complexity measure $\mu : \mathcal{F}_n \to \mathbb{R}$ that orders the set of Boolean functions in the same manner as their minimal circuit complexity, allowing us to more easily prove which functions are "small" and which are "large." Afterward, it would suffice to find a problem in NP whose functions are "large" according to our complexity measure.

This horizon is clearly highly ambitious and serves to guide our steps, but we need to establish simpler milestones along the way to act as checkpoints for our progress. Therefore, the objectives we will pursue, in order, are the following:

1. Understand the proofs of the best lower bounds on the size of Boolean circuits proven to date and present a pedagogical exposition of the contents.
2. Propose our own complexity measures based on the evident or underlying repetitive patterns presented by the truth tables of Boolean functions.
3. Build a software infrastructure that enables the experimental study of these measures and allows making conjectures about their capacity to prove lower bounds for Boolean circuit size.
4. Present the experimental study and either formally prove why the metrics fail to establish lower bounds, or conjecture why they do and propose improvements.

The objectives perfectly outline the roadmap followed throughout this work. Chapter 2 presents the pedagogical exposition of the series of papers that prove the best lower bound found to date, as well as the family of functions for which it is proven. Chapter 3 introduces our own complexity measures, the software infrastructure built, and the experimental study associated with each of them. During the development of these two chapters, certain limitations became evident regarding both the lower bound proofs and the proposed complexity measures. For this reason, Chapter 4 is additionally included to study the limitations and constraints of both approaches, proving results to guide future proof attempts using either method. Finally, Chapter 5 proposes distinct and promising lines of work to continue with the most successful developments of this document.

Conclusions and Future Work

We conclude this exposition by summarizing the conclusions reached after an exhaustive study of different approaches to searching for Boolean functions with super-polynomial minimum circuit sizes, and by proposing the next steps that would be interesting to explore for anyone wishing to extend the contents of this work.

Conclusions

The complexity of the Turing machine computation model has shifted attention toward simpler models, such as Boolean circuits, in attempts to prove that $P \neq NP$. Along this line, a new subproblem has emerged: finding decision problems that are not computable by polynomial-sized circuits with respect to the number of their inputs. This is precisely where the focus has been placed in recent times.

In the search for these functions, there are two main pathways. The first is based on reasoning directly related to the topology of the minimal circuit associated with Boolean functions. For this purpose, the primary available tool is the gate elimination technique, and the best lower bound obtained so far is $3.1n - o(n)$. However, recent work explored in section 4.2 of the previous chapter has highlighted how this technique must be applied if super-polynomial bounds are to be achieved, leaving open the question of how to construct proofs that take these observations into account. The second pathway is based on designing complexity measures that reason about properties presumably linked indirectly to Boolean circuit size. If we were able to construct a complexity measure that is uniquely bounded for decision problems in $P/poly$ and prove that a certain problem is unbounded, we would have found precisely one of those problems whose functions require superpolynomial circuits. Along those lines, the work presented in section 4.1, also from the previous chapter, shows the conditions that any complexity measure must satisfy if it hopes to have guarantees of success.

Following this latter line of thought, we have based our work on defining complexity measures that attempt to exploit the implication between boolean circuit size and the apparent or underlying repetitions in the truth tables of boolean functions. We have concluded that there is a direct relationship between the incompressibility of a truth table and the size of the Boolean circuit that computes it, proving that a formal foundation supports our approach. In this context, the most promising measure has been the one based on the Fourier normal form, which has allowed us to observe a correlation in empirical tests with the minimal circuit size in 5-bit to 1-bit Boolean functions. Furthermore, the measure seems promising because it satisfies the conditions imposed by demostraciones naturales to guarantee success.

Future Work

The experimental results obtained in subsection 3.2.1 seem to capture circuit complexity to some extent, matching even the correlation with circuit size found in some of the best metrics from previous works in this area. Therefore, it appears to be a promising line of inquiry to continue exploring, which also leaves several interesting questions open.

First of all, no argument has been proven —and, in fact, it seems difficult— to determine the maximum value that the complexity measure can reach and which is the "malicious" function for which it is attained. Similarly, the lower bound within each size class appears to be close enough to its true value to suspect that the absolute error could be polynomial with respect to n. Were we able to resolve both points, we could continue testing the result even if the correlation were not perfect. Indeed, if we proved that the highest size class reaches at least an exponential score (for which knowing the maximum value and the malicious function is useful) and demonstrated that the absolute error of the minimum of each size class is polynomial with respect to the size of the minimum circuit, then we would have proven the existence of a function with at least an exponential circuit complexity (those that score the minimum achievable within the highest size class). If, furthermore, we were able to prove that there exists an NP-complete problem with infinitely many boolean functions in that size class, then we would have shown that this problem does not belong to $P/poly$ and, therefore, that $P \neq NP$.

Secondly, the distribution of the mean score value per size class follows a monotonic progression that seems to respect the monotony of the minimal circuit size, and the standard deviation within each class does not appear excessively high (in line with what was mentioned in the previous paragraph). It would be interesting to further explore how we can decrease the standard deviation within the distributions of each class by refining the Fourier normal form with variations, such as the one presented in subsection 3.2.2. Probably the most useful line of work in this regard would be to address a selection criterion for prime implicants during the generation of the minimal DNF from which the FNF is constructed; in our development, this point only served to make the representation unique, but not optimal.

Finally, the fundamental harmonics seem "canonical" in the sense that they truly represent the building blocks from which the rest of the Boolean functions are constructed. However, the choice of Boolean operations used to combine them was limited to $\{\wedge, \vee\}$ by our own design, and it would be interesting to study the behavior of the complexity measure based on another universal gate set. Furthermore, it would be worthwhile to study how the composition of harmonics behaves with respect to the permutation of inputs, since in principle it should preserve the difficulty between Boolean functions within the same equivalence class given by [13].

Bibliografía

"Ese es el asunto: elegir de qué quiere uno arrepentirse en caso de que vengan mal dadas. Y yo de lo que no quiero arrepentirme, bajo ninguna presión, es de haber sido complaciente."
— Cabezas de Cartel

[1] Juan Diego Barrado Daganzo. *Experimental Study P vs NP*. GitHub repository. 2026. URL: https://github.com/JuanDiegoBarrado/Experimental-Study-P-vs-NP.

[2] Eli Ben-Sasson y Swastik Kopparty. «Affine dispersers from subspace polynomials». En: *Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing (STOC '09)*. ACM, 312009, págs. 65-74. DOI: 10.1145/1536414.1536426.

[3] Norbert Blum. «A Boolean function requiring 3n network size». En: *Theoretical Computer Science* 28.3 (1983), págs. 337-345. DOI: 10.1016/0304-3975(83)90029-4.

[4] Stephen A. Cook. «The complexity of theorem-proving procedures». En: *Logic, automata, and computational complexity: The works of Stephen A. Cook*. 2023, págs. 143-152. DOI: 10.1145/3588287.3588297.

[5] Yves Crama y Peter L. Hammer. *Boolean Models and Methods in Mathematics, Computer Science, and Engineering*. Cambridge University Press, 2010.

[6] Evgeny Demenkov y Alexander S. Kulikov. «An Elementary Proof of a $3n - o(n)$ Lower Bound on the Circuit Complexity of Affine Dispersers». En: *Mathematical Foundations of Computer Science 2011*. Ed. por Filip Murlak y Piotr Sankowski. Vol. 6907. Springer, 2011, págs. 256-265. DOI: 10.1007/978-3-642-22993-0_25.

[7] Magnus Gausdal Find, Alexander Golovnev, Edward A. Hirsch y Alexander S. Kulikov. «A Better Than 3n Lower Bound for the Circuit Complexity of an Explicit Function». En: *2016 IEEE 57th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 2016, págs. 89-98. DOI: 10.1109/FOCS.2016.19.

[8] Oded Goldreich, Shafi Goldwasser y Silvio Micali. «How to construct random functions». En: *Journal of the ACM* 33.4 (1986), págs. 792-807. DOI: 10.1145/6490.6503.

[9] Alexander Golovnev, Edward A. Hirsch, Alexander Knop y Alexander S. Kulikov. «On the limits of gate elimination». En: *Journal of Computer and System Sciences* 96 (2018), págs. 107-119. DOI: 10.1016/j.jcss.2018.04.005.

[10] Eduardo Gonzalez-Vaquero, Ismael Rodríguez, Daniel Loscos, Narciso Martí-Oliet y Javier Rodríguez-Laguna. «Approaching Circuit Complexity with Explainable Metrics». En: *Enviado para publicación* (2026).

[11] Adam P. Goucher. *optimal5*. Last modified: 2020. 2019. URL: https://gitlab.com/apgoucher/optimal5.

[12] Boris M. Kloss y Vadim A. Malyshev. «Estimates of the complexity of certain classes of functions». En: *Moscow University Mathematics Bulletin* 20 (1965), págs. 44-51.

[13] Donald Ervin Knuth. *The Art of Computer Programming*. Vol. 4A: *Combinatorial Algorithms, Part 1*. Addison-Wesley, 2011.

[14] Leonid A. Levin. «Universal sequential search problems». En: *Problems of Information Transmission* 9.3 (1973), págs. 265-266.

[15] Jiatu Li y Tianqi Yang. «3,1$n-O(n)$ circuit lower bounds for explicit functions». En: *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing*. ACM, 2022, pág. 14. DOI: 10.1145/3519935.3519976.

[16] Wolfgang J. Paul. «A 2.5 n-lower bound on the combinational complexity of Boolean functions». En: *Proceedings of the Seventh Annual ACM Symposium on Theory of Computing (STOC '75)*. ACM Press, 1975, págs. 27-36. DOI: 10.1145/800116.803750.

[17] Alexander A. Razborov y Steven Rudich. «Natural Proofs». En: *Journal of Computer and System Sciences* 55.1 (1997), págs. 24-35. DOI: 10.1006/jcss.1997.1494.

[18] C. P. Schnorr. «Zwei lineare untere Schranken für die Komplexität Boolescher Funktionen». En: *Computing* 13.2 (1974), págs. 155-171. DOI: 10.1007/BF02246615.

[19] Claude E. Shannon. «The synthesis of two-terminal switching circuits». En: *The Bell System Technical Journal* 28.1 (1949), págs. 59-98.

[20] Michael Sipser. *Introduction to the theory of computation*. Third edition, international edition. Cengage Learning, 2013.

[21] Larry J. Stockmeyer. «On the combinational complexity of certain symmetric Boolean functions». En: *Mathematical Systems Theory* 10.1 (1976), págs. 323-336. DOI: 10.1007/BF01683282.

Apéndice A

Artefactos adicionales

En este apéndice se detallan las tablas de verdad y las propiedades de las 16 puertas lógicas analizadas a lo largo del documento. Asimismo, se incluye el algoritmo basado en programación dinámica empleado para el cálculo del total de clasificaciones.

Tablas de Clasificación y Trivialización

	Tabla de verdad					
Puerta	$(0,0)$	$(0,1)$	$(1,0)$	$(1,1)$	Operación	Clase
p_0	0	0	0	0	0	$\odot$
p_1	0	0	0	1	$x \wedge y$	$\wedge$
p_2	0	0	1	0	$x \wedge \neg y$	$\wedge$
p_3	0	0	1	1	x	$\odot$
p_4	0	1	0	0	$\neg x \wedge y$	$\wedge$
p_5	0	1	0	1	y	$\odot$
p_6	0	1	1	0	$x \oplus y$	$\oplus$
p_7	0	1	1	1	$x \vee y$	$\wedge$
p_8	1	0	0	0	$\neg(x \vee y)$	$\wedge$
p_9	1	0	0	1	$\neg(x \oplus y)$	$\oplus$
p_{10}	1	0	1	0	$\neg y$	$\odot$
p_{11}	1	0	1	1	$x \vee \neg y$	$\wedge$
p_{12}	1	1	0	0	$\neg x$	$\odot$
p_{13}	1	1	0	1	$\neg x \vee y$	$\wedge$
p_{14}	1	1	1	0	$\neg(x \wedge y)$	$\wedge$
p_{15}	1	1	1	1	1	$\odot$

Tabla A.1: Clasificación de las 16 puertas lógicas en las clases $\odot$-puertas, $\oplus$-puertas y $\wedge$-puertas.

	x		y	
Puerta	0	1	0	1
p_0	0	0	0	0
p_1	0	y	0	x
p_2	0	y	x	0
p_3	0	1	x	x
p_4	y	0	0	$\neg x$
p_5	y	y	0	1
p_6	y	$\neg y$	x	$\neg x$
p_7	y	1	x	1
p_8	$\neg y$	0	$\neg x$	0
p_9	$\neg y$	y	$\neg x$	x
p_{10}	$\neg y$	$\neg y$	1	0
p_{11}	1	$\neg y$	1	x
p_{12}	1	0	$\neg x$	$\neg x$
p_{13}	1	y	$\neg x$	1
p_{14}	1	$\neg y$	1	$\neg x$
p_{15}	1	1	1	1

Tabla A.2: Transformación de cada una de las 16 puertas en una puerta de $\odot$-puertas según la sustitución constante hecha sobre sus entradas.

	x		y	
Puerta	$y \oplus 0$	$y \oplus 1$	$x \oplus 0$	$x \oplus 1$
p_6	0	1	0	1
p_9	1	0	1	0

Tabla A.3: Trivialización de las $\oplus$-puertas según la sustitución afín simple hecha sobre sus entradas.

Implementación Algorítmica

Algoritmo 1 Cómputo del total de clasificaciones distintas de longitudes desde 0 hasta n mediante Programación Dinámica

Require: Una tabla de verdad *sequence* de tamaño n
Ensure: Número total de clasificaciones (*total*)

```
1:  function NUM_CLASSIFICATIONS(sequence)
2:      n ← longitud(sequence)
3:      k ← n
4:      Inicializar array dp[0...n][0...k] con 0
5:      Inicializar array terminan[0...n][0...k][0...1] con 0
6:      for i = 0 to n do
7:          dp[i][0] ← 1
8:      end for
9:      for i = 1 to n do
10:         c ← sequence[i − 1]
11:         neg_c ← 1 − c
12:         for j = 1 to mín(i, k) do
13:             dp[i][j] ← dp[i − 1][j] + (dp[i − 1][j − 1] − terminan[i − 1][j][c])
14:             terminan[i][j][c] ← terminan[i − 1][j][c] + (dp[i − 1][j − 1] − terminan[i − 1][j][c])
15:             terminan[i][j][neg_c] ← dp[i][j] − terminan[i][j][c]
16:         end for
17:     end for
18:     total ← 0
19:     for j = 0 to k do
20:         total ← total + dp[n][j]
21:     end for
22:     return total
23: end function
```